

# Proyecto de programación: "Aprendizaje por refuerzo aplicado a Enduro-V0"

## Universidad Internacional de Valencia (VIU)

Máster Universitario en Inteligencia Artificial

Asignatura: Aprendizaje por Refuerzo (08MIAR)

### Integrantes del Grupo 17:

- Williams Merida Parard - Integrante 1
- Ángel María Encinas Moreno - Integrante 2
- Marina Riquelme Morell - Integrante 3
- María Elena Bain - Integrante 4



---

### LINK GitHub

[https://github.com/Williams-MP/VIU\\_Apr\\_por\\_Refuerzo\\_Grupo\\_17\\_2025\\_2026.git](https://github.com/Williams-MP/VIU_Apr_por_Refuerzo_Grupo_17_2025_2026.git)

---

## Historial de modificaciones respecto a la entrega anterior

Esta entrega incorpora las correcciones solicitadas en el feedback recibido, centrado en la trazabilidad del entrenamiento y en la disponibilidad de las evidencias (gráficas y correspondencia de pesos/logs) directamente en el documento entregado, no solo en el notebook ejecutado en local.

### 1. Gráficas de evolución del entrenamiento incluidas en el PDF/notebook (Sección 4.9)

*Feedback: "Si las gráficas con la evolución de las métricas desde el inicio del entrenamiento están disponibles en el notebook y no en el PDF, por favor inclúidlas en el PDF."*

Se añade la generación explícita de gráficas por propuesta que muestran la evolución completa del entrenamiento, coloreando cada tramo ("tanda") según los hiperparámetros usados y marcando visualmente qué tramos cuentan con log disponible y cuáles no. El objetivo es que estas gráficas queden persistidas como salida de celda dentro del propio documento entregado.

### 2. Tabla de correspondencia modelo-log-directorio de pesos-fase de entrenamiento (Sección 4.12)

*Feedback: "Respecto a los pesos del entrenamiento, o me indicáis en un anexo los contenidos de los logs, o [...] indicad en una tabla la correspondencia de cada modelo, log, directorio de los pesos y fase del entrenamiento en la que fueron realizados (ej. desde cero, se han sobrescrito, etc.)."*

Se añade una tabla generada automáticamente (4.12.1) que documenta, para cada una de las 8 propuestas: el directorio de pesos, el fichero de log disponible, y las distintas tandas de entrenamiento realizadas (rango de steps, hiperparámetros de esa tanda, y si el log de esa tanda está disponible, fue sobrescrito o se reinició). Se explicita además por qué algunos logs no cubren todo el histórico: `FileLogger` de keras-rl2 sobrescribe el fichero de log en cada nueva llamada a `dqn.fit()`, de modo que solo queda registrada la última tanda ejecutada, aunque los checkpoints de pesos de tandas anteriores sí se conservan.

### 3. Listado de checkpoints intermedios disponibles (Sección 4.12.2)

Se añade una tabla con todos los checkpoints guardados cada 100.000 steps por propuesta, junto con el fragmento de código necesario para cargar cualquiera de ellos y reproducir su evaluación, de forma que la evolución del modelo pueda verificarse en cualquier punto del entrenamiento y no solo en el checkpoint final.

### 4. Estructura de la entrega y reproducibilidad (Sección 4.13)

Se documenta explícitamente el árbol de carpetas del ZIP de entrega (checkpoints por propuesta + vídeo de demostración) y los pasos para ejecutar el notebook en local, de modo que un evaluador externo pueda ubicar y verificar cada pieza de evidencia sin conocer previamente cómo está organizado el proyecto.

### 5. Corrección de numeración e índice

Se corrige la numeración de subsecciones (Fases III y IV) para que los anclas del índice coincidan con el contenido real del documento.

---

## Índice

## Fase I. Infraestructura de despliegue y preparación del entorno de ejecución

- [1.1. Virtualización y configuración del entorno aislado local.](#)
- [1.2. Localización y enrutamiento del espacio de trabajo.](#)
- [1.3. Orquestación del almacenamiento persistente.](#)
- [1.4. Gestión de la pila de software y aprovisionamiento del ecosistema de dependencias.](#)

## Fase II. Especificaciones operativas y criterios de validación de la actividad

- **2.1.** Directrices del proyecto práctico.

## Fase III. Bloque de desarrollo experimental y desarrollo analítico

- **3.1.** Importación de módulos científicos.
- **3.2.** Configuración base y diseño del pipeline de preprocesamiento (AtariProcessor).
- **3.3.** Diseño y topología jerárquica de redes neuronales convolucionales.
- **3.4.** Infraestructura central de evaluación, registro de agentes y control de interrupciones.

## Fase IV. Bloque de desarrollo experimental: exploración y variantes

- **4.** Introducción: análisis de la dispersión de la señal de recompensa.
- **4.1.** Propuesta 1: DQN vanilla, EpsGreedyQPolicy fijo (baseline).
- **4.2.** Propuesta 2: política de exploración dinámico-lineal.
- **4.3.** Propuesta 3: política probabilística estocástica (Boltzmann).
- **4.4.** Propuesta 4: Double DQN.
- **4.5.** Propuesta 5: Dueling DQN.
- **4.6.** Propuesta 6: Double Dueling DQN.
- **4.7.** Propuesta 7: Double DQN con annealing de epsilon.
- **4.8.** Propuesta 8: Dueling DQN con annealing de epsilon.
- **4.9.** Extracción de métricas de rendimiento y representación gráfica.
- **4.10.** Evaluación comparativa multivariable y conclusiones de modelado.
- **4.11.** Justificación de los parámetros seleccionados y resultados obtenidos.

## Fase V. Referencias y anexos

- **Bibliografía.**
- **Anexo I. Evidencias de entrenamiento.**

## Fase I. Infraestructura de despliegue y preparación del entorno de ejecución

### 1.1. Virtualización y configuración del entorno aislado local

---

*En este apartado preliminar se formalizan los fundamentos técnicos para el aprovisionamiento de un entorno de ejecución local aislado. Se detalla la justificación del aislamiento de dependencias mediante gestores de entornos*

virtuales y los prerequisites del sistema operativo necesarios para la compilación nativa de las herramientas de simulación de Atari.

#### o Desarrollo matemático y fundamentos

La reproducibilidad de experimentos en aprendizaje por refuerzo profundo exige mitigar el problema de la deriva de dependencias (*dependency drift*). Un entorno computacional aislado se puede modelar formalmente como un subconjunto cerrado de paquetes y variables de entorno  $\mathcal{P}_{\text{env}}$  dentro del espacio global del sistema operativo  $\mathcal{P}_{\text{global}}$ . Para garantizar la estabilidad del pipeline, se impone una restricción de nulidad en la intersección con versiones en conflicto del sistema anfitrión:

$$\mathcal{P}_{\text{env}} \cap \mathcal{P}_{\text{global} \setminus \mathcal{P}_{\text{env}}} = \emptyset$$

Esta condición asegura que la resolución del árbol de dependencias sea determinista. Asimismo, el núcleo del simulador ALE (*Arcade Learning Environment*) interactúa con abstracciones de bajo nivel que requieren compilar código nativo. En sistemas Windows, la correcta correspondencia entre la lógica del agente y la función de transición del entorno  $f : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$  depende de la presencia de un compilador de C++ compatible (como las herramientas de compilación de Visual Studio), encargado de transformar los binarios del emulador sin alterar el determinismo de los marcos de trabajo superiores.

#### o Directrices de implementación

A continuación, se describen los pasos aplicados para la orquestación e inicialización del espacio de trabajo local.

- Se recurre a la distribución Anaconda para instanciar el entorno aislado mediante el comando de consola `conda create`, parametrizando de forma estricta la versión de Python en 3.8 para salvaguardar la compatibilidad con el ecosistema clásico de Keras.
- Se realiza la activación del espacio virtual en la terminal antes de efectuar cualquier inyección de paquetes, garantizando que el intrapasador de comandos apunte exclusivamente al directorio aislado.
- Se instalan las herramientas de compilación de C++ en el sistema operativo anfitrión o, en su defecto, se configura un subsistema de Linux para Windows (WSL2) que permita dar soporte a la compilación nativa del módulo `atari-py`.
- Se incorporan mediante el gestor del entorno las herramientas esenciales de control de versiones y el servidor web interactivo Jupyter, asegurando el lienzo técnico sobre el cual se ejecutará el cuaderno de la práctica.

#### **Configuración e inicialización del entorno virtual aislado**

```
In [2]: ## Creación de un entorno virtual aislado con una versión específica de Python.
## Se define el parámetro de entorno fijando la versión en 3.8 para garantizar
## la compatibilidad estricta con el ecosistema de dependencias de Keras y Tens
# conda create --name miar_rl python=3.8

## Activación del entorno virtual aislado recién construido.
```

```
## Este paso desvía el intrapasador de comandos hacia el directorio del entorno  
## evitando colisiones numéricas con las bibliotecas del sistema anfitrión.  
# conda activate miar_rl  
  
## Desplazamiento del puntero de la terminal hacia el directorio de trabajo del  
## Modifique la cadena de texto literal por la ruta absoluta de su espacio de a  
# cd "PATH_TO_FOLDER"  
  
## Instalación del sistema de control de versiones Git dentro del entorno activ  
## Herramienta indispensable para el enlazado dinámico y descarga del núcleo at  
# conda install git  
  
## Instalación del gestor de cuadernos interactivos Jupyter de forma local.  
## Proporciona la infraestructura base para la ejecución de las celdas de la pr  
# pip install jupyter
```

### **Activación del entorno de desarrollo Jupyter Notebook**

```
In [3]: ## Inicialización del servidor web local y apertura automática del cuaderno.  
## Lanza el entorno de desarrollo interactivo en el navegador predeterminado de  
# jupyter-notebook
```

### **1.1.5. Resultados y estado final**

La ejecución de los comandos de esta sección concluye la fase de virtualización e infraestructura básica en la máquina local. Al finalizar este bloque, el espacio de trabajo presenta las siguientes características:

- Se dispone de un entorno virtual aislado bajo la versión 3.8 de Python, lo que inmuniza el desarrollo frente a colisiones con las bibliotecas del sistema anfitrión y unifica la resolución de dependencias de Keras.
- Se cuenta con el soporte de bajo nivel del sistema de control de versiones Git, indispensable para la descarga y el enlazado dinámico de los componentes del emulador de Atari, y con el servidor interactivo de Jupyter en plena ejecución local.

Este lienzo computacional limpio y estandarizado garantiza que los experimentos posteriores sean plenamente reproducibles, sentando las bases operativas para realizar la gestión de rutas y persistencia que se detalla en el próximo apartado.

---

## **1.2. Localización y enrutamiento del espacio de trabajo (entornos locales frente a Google Colab)**

---

*En este apartado se describe el mecanismo automatizado para la detección del contexto operativo de ejecución. Mediante el control secuencial de excepciones, el sistema discrimina dinámicamente entre una infraestructura local y un entorno distribuido en la nube, adaptando las variables de enrutamiento del núcleo.*

## ○ Desarrollo matemático y lógica de control

La abstracción del espacio de trabajo requiere un operador lógico capaz de mapear el espacio de rutas del sistema de archivos en función de la plataforma anfitriona. Se define un espacio de estados de infraestructura  $\Omega = \{\text{Local}, \text{Colab}\}$ . El flujo de control evalúa la presencia de la biblioteca específica del entorno distribuido a través de una función indicadora basada en el éxito de la importación:

$$\mathbb{I}_{\text{Colab}} = \begin{cases} 1 & \text{si } \exists \text{ google.colab.drive} \\ 0 & \text{en otro caso} \end{cases}$$

Esta variable booleana, parametrizada en el código mediante la bandera `IN_COLAB`, gobierna la bifurcación del flujo del programa. Si  $\mathbb{I}_{\text{Colab}} = 1$ , la ruta base del sistema se redirige hacia el punto de montaje virtual de la nube, modificando el directorio de trabajo activo del proceso ejecutor de Python mediante una operación de cambio de directorio. De este modo, la localización de los elementos del programa se independiza de la máquina física donde se inicialice el hilo de ejecución.

## ○ Directrices de implementación

A continuación, se describen los criterios algorítmicos aplicados para la automatización del enrutamiento.

- Se encapsula la importación del módulo de Google dentro de una estructura de control de excepciones de tipo try-except para interceptar fallos en entornos locales sin detener el programa.
- Se asigna el valor lógico de la bandera de control en consonancia con el resultado de la importación de la dependencia de la nube.
- Se ejecuta un bloque condicional que bifurca la salida por consola e informa explícitamente al operador sobre la plataforma en la que se ha inicializado el motor de ejecución.

## ***Detección automatizada de la plataforma y enrutamiento condicional***

### Instrucciones para ejecutar en local (sin Google Colab)

Si ejecuta este notebook fuera de Google Colab:

1. Coloque el notebook en la misma carpeta que las subcarpetas de pesos ( `p1_eps_fijo/` , `p2_annealed/` , ..., `p8_dueling_dqn_annealed/` )
2. `IN_COLAB` se detectará automáticamente como `False`
3. El notebook usará la carpeta actual como raíz — no necesita Drive
4. Las celdas de entrenamiento están desactivadas ( `RUN_TRAINING = False` ) y no necesitan ejecutarse para evaluar los modelos
5. **Modelo final: P5 (Dueling DQN)** archivo:  
`p5_dueling_dqn/dqn_Enduro-v0_p5_weights_2300000.h5f`

```
In [4]: # Definición de la ruta base destinada al punto de montaje del volumen virtual.
mount='/content/gdrive'

# Especificación del directorio raíz jerárquico dentro de Google Drive.
# Almacena de forma persistente los archivos de registro y puntos de control.
drive_root = mount + "/My Drive/VIU_Apr_por_Refuerzo_Grupo_17_2025_2026/checkpoi

# Bloque de control de excepciones para la discriminación dinámica del entorno.
# Evalúa el éxito de la importación del módulo de la nube para aislar el context
try:
    from google.colab import drive
    IN_COLAB=True
except:
    IN_COLAB=False
```

### 1.2.5. Resultados y estado final

La resolución de esta lógica de control discrimina el entorno de ejecución de forma preventiva. Al concluir este bloque, el espacio operativo presenta las siguientes características:

- Se ha inicializado la bandera lógica encargada de bifurcar el flujo del programa, lo que permite que el cuaderno identifique de manera automática si se ejecuta en una infraestructura local o en la nube.
- Se han parametrizado las cadenas de texto correspondientes a las rutas virtuales de almacenamiento secundario, dejando el sistema preparado para realizar la vinculación de los directorios sin riesgo de interrumpir el hilo de proceso.

Este estado intermedio asegura el aislamiento de la configuración de la plataforma, permitiendo avanzar hacia el montaje físico y la estructuración del árbol jerárquico de carpetas en el próximo apartado.

---

## 1.3. Orquestación del almacenamiento persistente y mitigación de la volatilidad del hilo de ejecución

---

*En este apartado se detalla el proceso técnico para el enlace físico de la unidad de almacenamiento secundario y la estructuración jerárquica del sistema de archivos. Se analiza la partición del espacio en directorios aislados destinados a garantizar la persistencia de los pesos neuronales y registros históricos de cada una de las propuestas experimentales.*

### ○ Desarrollo matemático y formalización

El proceso de optimización de un agente de aprendizaje por refuerzo profundo a lo largo de un horizonte temporal extenso es propenso a sufrir interrupciones debidas a fallos de hardware, desconexiones de red o la recolección estocástica de recursos en entornos de computación efímeros. Para modelar la persistencia de datos, denotamos el estado de

los parámetros de la red en el paso temporal  $t$  como un vector de pesos  $\theta_t \in \Theta$ . Si el hilo de ejecución experimenta una desconexión voluntaria o involuntaria, la trayectoria de aprendizaje alojada en la memoria volátil se destruye.

Para mitigar esta vulnerabilidad, se define un operador de salvaguarda periódica  $\mathcal{G} : \Theta \times \mathbb{N} \rightarrow \mathcal{S}_{\text{persistente}}$  que escribe el estado del agente en un volumen secundario estable a intervalos fijos. Debido a que el proyecto contempla múltiples configuraciones algorítmicas concurrentes, el espacio global de almacenamiento persistente se particiona en una familia de subconjuntos cerrados y disjuntos:

$$\mathcal{M} = \{\mathcal{M}_1, \mathcal{M}_2, \dots, \mathcal{M}_n\} \quad \text{tal que} \quad \mathcal{M}_i \cap \mathcal{M}_j = \emptyset \quad \forall i \neq j$$

Donde cada subdirectorio  $\mathcal{M}_i$  actúa como un contenedor aislado y dedicado exclusivamente a los puntos de control (*checkpoints*) y archivos de registro de una propuesta experimental concreta. Esta segregación impide la sobreescritura cruzada de los tensores de pesos y asegura que cada variante del algoritmo pueda reanudar de manera independiente su proceso de descenso de gradiente desde el último estado válido almacenado.

#### o Directrices de implementación

A continuación, se describen los criterios que gobiernan la estructuración física del almacenamiento en el script del proyecto.

- Se invoca el método de vinculación del almacenamiento en la nube únicamente si la bandera de contexto previamente evaluada confirma que la ejecución se desarrolla en una infraestructura distribuida.
- Se modifica el directorio operativo activo del núcleo de Python hacia la ruta raíz validada, asegurando que todas las operaciones de lectura y escritura relativas utilicen rutas absolutas consistentes.
- Se definen mediante variables explícitamente declaradas las rutas destinadas a las ocho propuestas analizadas, asociando cada identificador a una carpeta unívoca dentro del árbol de directorios del proyecto.
- Se ejecuta una estructura iterativa automatizada encargada de verificar la existencia de cada carpeta física y crearlas de forma recursiva en caso de ausencia para evitar excepciones de entrada o salida.

### **Vinculación del volumen virtual de almacenamiento y estructuración jerárquica de directorios**

```
In [5]: import os

# Bifurcación algorítmica para la gestión del sistema de archivos según el entorno
# Realiza el montaje físico de la unidad virtual solo si el contexto es distribuido
if IN_COLAB:
    print("We're running Colab")
    # Vinculación del almacenamiento en la nube para asegurar la persistencia.
    drive.mount(mount)
    # Creación del directorio raíz en caso de que no exista previamente.
```

```

    os.makedirs(drive_root, exist_ok=True)
    print("\nColab: Changing directory to", drive_root)
    # Modificación del directorio de trabajo del núcleo o kernel del proceso.
    os.chdir(drive_root)
else:
    print("No estamos en Colab (IN_COLAB=False); se usará el directorio actual d

# Recuperación y validación por consola de la ruta de trabajo activa.
WORK_DIR = os.getcwd()
print("\nDirectorio de trabajo actual:", WORK_DIR)
print("Archivos en el directorio:")
print(os.listdir(WORK_DIR))

# Definición de las rutas absolutas para los resultados de cada propuesta experi
RESULTS_DIR = WORK_DIR
WEIGHTS_DIR_P1 = os.path.join(RESULTS_DIR, 'p1_eps_fijo')
WEIGHTS_DIR_P2 = os.path.join(RESULTS_DIR, 'p2_annealed')
WEIGHTS_DIR_P3 = os.path.join(RESULTS_DIR, 'p3_boltzmann')
WEIGHTS_DIR_P4 = os.path.join(RESULTS_DIR, 'p4_double_dqn')
WEIGHTS_DIR_P5 = os.path.join(RESULTS_DIR, 'p5_dueling_dqn')
WEIGHTS_DIR_P6 = os.path.join(RESULTS_DIR, 'p6_double_dueling')
WEIGHTS_DIR_P7 = os.path.join(RESULTS_DIR, 'p7_double_dqn_annealed')
WEIGHTS_DIR_P8 = os.path.join(RESULTS_DIR, 'p8_dueling_dqn_annealed')

# Bucle estructurado para la creación sistemática de carpetas de almacenamiento
for d in [RESULTS_DIR, WEIGHTS_DIR_P1, WEIGHTS_DIR_P2, WEIGHTS_DIR_P3,
          WEIGHTS_DIR_P4, WEIGHTS_DIR_P5, WEIGHTS_DIR_P6, WEIGHTS_DIR_P7, WEIGHTS_DIR_P8]:
    os.makedirs(d, exist_ok=True)

# Impresión en consola de las rutas preparadas para la verificación visual del u
print("\nCarpetas de checkpoints preparadas (en Drive):")
print(" -", WEIGHTS_DIR_P1)
print(" -", WEIGHTS_DIR_P2)
print(" -", WEIGHTS_DIR_P3)
print(" -", WEIGHTS_DIR_P4)
print(" -", WEIGHTS_DIR_P5)
print(" -", WEIGHTS_DIR_P6)
print(" -", WEIGHTS_DIR_P7)
print(" -", WEIGHTS_DIR_P8)

```

We're running Colab

Drive already mounted at /content/gdrive; to attempt to forcibly remount, call `drive.mount("/content/gdrive", force_remount=True)`.

Colab: Changing directory to /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerzo\_Grupo\_17\_2025\_2026/checkpoints

Directorio de trabajo actual: /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerzo\_Grupo\_17\_2025\_2026/checkpoints

Archivos en el directorio:

```
['resultados_enduro_dqn', 'p1_eps_fijo', 'p2_annealed', 'p3_boltzmann', 'p4_double_dqn', 'p5_dueling_dqn', 'p6_double_dueling', 'p7_double_dqn_annealed', 'p8_dueling_dqn_annealed', 'video_demo_p5', 'Entrega_Grupo_17.zip']
```

Carpetas de checkpoints preparadas (en Drive):

- /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerzo\_Grupo\_17\_2025\_2026/checkpoints/p1\_eps\_fijo
- /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerzo\_Grupo\_17\_2025\_2026/checkpoints/p2\_annealed
- /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerzo\_Grupo\_17\_2025\_2026/checkpoints/p3\_boltzmann
- /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerzo\_Grupo\_17\_2025\_2026/checkpoints/p4\_double\_dqn
- /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerzo\_Grupo\_17\_2025\_2026/checkpoints/p5\_dueling\_dqn
- /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerzo\_Grupo\_17\_2025\_2026/checkpoints/p6\_double\_dueling
- /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerzo\_Grupo\_17\_2025\_2026/checkpoints/p7\_double\_dqn\_annealed
- /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerzo\_Grupo\_17\_2025\_2026/checkpoints/p8\_dueling\_dqn\_annealed

### 1.3.5. Resultados y estado final

La ejecución de este bloque consolida la infraestructura física del sistema de archivos. Al finalizar esta etapa, el espacio operativo presenta las siguientes características:

- Se ha establecido una matriz jerárquica de ocho directorios diferenciados en el almacenamiento secundario, asegurando que los pesos de la red neuronal y los registros de cada propuesta experimental subsistan de forma independiente a la volatilidad del hilo de ejecución.
- El entorno cuenta con rutas absolutas validadas por el intérprete de comandos, lo que automatiza los flujos de lectura y escritura sin riesgo de provocar colisiones o pérdidas accidentales de datos entre las distintas variantes del algoritmo.

Este estado de persistencia proporciona un lienzo tecnológico protegido, permitiendo avanzar hacia la gestión de la pila de software y el aprovisionamiento de dependencias que se detallará a continuación.

---

## 1.4. Gestión de la pila de software y aprovisionamiento del ecosistema de dependencias

---

*En este apartado se formaliza la consistencia del entorno de ejecución mediante la fijación estricta de las versiones de la pila de software. Se detalla el pipeline de instalación condicional diseñado para resolver las incompatibilidades operativas entre el emulador de Atari, el marco de aprendizaje por refuerzo y el motor de ejecución de tensores.*

#### ◦ Desarrollo matemático y consistencia del entorno

La construcción de un sistema de aprendizaje por refuerzo profundo es altamente sensible a la mutabilidad de las interfaces de programación de aplicaciones (API) de las bibliotecas subyacentes. Un entorno de ejecución se puede definir matemáticamente como un vector de estado de software  $\mathbf{V} = [v_1, v_2, \dots, v_m]^T$ , donde cada componente  $v_i$  representa la versión específica de un paquete de cómputo. Denotamos la función de transición del simulador como  $\mathcal{T} : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$  y el operador de actualización de la red neuronal como  $\mathcal{U}(\theta)$ . La compatibilidad binaria e interna del sistema exige que las firmas de los tipos de datos de los tensores se mantengan invariables, lo que formaliza una restricción de estabilidad:

$$\Psi(\mathbf{V}) = \mathbf{1}$$

Donde  $\Psi$  es una función de evaluación de consistencia. Si se altera un elemento del vector (por ejemplo, una actualización menor en la biblioteca del simulador o en el backend de álgebra lineal), la función colapsa a cero, provocando errores en el enlazado dinámico de los ejecutables de C++ del emulador o desajustes en la asignación de memoria de los lotes de experiencia. Para inmunizar el proyecto ante este problema, se implementa un mecanismo de aprovisionamiento dirigido que fija de forma determinista los componentes críticos de  $\mathbf{V}$ , bloqueando la pila de abstracción técnica en un punto de operación estable y reproducible.

#### ◦ Directrices de implementación

A continuación, se describen las directrices que gobiernan la instalación automatizada de los paquetes de software.

- Se utiliza la bandera condicional del entorno para segregar el flujo de instalación, aplicando un conjunto de dependencias optimizado para el aprovisionamiento en la nube o un despliegue nativo en local.
- Se realiza la descarga dirigida del emulador de Atari recurriendo a un repositorio especializado que proporciona los binarios precompilados necesarios para evitar fallos de enlazado en plataformas modernas.
- Se fijan los núcleos analíticos del proyecto determinando el uso exclusivo de las versiones clásicas de la biblioteca de simulación y de la infraestructura de agentes para preservar la coherencia en las llamadas a los métodos del proceso de entrenamiento.
- Se incorporan herramientas de procesamiento de imágenes y manipulación de estructuras numéricas alineadas con el gestor de memoria, asegurando que el flujo de datos no sufra degradación por diferencias en el tipado de los arreglos.

## Control condicional e instalación dirigida de la pila de dependencias

```
In [6]: if IN_COLAB:
        %pip install tensorflow
        %pip install tf-keras
        # %pip install tensorflow==2.18.0
        # %pip install tf-keras==2.18.0
        %pip install gym==0.17.3
        %pip install git+https://github.com/Kojoley/atari-py.git
        %pip install keras-rl2==1.0.5
        %pip install --upgrade ml_dtypes
    else:
        %pip install gym==0.17.3
        %pip install git+https://github.com/Kojoley/atari-py.git
        %pip install pygame==1.5.0
        %pip install h5py==3.1.0
        %pip install Pillow==9.5.0
        %pip install keras-rl2==1.0.5
        %pip install Keras==2.2.4
        %pip install tensorflow==2.5.3
        %pip install torch==2.0.1
        %pip install agents==1.4.0
```

Requirement already satisfied: tensorflow in /usr/local/lib/python3.11/dist-packages (2.18.0)

Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (1.4.0)

Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (1.6.3)

Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (25.2.10)

Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (0.6.0)

Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (0.2.0)

Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (18.1.1)

Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (3.4.0)

Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packages (from tensorflow) (24.2)

Requirement already satisfied: protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<6.0.0dev,>=3.20.3 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (5.29.5)

Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (2.32.3)

Requirement already satisfied: setuptools in /usr/local/lib/python3.11/dist-packages (from tensorflow) (75.2.0)

Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (1.17.0)

Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (3.1.0)

Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (4.14.1)

Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (1.17.2)

Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (1.73.1)

Requirement already satisfied: tensorboard<2.19,>=2.18 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (2.18.0)

Requirement already satisfied: keras>=3.5.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (3.8.0)

Requirement already satisfied: numpy<2.1.0,>=1.26.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (2.0.2)

Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (3.14.0)

Collecting ml-dtypes<0.5.0,>=0.4.0 (from tensorflow)

  Downloading ml\_dtypes-0.4.1-cp311-cp311-manylinux\_2\_17\_x86\_64.manylinux2014\_x86\_64.whl.metadata (20 kB)

Requirement already satisfied: tensorflow-io-gcs-filesystem>=0.23.1 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (0.37.1)

Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.11/dist-packages (from astunparse>=1.6.0->tensorflow) (0.45.1)

Requirement already satisfied: rich in /usr/local/lib/python3.11/dist-packages (from keras>=3.5.0->tensorflow) (13.9.4)

Requirement already satisfied: namex in /usr/local/lib/python3.11/dist-packages (from keras>=3.5.0->tensorflow) (0.1.0)

Requirement already satisfied: optree in /usr/local/lib/python3.11/dist-packages (from keras>=3.5.0->tensorflow) (0.16.0)

Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.4.2)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.10)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.21.0->tensorflow) (2.4.0)  
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.21.0->tensorflow) (2025.7.14)  
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.11/dist-packages (from tensorboard<2.19,>=2.18->tensorflow) (3.8.2)  
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.11/dist-packages (from tensorboard<2.19,>=2.18->tensorflow) (0.7.2)  
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from tensorboard<2.19,>=2.18->tensorflow) (3.1.3)  
Requirement already satisfied: MarkupSafe>=2.1.1 in /usr/local/lib/python3.11/dist-packages (from werkzeug>=1.0.1->tensorboard<2.19,>=2.18->tensorflow) (3.0.2)  
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.11/dist-packages (from rich->keras>=3.5.0->tensorflow) (3.0.0)  
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.11/dist-packages (from rich->keras>=3.5.0->tensorflow) (2.19.2)  
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.11/dist-packages (from markdown-it-py>=2.2.0->rich->keras>=3.5.0->tensorflow) (0.1.2)  
Downloading ml\_dtypes-0.4.1-cp311-cp311-manylinux\_2\_17\_x86\_64.manylinux2014\_x86\_64.whl (2.2 MB)

---

2.2/2.2 MB 42.0 MB/s eta 0:00:00a 0:00:01

Installing collected packages: ml-dtypes

Attempting uninstall: ml-dtypes

Found existing installation: ml\_dtypes 0.5.4

Uninstalling ml\_dtypes-0.5.4:

Successfully uninstalled ml\_dtypes-0.5.4

Successfully installed ml-dtypes-0.4.1

Requirement already satisfied: tf-keras in /usr/local/lib/python3.11/dist-packages (2.18.0)

Requirement already satisfied: tensorflow<2.19,>=2.18 in /usr/local/lib/python3.11/dist-packages (from tf-keras) (2.18.0)

Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.19,>=2.18->tf-keras) (1.4.0)

Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.19,>=2.18->tf-keras) (1.6.3)

Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.19,>=2.18->tf-keras) (25.2.10)

Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.19,>=2.18->tf-keras) (0.6.0)

Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.19,>=2.18->tf-keras) (0.2.0)

Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.19,>=2.18->tf-keras) (18.1.1)

Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.19,>=2.18->tf-keras) (3.4.0)

Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.19,>=2.18->tf-keras) (24.2)

Requirement already satisfied: protobuf!=4.21.0,!4.21.1,!4.21.2,!4.21.3,!4.21.4,!4.21.5,<6.0.0dev,>=3.20.3 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.19,>=2.18->tf-keras) (5.29.5)

Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.19,>=2.18->tf-keras) (2.32.3)

Requirement already satisfied: setuptools in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.19,>=2.18->tf-keras) (75.2.0)

Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.19,>=2.18->tf-keras) (1.17.0)

Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.19,>=2.18->tf-keras) (3.1.0)

Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/python

3.11/dist-packages (from tensorflow<2.19,>=2.18->tf-keras) (4.14.1)  
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.19,>=2.18->tf-keras) (1.17.2)  
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.19,>=2.18->tf-keras) (1.73.1)  
Requirement already satisfied: tensorboard<2.19,>=2.18 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.19,>=2.18->tf-keras) (2.18.0)  
Requirement already satisfied: keras>=3.5.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.19,>=2.18->tf-keras) (3.8.0)  
Requirement already satisfied: numpy<2.1.0,>=1.26.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.19,>=2.18->tf-keras) (2.0.2)  
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.19,>=2.18->tf-keras) (3.14.0)  
Requirement already satisfied: ml-dtypes<0.5.0,>=0.4.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.19,>=2.18->tf-keras) (0.4.1)  
Requirement already satisfied: tensorflow-io-gcs-filesystem>=0.23.1 in /usr/local/lib/python3.11/dist-packages (from tensorflow<2.19,>=2.18->tf-keras) (0.37.1)  
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.11/dist-packages (from astunparse>=1.6.0->tensorflow<2.19,>=2.18->tf-keras) (0.45.1)  
Requirement already satisfied: rich in /usr/local/lib/python3.11/dist-packages (from keras>=3.5.0->tensorflow<2.19,>=2.18->tf-keras) (13.9.4)  
Requirement already satisfied: namex in /usr/local/lib/python3.11/dist-packages (from keras>=3.5.0->tensorflow<2.19,>=2.18->tf-keras) (0.1.0)  
Requirement already satisfied: optree in /usr/local/lib/python3.11/dist-packages (from keras>=3.5.0->tensorflow<2.19,>=2.18->tf-keras) (0.16.0)  
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.21.0->tensorflow<2.19,>=2.18->tf-keras) (3.4.2)  
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.21.0->tensorflow<2.19,>=2.18->tf-keras) (3.10)  
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.21.0->tensorflow<2.19,>=2.18->tf-keras) (2.4.0)  
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.21.0->tensorflow<2.19,>=2.18->tf-keras) (2025.7.14)  
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.11/dist-packages (from tensorboard<2.19,>=2.18->tensorflow<2.19,>=2.18->tf-keras) (3.8.2)  
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.11/dist-packages (from tensorboard<2.19,>=2.18->tensorflow<2.19,>=2.18->tf-keras) (0.7.2)  
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from tensorboard<2.19,>=2.18->tensorflow<2.19,>=2.18->tf-keras) (3.1.3)  
Requirement already satisfied: MarkupSafe>=2.1.1 in /usr/local/lib/python3.11/dist-packages (from werkzeug>=1.0.1->tensorboard<2.19,>=2.18->tensorflow<2.19,>=2.18->tf-keras) (3.0.2)  
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.11/dist-packages (from rich->keras>=3.5.0->tensorflow<2.19,>=2.18->tf-keras) (3.0.0)  
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.11/dist-packages (from rich->keras>=3.5.0->tensorflow<2.19,>=2.18->tf-keras) (2.19.2)  
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.11/dist-packages (from markdown-it-py>=2.2.0->rich->keras>=3.5.0->tensorflow<2.19,>=2.18->tf-keras) (0.1.2)  
Requirement already satisfied: gym==0.17.3 in /usr/local/lib/python3.11/dist-packages (0.17.3)  
Requirement already satisfied: scipy in /usr/local/lib/python3.11/dist-packages (from gym==0.17.3) (1.15.3)  
Requirement already satisfied: numpy>=1.10.4 in /usr/local/lib/python3.11/dist-packages (from gym==0.17.3) (2.0.2)

Requirement already satisfied: pygame<=1.5.0,>=1.4.0 in /usr/local/lib/python3.11/dist-packages (from gym==0.17.3) (1.5.0)

Requirement already satisfied: cloudpickle<1.7.0,>=1.2.0 in /usr/local/lib/python3.11/dist-packages (from gym==0.17.3) (1.6.0)

Requirement already satisfied: future in /usr/local/lib/python3.11/dist-packages (from pygame<=1.5.0,>=1.4.0->gym==0.17.3) (1.0.0)

Collecting git+https://github.com/Kojoley/atari-py.git

Cloning https://github.com/Kojoley/atari-py.git to /tmp/pip-req-build-adggytsj

Running command git clone --filter=blob:none --quiet https://github.com/Kojoley/atari-py.git /tmp/pip-req-build-adggytsj

Resolved https://github.com/Kojoley/atari-py.git to commit 86a1e05c0a95e9e6233c3a413521fdb34ca8a089

Preparing metadata (setup.py) ... done

Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (from atari-py==1.2.2) (2.0.2)

Requirement already satisfied: keras-rl2==1.0.5 in /usr/local/lib/python3.11/dist-packages (1.0.5)

Requirement already satisfied: tensorflow in /usr/local/lib/python3.11/dist-packages (from keras-rl2==1.0.5) (2.18.0)

Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow->keras-rl2==1.0.5) (1.4.0)

Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow->keras-rl2==1.0.5) (1.6.3)

Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.11/dist-packages (from tensorflow->keras-rl2==1.0.5) (25.2.10)

Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in /usr/local/lib/python3.11/dist-packages (from tensorflow->keras-rl2==1.0.5) (0.6.0)

Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.11/dist-packages (from tensorflow->keras-rl2==1.0.5) (0.2.0)

Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow->keras-rl2==1.0.5) (18.1.1)

Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.11/dist-packages (from tensorflow->keras-rl2==1.0.5) (3.4.0)

Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packages (from tensorflow->keras-rl2==1.0.5) (24.2)

Requirement already satisfied: protobuf!=4.21.0,!4.21.1,!4.21.2,!4.21.3,!4.21.4,!4.21.5,<6.0.0dev,>=3.20.3 in /usr/local/lib/python3.11/dist-packages (from tensorflow->keras-rl2==1.0.5) (5.29.5)

Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow->keras-rl2==1.0.5) (2.32.3)

Requirement already satisfied: setuptools in /usr/local/lib/python3.11/dist-packages (from tensorflow->keras-rl2==1.0.5) (75.2.0)

Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow->keras-rl2==1.0.5) (1.17.0)

Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow->keras-rl2==1.0.5) (3.1.0)

Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/python3.11/dist-packages (from tensorflow->keras-rl2==1.0.5) (4.14.1)

Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow->keras-rl2==1.0.5) (1.17.2)

Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.11/dist-packages (from tensorflow->keras-rl2==1.0.5) (1.73.1)

Requirement already satisfied: tensorboard<2.19,>=2.18 in /usr/local/lib/python3.11/dist-packages (from tensorflow->keras-rl2==1.0.5) (2.18.0)

Requirement already satisfied: keras>=3.5.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow->keras-rl2==1.0.5) (3.8.0)

Requirement already satisfied: numpy<2.1.0,>=1.26.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow->keras-rl2==1.0.5) (2.0.2)

Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow->keras-rl2==1.0.5) (3.14.0)

Requirement already satisfied: ml-dtypes<0.5.0,>=0.4.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow->keras-rl2==1.0.5) (0.4.1)

Requirement already satisfied: tensorflow-io-gcs-filesystem>=0.23.1 in /usr/local/lib/python3.11/dist-packages (from tensorflow->keras-rl2==1.0.5) (0.37.1)

Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.11/dist-packages (from astunparse>=1.6.0->tensorflow->keras-rl2==1.0.5) (0.45.1)

Requirement already satisfied: rich in /usr/local/lib/python3.11/dist-packages (from keras>=3.5.0->tensorflow->keras-rl2==1.0.5) (13.9.4)

Requirement already satisfied: namex in /usr/local/lib/python3.11/dist-packages (from keras>=3.5.0->tensorflow->keras-rl2==1.0.5) (0.1.0)

Requirement already satisfied: optree in /usr/local/lib/python3.11/dist-packages (from keras>=3.5.0->tensorflow->keras-rl2==1.0.5) (0.16.0)

Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.21.0->tensorflow->keras-rl2==1.0.5) (3.4.2)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.21.0->tensorflow->keras-rl2==1.0.5) (3.10)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.21.0->tensorflow->keras-rl2==1.0.5) (2.4.0)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.21.0->tensorflow->keras-rl2==1.0.5) (2025.7.14)

Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.11/dist-packages (from tensorboard<2.19,>=2.18->tensorflow->keras-rl2==1.0.5) (3.8.2)

Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.11/dist-packages (from tensorboard<2.19,>=2.18->tensorflow->keras-rl2==1.0.5) (0.7.2)

Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from tensorboard<2.19,>=2.18->tensorflow->keras-rl2==1.0.5) (3.1.3)

Requirement already satisfied: MarkupSafe>=2.1.1 in /usr/local/lib/python3.11/dist-packages (from werkzeug>=1.0.1->tensorboard<2.19,>=2.18->tensorflow->keras-rl2==1.0.5) (3.0.2)

Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.11/dist-packages (from rich->keras>=3.5.0->tensorflow->keras-rl2==1.0.5) (3.0.0)

Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.11/dist-packages (from rich->keras>=3.5.0->tensorflow->keras-rl2==1.0.5) (2.19.2)

Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.11/dist-packages (from markdown-it-py>=2.2.0->rich->keras>=3.5.0->tensorflow->keras-rl2==1.0.5) (0.1.2)

Requirement already satisfied: ml\_dtypes in /usr/local/lib/python3.11/dist-packages (0.4.1)

Collecting ml\_dtypes

Using cached ml\_dtypes-0.5.4-cp311-cp311-manylinux\_2\_27\_x86\_64.manylinux\_2\_28\_x86\_64.whl.metadata (8.9 kB)

Requirement already satisfied: numpy>=1.21 in /usr/local/lib/python3.11/dist-packages (from ml\_dtypes) (2.0.2)

Using cached ml\_dtypes-0.5.4-cp311-cp311-manylinux\_2\_27\_x86\_64.manylinux\_2\_28\_x86\_64.whl (5.0 MB)

Installing collected packages: ml\_dtypes

Attempting uninstall: ml\_dtypes

Found existing installation: ml-dtypes 0.4.1

Uninstalling ml-dtypes-0.4.1:

Successfully uninstalled ml-dtypes-0.4.1

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.

tensorflow 2.18.0 requires ml-dtypes<0.5.0,>=0.4.0, but you have ml-dtypes 0.5.4 which is incompatible.

Successfully installed ml\_dtypes-0.5.4

### 1.4.5. Resultados y estado final

Con la culminación de este pipeline de aprovisionamiento dirigido, se da por finalizada la **Fase I. Infraestructura de despliegue y preparación del entorno de ejecución**. El estado del sistema de desarrollo se consolida bajo las siguientes especificaciones:

- La pila de software se encuentra completamente inmovilizada, garantizando que el operador de actualización de la red y la función de transición del simulador interactúen mediante estructuras de tensores idénticas.
- Se resuelven de forma preventiva los conflictos de compilación del emulador clásico ALE, asegurando la reproducibilidad determinista del proceso de aprendizaje con independencia de la plataforma.

Este entorno provee las garantías necesarias para adentrarse en la definición conceptual de las reglas de control y los criterios académicos de evaluación que componen la siguiente sección.

---

---

## Fase II. Especificaciones operativas y criterios de validación de la actividad

### 2.1. Directrices del proyecto práctico

---

*En este apartado se compendia el marco normativo de la actividad. Se formaliza el umbral de rendimiento exigido en la fase de evaluación, así como la estructura analítica de la solución basada en aprendizaje por refuerzo profundo.*

#### ◦ Formalización de los criterios de validación

El objetivo central de la actividad radica en capacitar a un agente inteligente para resolver de forma autónoma el entorno de simulación clásica de Atari `Enduro-v0` empleando la arquitectura de red Q profunda (*Deep Q-Network*, DQN). Para evaluar el éxito del proceso de optimización, se establece un criterio de suficiencia condicional basado en el rendimiento empírico del agente durante la fase de inferencia pura (modo de prueba o *test*), donde este debe abstenerse de realizar actualizaciones de los pesos o de inducir ruido estocástico artificial.

Matemáticamente, si se denota como  $R_e$  la recompensa total acumulada en un episodio de prueba  $e$  perteneciente al conjunto de episodios de evaluación  $\mathcal{E}$  (donde el cardinal del conjunto equivale a  $|\mathcal{E}| = N$ ), el cumplimiento del requisito mínimo de la actividad exige que la recompensa media esperada sea igual o superior al umbral paramétrico  $\tau_{\text{grupo}}$  asignado al equipo de desarrollo:

$$\mathbb{E}[R] = \frac{1}{N} \sum_{e \in \mathcal{E}} R_e \geq \tau_{\text{grupo}}$$

Este estimador numérico se extraerá directamente de los registros generados por el método de evaluación final del agente. El desarrollo de la solución se articula obligatoriamente en tres fases secuenciales e interdependientes:

1. Construcción del aproximador funcional de base convolucional, parametrizado en consonancia con las dimensiones del espacio de estados.
2. Evaluación del agente a través de, al menos, tres propuestas metodológicas diferenciadas, orientadas a mitigar la varianza y mejorar la política de control final.
3. Análisis cuantitativo de los resultados basado en la evidencia estadística, incorporando un mínimo de tres gráficas comparativas que ilustren de forma inequívoca la convergencia del algoritmo.

#### ○ Resultados y estado final

La asimilación de este marco normativo concluye la **Fase II. Especificaciones operativas y criterios de validación de la actividad**. El estado de control de la práctica se consolida bajo los siguientes puntos:

- Se han interiorizado las restricciones analíticas de la rúbrica, asumiendo la obligatoriedad de implementar una red convolucional pura dedicada a extraer las características geométricas de la pantalla sin alterar la resolución pixelar mediante operadores de reducción espacial (*max pooling*).
- Queda definido el protocolo de validación final basado en la recompensa media esperada, parametrizando el umbral numérico que determinará la suficiencia de los agentes en la fase de inferencia.

Con las directrices plenamente integradas y la infraestructura física resuelta en la sección previa, el entorno de desarrollo se encuentra maduro para iniciar el bloque experimental central que compone la siguiente sección.

---

## Fase III. Bloque de desarrollo experimental y desarrollo analítico

### 3.1. Importación de módulos científicos y resolución preventiva de colisiones del sistema de tipos

---

*En este apartado se inicializa el ecosistema analítico del proyecto mediante la carga coordinada de las bibliotecas de álgebra lineal y aprendizaje profundo. Se aborda la resolución preventiva de discrepancias en la representación binaria de datos entre las interfaces clásicas y los motores de ejecución de tensores.*

#### ○ Desarrollo analítico y consistencia de tipos

La integración de bibliotecas desarrolladas en distintas etapas computacionales suele generar desajustes en el sistema de tipos. Específicamente, la interfaz clásica de control de agentes `keras-r12` indexa el contador global de interacciones mediante una codificación entera de rango reducido de 16 bits ( $\mathbb{Z}_{16}$ ), mientras que el motor numérico moderno de `numpy` y el backend de `tensorflow` operan de forma nativa con una precisión extendida de 64 bits ( $\mathbb{Z}_{64}$ ).

Cuando el volumen de interacciones supera el límite representable en 16 bits sin signo (65.535 pasos o 32.767 si se evalúa con signo), se produce un desbordamiento aritmético silencioso:

$$\exists x \in \mathbb{Z}_{64} \quad \text{tal que} \quad x > \max(\mathbb{Z}_{16}) \implies x \pmod{2^{16}} \neq x$$

Esta colisión de tipos altera las ecuaciones lógicas de la memoria del búfer de repetición y el bucle de optimización de Bellman, provocando que el entrenamiento colapse. Para neutralizar este error de forma no invasiva, se aplica un parche de inyección dinámica que reasigna el puntero de tipos interno de la biblioteca hacia el formato extendido de 64 bits, imponiendo una restricción de identidad estructural:

$$\mathcal{T}_{\text{target}}(\text{int16}) \equiv \mathcal{T}_{\text{source}}(\text{int64})$$

Asimismo, el proyecto contempla el uso de arquitecturas avanzadas de control, requiriendo habilitar la deserialización no restrictiva en el módulo `tf_keras`. La arquitectura *Dueling DQN* se construye separando las cabezas lógicas de valor y ventaja mediante capas `Lambda` que contienen funciones anónimas de Python. Para que el motor pueda clonar estas estructuras de manera simétrica y generar la red objetivo de Bellman, es mandatorio anular por código las directivas de seguridad que restringen la deserialización insegura de capas personalizadas en el hilo principal.

#### ○ Directrices de implementación

A continuación, se exponen las pautas esenciales que guían la inyección de módulos y la armonización de la infraestructura numérica:

- Se configuran las variables globales del sistema operativo para forzar el uso de la API heredada de Keras en el backend de TensorFlow.
- Se cargan ordenadamente las dependencias científicas de cálculo matricial, el emulador y las abstracciones lógicas del agente.
- Se intercepta el núcleo numérico de `keras-r12` reasignando el indexador a 64 bits para inmunizar los contadores contra desbordamientos aritméticos.
- Se ejecuta una comprobación de hardware y una inyección de permisos de deserialización para dar soporte nativo a las capas en duelo.

### ***Inicialización del backend de aprendizaje profundo y GPU***

```
In [7]: import os

# Configuración obligatoria del sistema operativo antes de importar librerías.
# Fuerza el uso de la API clásica de Keras sobre las versiones modernas de Tensor
os.environ['TF_USE_LEGACY_KERAS'] = "1"
```

```

import tensorflow.keras as tf
import tensorflow as tf1
from keras import __version__

# Sincronización de metadatos de versión para asegurar la consistencia del enlace
tf.__version__ = __version__

# Diagnóstico visual de dispositivos de aceleración por hardware en la máquina a
print(tf1.__version__)
print(tf1.config.list_physical_devices('GPU'))
print(tf1.config.list_logical_devices('GPU'))

```

2.18.0

```
[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

```
[LogicalDevice(name='/device:GPU:0', device_type='GPU')]
```

### ***Importación de dependencias científicas y Keras-RL2***

```

In [8]: from __future__ import division

from PIL import Image
import numpy as np
import gym

# Importación de las capas convolucionales y de estructuración espacial profunda
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Activation, Flatten, Convolution2D, P

# Adaptación jerárquica del optimizador según el entorno (Colab local frente a n
if IN_COLAB:
    from tensorflow.keras.optimizers import Adam
else:
    from tensorflow.keras.optimizers import Adam

import tensorflow.keras.backend as K

# Importación de abstracciones, políticas estocásticas y callbacks del agente.
from rl.agents.dqn import DQNAgent
from rl.policy import LinearAnnealedPolicy, BoltzmannQPolicy, EpsGreedyQPolicy
from rl.memory import SequentialMemory
from rl.core import Processor
from rl.callbacks import FileLogger, ModelIntervalCheckpoint

```

### ***Inyección dinámica de parches numéricos y operativos***

```

In [9]: # Parche preventivo en el sistema de tipos de la biblioteca keras-rl2.
# Corrección ortogonal que sustituye enteros de 16 bits por enteros de 64 bits,
# evitando desbordamientos numéricos silenciosos en el contador de interacciones
import rl.core as rl_core
rl_core.np.int16 = np.int64

```

```

In [10]: # — PARCHE OPERATIVO: Permisos de clonación para capas de la Dueling Network —
import tf_keras
# Anula de forma controlada el bloqueo de seguridad para permitir que el optimiz
# cree y actualice la arquitectura objetivo basada en capas Lambda con Lambdas d
tf_keras.__internal__.enable_unsafe_deserialization()

```

## **3.1.5. Resultados y estado final**

La ejecución estructurada de las celdas de este apartado asienta y protege las variables base del experimento. Al concluir este bloque, el espacio de trabajo consolida las siguientes condiciones:

- Las dependencias matriciales, funcionales y estructurales se encuentran alojadas correctamente en la memoria activa del kernel.
- El sistema de tipos queda unificado bajo una precisión de 64 bits, blindando los contadores del búfer frente a desbordamientos aritméticos continuos.
- El backend técnico dispone de permisos de deserialización asíncrona, dejando el entorno listo para compilar con éxito tanto modelos lineales clásicos como variantes avanzadas en duelo.

Con la consistencia de tipos y parches algorítmicos resueltos, el espacio de desarrollo se encuentra preparado para avanzar hacia la instanciación física del simulador y el diseño del pipeline de preprocesamiento espacial en la siguiente sección.

---

## 3.2. Configuración base y diseño del pipeline de preprocesamiento para el entorno de Atari (AtariProcessor)

---

*En este apartado se detalla el pipeline de transformación de las observaciones nativas de `Enduro-v0` mediante la clase `AtariProcessor`, reduciendo la dimensionalidad espacio-temporal para optimizar la convergencia del algoritmo.*

### ◦ Lógica analítica del preprocesamiento

Los fotogramas RGB nativos ( $210 \times 160 \times 3$ ) se compactan a matrices de  $84 \times 84$  píxeles mediante tres transformaciones secuenciales acopladas para mitigar el coste computacional del búfer de experiencia:

1. En primer lugar, se elimina la redundancia cromática del fotograma transformando los tres canales nativos en un único canal de escala de grises mediante la combinación lineal de luminancia estandarizada:

$$Y = 0.299R + 0.587G + 0.114B$$

2. Seguidamente, la matriz resultante se somete a una reducción dimensional basada en interpolación bilineal, adaptando la resolución a un espacio estandarizado de  $84 \times 84$  píxeles.
3. Finalmente, se aplica una normalización lineal que mapea los valores de los píxeles al intervalo de coma flotante  $[0.0, 1.0]$  mediante el operador de escala:

$$\bar{B} = \frac{B}{255.0}$$

Esta última operación estabiliza el descenso de gradiente durante la optimización. Para salvaguardar la memoria RAM, la normalización se ejecuta de forma diferida solo sobre los lotes de muestras seleccionados para el entrenamiento, lo que permite conservar el historial del búfer en formato entero de 8 bits sin signo ( `uint8` ), reduciendo el almacenamiento en un factor de tres.

#### o Directrices de implementación

- La conversión forzada a un canal de luminancia disminuye la carga del búfer de repetición de experiencias.
- El escalamiento numérico a coma flotante se reserva exclusivamente para los minilotes de entrenamiento, protegiendo los recursos de la máquina anfitriona.
- El flujo de premios del simulador se confina al intervalo  $[-1.0, 1.0]$  mediante un operador de truncamiento, homogeneizando la magnitud de las actualizaciones de la red frente a la dispersión de puntuaciones de Enduro.

### **Inicialización del entorno operativo y asignación de parámetros estandarizados**

```
In [11]: # Dimensiones espaciales estandarizadas a las que se reducirán los fotogramas de
INPUT_SHAPE = (84, 84)
# Cantidad de fotogramas secuenciales consecutivos que se apilarán por estado.
WINDOW_LENGTH = 4

# Identificador literal del entorno de simulación de carreras clásicas de Atari.
env_name = 'Enduro-v0'

# Inicialización física de la instancia del simulador a través de OpenAI Gym.
env = gym.make(env_name)

# Fijación de las semillas pseudoaleatorias para garantizar la reproducibilidad
np.random.seed(123)
env.seed(123)

# Recuperación del cardinal del espacio discreto de acciones que el agente puede
nb_actions = env.action_space.n

# Impresión en la terminal de los metadatos estructurales del entorno para la va
print("Entorno:", env_name)
print("Número de acciones disponibles:", nb_actions)
print("Significado de las acciones:", env.unwrapped.get_action_meanings())
print("Formato de las observaciones:", env.observation_space)
```

Entorno: Enduro-v0

Número de acciones disponibles: 9

Significado de las acciones: ['NOOP', 'FIRE', 'RIGHT', 'LEFT', 'DOWN', 'DOWNRIGHT', 'DOWNLEFT', 'RIGHTFIRE', 'LEFTFIRE']

Formato de las observaciones: Box(0, 255, (210, 160, 3), uint8)

### **Implementación modular de la clase de preprocesamiento visual y acotación de premios**

```
In [12]: class AtariProcessor(Processor):
        """Clase especializada en la transformación y acondicionamiento intermedio d
        Optimiza el consumo de la memoria RAM y estabiliza el cálculo del descenso d
```

```

def process_observation(self, observation):
    # Validación estructural para certificar que el fotograma nativo posee t
    assert observation.ndim == 3

    # Transformación del arreglo numérico crudo en un objeto de imagen para
    img = Image.fromarray(observation)

    # Reducción dimensional mediante interpolación bilineal y descarte de la
    # Se genera una sola matriz compacta basada exclusivamente en un canal d
    img = img.resize(INPUT_SHAPE).convert('L')

    # Reconstrucción del arreglo de computación matricial a partir de la ima
    processed_observation = np.array(img)

    # Verificación analítica de que las dimensiones de salida se adecuan a l
    assert processed_observation.shape == INPUT_SHAPE

    # Retorno del fotograma indexado en enteros sin signo de 8 bits para pro
    return processed_observation.astype('uint8')

def process_state_batch(self, batch):
    # Conversión del lote de estados muestreados hacia precisión de coma flo
    # Se aplica una división por 255.0 para escalar linealmente los píxeles
    processed_batch = batch.astype('float32') / 255.
    return processed_batch

def process_reward(self, reward):
    # Truncamiento de las recompensas acumuladas para confinarlas al interva
    # Neutraliza la varianza de la señal de actualización frente a puntuacio
    return float(np.clip(reward, -1., 1.))

```

### 3.2.3. Resultados y estado final

La ejecución de este bloque consolida el pipeline de acondicionamiento de datos visuales. Al concluir la sección, el sistema presenta el siguiente estado operativo confirmado:

- El simulador se encuentra instanciado bajo un régimen determinista, asegurando consistencia matemática en el muestreo de trayectorias con independencia del hilo ejecutor.
- La clase de procesamiento intermedio queda integrada en la memoria del intérprete, garantizando una reducción del volumen de datos en un factor de tres antes de almacenar los tensores en la memoria histórica.

Con el pipeline de observaciones plenamente operativo y optimizado, el entorno de desarrollo se encuentra preparado para detallar la topología del aproximador funcional jerárquico convolucional en la siguiente sección.

---

## 3.3. Diseño y topología jerárquica de redes neuronales convolucionales profundas de aproximación funcional

---

*En este apartado se formaliza la arquitectura del aproximador no lineal profundo para la estimación de los valores de la función  $Q$ . Se detalla la estructura jerárquica basada en operadores convolucionales puros encargados de extraer las características geométricas del entorno espacial de juego sin alterar su resolución mediante capas de reducción.*

#### ○ Fundamentación matemática y topológica

Para resolver un entorno con un espacio de estados continuo o de alta dimensionalidad como la matriz visual de Atari, la ecuación de optimalidad de Bellman se vuelve intratable de forma exacta. Por ello, se introduce un aproximador funcional parametrizado mediante una red neuronal convolucional profunda, denotado como  $Q(s, a; \theta) \approx Q^*(s, a)$ , donde  $\theta$  representa el vector global de pesos sinápticos. La topología de la red adopta la arquitectura canónica de Mnih et al. (2015), compuesta por tres capas convolucionales consecutivas que operan directamente sobre el tensor apilado de fotogramas, mapeando las características visuales hacia representaciones latentes de abstracción creciente.

Analíticamente, la red evita el uso de operadores de reducción espacial como *max pooling*, ya que estos inducen invarianza traslacional y destruyen la localización precisa de los elementos en movimiento, una información crítica para estimar la velocidad y la trayectoria de los vehículos en Enduro. En su lugar, el escalado espacial se delega a las convoluciones con zancadas o *strides*. El mapeo final se proyecta sobre una capa densamente conectada de 512 unidades de activación lineal rectificada (relu), bifurcándose en la capa de salida hacia tantas neuronas lineales como acciones discretas posea el entorno, devolviendo el vector completo de valores  $Q$  en una única inferencia hacia adelante:

$$Q(s, \cdot; \theta) \in \mathbb{R}^{|A|}$$

#### ○ Directrices de implementación

- Se emplea una estructura secuencial que permuta los ejes del tensor de entrada para garantizar la compatibilidad con el formato de canales del motor de ejecución.
- Tres capas convolucionales consecutivas con núcleos decrecientes y zancadas asimétricas reducen el espacio geométrico y expanden los canales de características.
- El tensor resultante se aplana mediante una proyección lineal antes de alimentar el bloque de procesamiento completamente conectado.
- Una variante topológica alternativa integra la activación lineal en la capa de salida para servir de base a las bifurcaciones de la arquitectura en duelo.

#### **Implementación de la arquitectura convolucional profunda estándar**

```
In [13]: # Composición de la tupla dimensional para la entrada de la red (canales apilado)
input_shape = (WINDOW_LENGTH,) + INPUT_SHAPE

def build_model(nb_actions, input_shape):
    """Construye la arquitectura convolucional profunda estándar propuesta por M
```

```

model = Sequential()

# Adaptación dinámica del orden de los ejes en función de la configuración d
if K.image_data_format() == 'channels_last':
    # Reordenamiento hacia el formato de ancho, alto y canales.
    model.add(Permute((2, 3, 1), input_shape=input_shape))
elif K.image_data_format() == 'channels_first':
    # Reordenamiento hacia el formato de canales, ancho y alto.
    model.add(Permute((1, 2, 3), input_shape=input_shape))
else:
    raise RuntimeError('Unknown image_dim_ordering.')
```

*# Primera capa convolucional encargada de procesar parches espaciales grande*

```

model.add(Convolution2D(32, (8, 8), strides=(4, 4)))
model.add(Activation('relu'))
```

*# Segunda capa convolucional dedicada a extraer características visuales de*

```

model.add(Convolution2D(64, (4, 4), strides=(2, 2)))
model.add(Activation('relu'))
```

*# Tercera capa convolucional para capturar patrones de movimiento locales y*

```

model.add(Convolution2D(64, (3, 3), strides=(1, 1)))
model.add(Activation('relu'))
```

*# Operación de aplanado para colapsar la estructura espacial tridimensional*

```

model.add(Flatten())
```

*# Bloque de procesamiento intermedio completamente conectado para la abstrac*

```

model.add(Dense(512))
model.add(Activation('relu'))
```

*# Capa lineal de salida que mapea las representaciones latentes al espacio d*

```

model.add(Dense(nb_actions))
model.add(Activation('linear'))
```

```

return model
```

*# Instanciación analítica del aproximador base y visualización resumida de su to*

```

model = build_model(nb_actions, input_shape)
print(model.summary())
```

Model: "sequential"

| Layer (type)                        | Output Shape       | Param # |
|-------------------------------------|--------------------|---------|
| =====                               |                    |         |
| permute (Permute)                   | (None, 84, 84, 4)  | 0       |
| conv2d (Conv2D)                     | (None, 20, 20, 32) | 8224    |
| activation (Activation)             | (None, 20, 20, 32) | 0       |
| conv2d_1 (Conv2D)                   | (None, 9, 9, 64)   | 32832   |
| activation_1 (Activation)           | (None, 9, 9, 64)   | 0       |
| conv2d_2 (Conv2D)                   | (None, 7, 7, 64)   | 36928   |
| activation_2 (Activation)           | (None, 7, 7, 64)   | 0       |
| flatten (Flatten)                   | (None, 3136)       | 0       |
| dense (Dense)                       | (None, 512)        | 1606144 |
| activation_3 (Activation)           | (None, 512)        | 0       |
| dense_1 (Dense)                     | (None, 9)          | 4617    |
| activation_4 (Activation)           | (None, 9)          | 0       |
| =====                               |                    |         |
| Total params: 1688745 (6.44 MB)     |                    |         |
| Trainable params: 1688745 (6.44 MB) |                    |         |
| Non-trainable params: 0 (0.00 Byte) |                    |         |
| =====                               |                    |         |
| None                                |                    |         |

### ***Arquitectura alternativa adaptada para la ramificación de redes en "dueling"***

```
In [14]: def build_model_dueling_ready(nb_actions, input_shape):
          """Construye la variante topológica optimizada para la arquitectura en dueling

          Integra la función de activación lineal dentro del operador denso de salida
          para preservar el acceso jerárquico a la capa oculta de 512 unidades.
          """
          model = Sequential()

          # Ajuste condicional del orden transaccional de los tensores visuales anfitrión
          if K.image_data_format() == 'channels_last':
              model.add(Permute((2, 3, 1), input_shape=input_shape))
          elif K.image_data_format() == 'channels_first':
              model.add(Permute((1, 2, 3), input_shape=input_shape))
          else:
              raise RuntimeError('Unknown image_dim_ordering.')

          # Estructura convolucional idéntica encargada de asegurar la homogeneidad an
          model.add(Convolution2D(32, (8, 8), strides=(4, 4)))
          model.add(Activation('relu'))
          model.add(Convolution2D(64, (4, 4), strides=(2, 2)))
          model.add(Activation('relu'))
          model.add(Convolution2D(64, (3, 3), strides=(1, 1)))
          model.add(Activation('relu'))
```

```
# Aplanado estructural y proyección lineal de control.
model.add(Flatten())
model.add(Dense(512))
model.add(Activation('relu'))

# Operador denso terminal con activación lineal fusionada de forma compacta.
# Permite que la biblioteca identifique correctamente la capa previa como nú
model.add(Dense(nb_actions, activation='linear'))

return model
```

### 3.3.3. Resultados y estado final

La ejecución coordinada de estas topologías consolida el aproximador funcional del sistema. Al finalizar este bloque, el estado operativo presenta las siguientes condiciones confirmadas:

- Las funciones de construcción de red se hallan cargadas en memoria, garantizando la extracción de vectores de valor Q adaptados a las nueve acciones de Enduro.
- El entorno dispone de un gemelo arquitectónico preparado para soportar el desacoplamiento de valor y ventaja de las propuestas en duelo.

Con los aproximadores definidos de forma consistente, el sistema cuenta con las garantías necesarias para inicializar las funciones auxiliares de control y el registro general de agentes en la sección inmediata.

## 3.4. Infraestructura central de evaluación, registro de agentes y control de interrupciones

*En este apartado se instrumenta la arquitectura de soporte encargada de la evaluación estandarizada, el registro centralizado de experimentos y la tolerancia a fallos del sistema. Se consolidan los envoltorios lógicos y las extensiones de las políticas de exploración necesarios para garantizar la persistencia e invulnerabilidad del hilo de ejecución frente a detenciones efímeras en entornos distribuidos o locales.*

### ○ Lógica analítica de control y tolerancia a fallos

La robustez del flujo experimental frente a la inestabilidad inherente de las conexiones de red y los límites de tiempo de las máquinas virtuales exige una infraestructura con tolerancia estricta a interrupciones. El guardado ordinario de parámetros mediante puntos de control persiste de forma exclusiva el tensor de pesos sinápticos  $\theta_t$ , pero desatiende por completo el estado interno de la política de exploración indexada en la memoria del intérprete (como el decaimiento progresivo de los coeficientes de aleatoriedad  $\epsilon$  o de temperatura  $\tau$ ). Al reanudar una ejecución interrumpida, los componentes clásicos calculan su decaimiento basándose de forma única en el contador

de pasos relativo del proceso, el cual se inicializa en cero en cada invocación del método de ajuste, provocando una indeseable saturación de ruido exploratorio que emborrona el conocimiento previamente adquirido.

Para neutralizar esta patología funcional de manera elegante, se diseña un sistema de control de desajustes temporales dividido en dos fases operativas coordinadas:

1. Un analizador histórico que examina los ficheros estructurados en formato JSON generados de forma asíncrona por el registrador para extraer el último valor escalar válido del parámetro de exploración.
2. Una extensión de la clase de planificación lineal ( `OffsetLinearAnnealedPolicy` ) que intercepta la lectura del paso actual del agente adicionando un desfase cronológico absoluto o *step offset*. De este modo, la ecuación del decaimiento se evalúa utilizando la coordenada temporal global real del experimento, mitigando cualquier discontinuidad analítica en la recolección de trayectorias de éxito.

#### o Directrices de implementación

- Se parametrizan las constantes numéricas que gobiernan el horizonte temporal del entrenamiento y el cardinal de episodios para la inferencia estadística comparativa.
- Un método unificado automatiza la carga de los tensores binarios almacenados en el disco secundario antes de dar inicio a las rutinas de validación.
- Un catálogo centralizado indexa las variables físicas de enrutamiento y los prefijos lógicos de las ocho propuestas del proyecto.
- Se extiende la clase del callback de guardado por intervalos agregando un desfase numérico absoluto para impedir la sobreescritura accidental de los binarios previos.

#### **Variable global `nb_steps_train`**

```
In [15]: # Especificación del volumen global de interacciones para el ciclo completo de e
NB_STEPS_TRAIN = 2300000 # ajustar según tiempo disponible / checkpoints
```

#### **Variable global `nb_episodes`**

```
In [16]: # Cardinal fijado para las iteraciones de evaluación en modo de inferencia pura.
NB_EPISODES = 100
```

#### **Función para la carga de pesos en fase de prueba**

```
In [17]: def load_weights_for_test(agent, proposal_id, weights_dir):
        """Carga el último checkpoint disponible para poder hacer test."""
        ckpt, step = find_latest_checkpoint(weights_dir, f'dqn_Enduro-v0_{proposal_id}')
        if ckpt is not None:
            # Carga de los tensores de pesos sinápticos desde la ruta física localizada
            agent.load_weights(ckpt)
            print(f"[{proposal_id}] Pesos cargados para test desde: {ckpt} (step {step})")
        else:
            # Alerta visual en consola si el directorio persistente carece de binarios
            print(f"[{proposal_id}] ⚠ Sin checkpoints. Entrena primero.")
```

#### **Función para la selección de la propuesta y ejecución de la prueba**

```
In [18]: def test_proposal(proposal_id, agents_registry, nb_episodes=NB_EPISODES):
        """Evalúa una propuesta cargando de forma automática su último punto de control.

        Recupera la información del registro centralizado y ejecuta el agente en modo test.

        """
        if proposal_id not in agents_registry:
            print(f"[{proposal_id}] no está en el registro. Opciones: {list(agents_registry.keys())}")
            return

        info = agents_registry[proposal_id]

        if info['agent'] is None:
            print(f"[{proposal_id}] El agente no está en memoria todavía. Ejecuta primer episodio.")
            return
        # Localización del fichero indexado con la coordenada temporal más avanzada
        weights_filename, last_step = find_latest_checkpoint(info['dir'], info['prefix'])

        if weights_filename is None:
            print(f"[{proposal_id}] Sin checkpoints en {info['dir']}. Entrena primer episodio.")
            return

        print(f"[{proposal_id}] Cargando pesos desde: {weights_filename} (step {last_step}).")
        info['agent'].load_weights(weights_filename)

        # Inferencia pura sin actualizaciones de la red ni inyección de ruido estocástico
        test_history = info['agent'].test(env, nb_episodes=nb_episodes, visualize=False)

        # Cálculo y presentación por consola de los estimadores estadísticos de recompensa
        mean_reward = np.mean(test_history.history['episode_reward'])
        std_reward = np.std(test_history.history['episode_reward'])
        print(f"[{proposal_id}] Recompensa media en {nb_episodes} episodios: {mean_reward} ± {std_reward}")
        return test_history
```

### Registro centralizado de propuestas experimentales

```
In [19]: # Catálogo indexado para la administración unificada y control periférico de las propuestas
AGENTS_REGISTRY = {
    'p1': {'dir': WEIGHTS_DIR_P1, 'prefix': 'dqn_Enduro-v0_p1', 'agent': None},
    'p2': {'dir': WEIGHTS_DIR_P2, 'prefix': 'dqn_Enduro-v0_p2', 'agent': None},
    'p3': {'dir': WEIGHTS_DIR_P3, 'prefix': 'dqn_Enduro-v0_p3', 'agent': None},
    'p4': {'dir': WEIGHTS_DIR_P4, 'prefix': 'dqn_Enduro-v0_p4', 'agent': None},
    'p5': {'dir': WEIGHTS_DIR_P5, 'prefix': 'dqn_Enduro-v0_p5', 'agent': None},
    'p6': {'dir': WEIGHTS_DIR_P6, 'prefix': 'dqn_Enduro-v0_p6', 'agent': None},
    'p7': {'dir': WEIGHTS_DIR_P7, 'prefix': 'dqn_Enduro-v0_p7', 'agent': None},
    'p8': {'dir': WEIGHTS_DIR_P8, 'prefix': 'dqn_Enduro-v0_p8', 'agent': None},
}
```

### Función de construcción y compilación del agente DQN

```
In [20]: def build_dqn_agent(model, nb_actions, memory, processor, policy,
                             nb_steps_warmup, target_model_update,
                             train_interval=4, gamma=.99,
                             enable_double_dqn=False, enable_dueling_network=False,
                             dueling_type='avg'):
        """Encapsula la instanciación estructural y la compilación del agente de aprendizaje por refuerzo con DQN.

        """
        dqn = DQNAgent(model=model, nb_actions=nb_actions, policy=policy,
                       memory=memory, processor=processor,
                       nb_steps_warmup=nb_steps_warmup, gamma=gamma,
```

```

        target_model_update=target_model_update,
        train_interval=train_interval,
        enable_double_dqn=enable_double_dqn,
        enable_dueling_network=enable_dueling_network,
        dueling_type=dueling_type)
# Compilación analítica del agente aplicando la tasa de aprendizaje estándar
dqn.compile(Adam(learning_rate=.00025), metrics=['mae'])
return dqn

```

### Funciones auxiliares y clases para la gestión de puntos de control reanudables

```

In [21]: class ResumableModelIntervalCheckpoint(ModelIntervalCheckpoint):
        """Clase adaptada del callback de guardado por intervalos para garantizar la
        Previene la colisión de ficheros de pesos en el disco duro adicionando un de
        """

        def __init__(self, filepath, interval, step_offset=0, verbose=0):
            super().__init__(filepath, interval, verbose=verbose)
            self.step_offset = step_offset

        def on_step_end(self, step, logs={}):
            self.total_steps += 1
            if self.total_steps % self.interval != 0:
                return
            # Adición del desfase numérico para mantener la secuencia lineal exacta
            absolute_step = self.step_offset + self.total_steps
            filepath = self.filepath.format(step=absolute_step, **logs)
            if self.verbose > 0:
                print(f'Step {absolute_step}: guardando modelo en {filepath}')
            self.model.save_weights(filepath, overwrite=True)

    def find_latest_checkpoint(weights_dir, prefix):
        """Rastrea el árbol jerárquico de directorios para identificar el punto de c
        import re
        import glob

        # Mapeo de archivos de índice que cumplen con la sintaxis estandarizada de g
        pattern = os.path.join(weights_dir, f'{prefix}_weights_*.h5f.index')
        candidates = glob.glob(pattern)
        best_step = -1
        best_path = None
        for c in candidates:
            # Expresión regular encargada de aislar de forma unívoca el valor del pa
            m = re.search(r'_weights_(\d+)\.h5f\.index$', c)
            if m:
                step = int(m.group(1))
                if step > best_step:
                    best_step = step
                    best_path = c[:-len('.index')]
        if best_path is None:
            return None, 0
        return best_path, best_step

```

### Funciones y clases adaptativas para la reanudación del decaimiento exploratorio

```

In [22]: def read_last_annealed_value_from_log(log_path, attr='eps'):
        """Deserializa de forma síncrona las trazas históricas JSON para recuperar e
        import json

```

```

import os

metric_key = f'mean_{attr}'
if not os.path.exists(log_path):
    return None, 0

with open(log_path, 'r') as f:
    data = json.load(f)

if metric_key not in data or not data[metric_key]:
    return None, 0

# Extracción del último valor registrado para neutralizar reinicios accident
last_value = data[metric_key][-1]
last_step = data['nb_steps'][-1] if 'nb_steps' in data and data['nb_steps']
return last_value, last_step

class OffsetLinearAnnealedPolicy(LinearAnnealedPolicy):
    """Estrategia de planificación adaptativa con inyección de desfase cronológico
Evita el reinicio del decaimiento de aleatoriedad obligando a evaluar la fun
    """

    def __init__(self, inner_policy, attr, value_max, value_min, value_test,
                 nb_steps, step_offset=0):
        super().__init__(inner_policy, attr, value_max, value_min, value_test, n
        self.step_offset = step_offset

    def get_current_value(self):
        if self.agent.training:
            a = -float(self.value_max - self.value_min) / float(self.nb_steps)
            b = float(self.value_max)
            # Adición del desajuste temporal sobre el contador interno para calc
            absolute_step = self.step_offset + self.agent.step
            value = max(self.value_min, a * float(absolute_step) + b)
        else:
            value = self.value_test
        return value

```

### 3.4.4. Resultados y estado final

La instrumentación secuencial de las celdas de este apartado consolida la infraestructura periférica de control de la simulación. Al concluir este bloque, el espacio de trabajo presenta las siguientes condiciones técnicas confirmadas:

- Los métodos analíticos de evaluación automatizada y las constantes del horizonte operativo quedan inmovilizados en memoria, garantizar un marco comparativo homogéneo.
- Las extensiones especializadas del callback de guardado y de la política lineal adaptativa blindan los experimentos contra la volatilidad de las desconexiones mediante la inyección del desfase cronológico absoluto.
- El registro unificado se halla inicializado y listo para catalogar la información estructural y paramétrica de los ocho perfiles algorítmicos.

Con la infraestructura central completamente resuelta, el entorno de desarrollo cuenta con las absolutas garantías técnicas necesarias para adentrarse en la formalización

teórica del dilema de la dispersión de recompensa y el inicio formal del bucle de entrenamiento de los agentes en la siguiente sección.

---

## Fase IV. Bloque de desarrollo experimental: exploración y variantes

### 4. Introducción: análisis de la dispersión de la señal de recompensa y estrategias de exploración

---

*En este apartado se diagnostica el problema de la dispersión de la recompensa inherente al entorno de simulación y se fundamentan teóricamente las estrategias estocásticas empleadas para garantizar un muestreo efectivo del espacio de estados.*

El entorno de conducción de Atari, `Enduro-v0`, plantea un desafío superlativo para los algoritmos de aproximación del valor de calidad debido a la extrema dispersión de su señal de retroalimentación (*sparse reward*). Desde una perspectiva algorítmica, el simulador emite un refuerzo escalar positivo ( $r_t = +1$ ) única y exclusivamente en el instante en que el agente rebasa a un vehículo adversario, careciendo de penalizaciones explícitas o recompensas graduales por el mantenimiento de una velocidad óptima de cruce.

Bajo una política inicial puramente aleatoria, denotada formalmente como:

$$\pi_0(a|s) = \frac{1}{|\mathcal{A}|}$$

La probabilidad conjunta de ejecutar la secuencia exacta de acciones requerida para acelerar de forma ininterrumpida e interceptar a los rivales es prácticamente nula. Al disponer de un espacio de acciones discreto compuesto por nueve primitivas matriciales, iniciar y mantener el avance depende de una sucesión continua de eventos improbables bajo una distribución de probabilidad uniforme.

Esta fenomenología acarrea una consecuencia crítica durante la fase de inicialización o periodo de calentamiento (*warmup*): la memoria de repetición de experiencias (*replay buffer*) se satura sistemáticamente con millones de transiciones de la forma  $(s_t, a_t, r_t = 0, s_{t+1})$  carentes de utilidad pedagógica. En ausencia de señales de error de diferencia temporal (error TD) significativas, el gradiente de la función de coste se desvanece, provocando que la actualización de los pesos sinápticos de la red neuronal convolucional se estanque y la aproximación de los valores  $Q(s, a)$  colapse hacia el reposo absoluto.

#### ○ Fundamentación de las estrategias de exploración algorítmica

Para evitar la parálisis de la fase de entrenamiento y resolver el dilema clásico entre la exploración (descubrimiento estocástico de nuevas trayectorias latentes) y la explotación (maximización del retorno esperado basándose en la experiencia acumulada), resulta imperativo sustituir la aleatoriedad plana por mecanismos de control sofisticados. A tal efecto, en los modelos experimentales subsiguientes se articulan y evalúan tres paradigmas metodológicos:

1. **Política de control ávida estática (mecanismo  $\epsilon$ -greedy basal):** Introduce una tasa de entropía constante en la toma de decisiones, forzando al agente a muestrear acciones aleatorias con una probabilidad fija  $\epsilon$ , independientemente de su nivel de convergencia. Sirve como línea base (*baseline*) para aislar y contrastar la viabilidad del aprendizaje bajo condiciones de muestreo inmutables.
2. **Exploración dinámico-lineal (decaimiento de  $\epsilon$ ):** Implementa un esquema de temple simulado (*annealing*), en el cual la probabilidad de actuar de forma impredecible decrece de manera monotónica conforme transcurren los ciclos de interacción temporal. Esta planificación garantiza un rastreo exhaustivo en fases tempranas del descenso de gradiente, consolidando la explotación sistemática una vez que la red neuronal ha interiorizado la topología bidimensional de recompensas.
3. **Distribución probabilística de Boltzmann (*Softmax*):** Lejos de inyectar ruido uniforme, esta política de selección estocástica pondera la probabilidad de ejecución de cada acción en función de la calidad relativa estimada por el modelo  $Q$ , modulada por un hiperparámetro de temperatura termodinámica ( $\tau$ ). De este modo, el agente descarta de forma temprana las acciones con un coste de oportunidad evidente, concentrando sus esfuerzos exploratorios de forma inteligente en aquellas alternativas que resultan estadísticamente prometedoras.

```
In [23]: # ANÁLISIS COMPLETO DE TODAS LAS PROPUESTAS
# Lee directamente desde Drive: Logs JSON + checkpoints guardados.

# Importación de librerías para la gestión de trazas, expresiones regulares y vi
import json, glob, re
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

REWARD_THRESHOLD = 200 # coches necesarios para completar el día 1 en Enduro
WINDOW_ROLLING   = 20  # ventana de media móvil en las gráficas

# — 1. Recopilar información de cada propuesta —————
def get_proposal_info(key, info):
    result = {
        'key':          key,
        'checkpoints':  [],
        'last_step':    0,
        'log':          None,
        'log_path':     None,
    }

    # — Checkpoints —————
    # Buscamos archivos .h5f.index dentro de info['dir']
    ckpt_pattern = os.path.join(info['dir'], f"{info['prefix']}_weights_*.h5f.in
    for path in glob.glob(ckpt_pattern):
```

```

        m = re.search(r'_weights_(\d+)\.h5f\.index$', path)
        if m:
            result['checkpoints'].append(int(m.group(1)))
    result['checkpoints'].sort()
    if result['checkpoints']:
        result['last_step'] = result['checkpoints'][-1]

# — Log JSON —————
# Nombre del Log: dqn_{env_name}_{key}_log.json dentro de info['dir']
log_path = os.path.join(info['dir'], f"dqn_{env_name}_{key}_log.json")
result['log_path'] = log_path
if os.path.exists(log_path):
    try:
        with open(log_path) as f:
            result['log'] = json.load(f)
    except Exception as e:
        print(f" [AVISO] No se pudo leer el log de {key}: {e}")

return result

proposals = {k: get_proposal_info(k, v) for k, v in AGENTS_REGISTRY.items()}

# — 2. Tabla resumen —————
print()
print("="*82)
print(" RESUMEN DE ESTADO – TODAS LAS PROPUESTAS")
print("="*82)
print(f"  {'':>4} {'Ckpts':>5} {'Último ckpt':>12} {'Ep log':>7} "
      f"{'Media':>7} {'Std':>6} {'%>=200':>7} {'Tendencia':>12} {'Epsilon':>10}")
print(" " + "-"*80)

for key, p in proposals.items():
    n_ckpt = len(p['checkpoints'])
    last_step = p['last_step']
    d = p['log']

    if d is None or len(d.get('episode_reward', [])) == 0:
        print(f"  {key.upper():>4} {n_ckpt:>5} {last_step:>12}, {'- sin log o si'
              continue

    r = np.array(d['episode_reward'])
    n = len(r)
    pct = 100 * (r >= REWARD_THRESHOLD).sum() / n

    q = max(1, n // 4)
    delta = r[-q:].mean() - r[:q].mean()
    tend = "📈 SUBE" if delta > 10 else ("📉 BAJA" if delta < -10 else "↔️ ESTA")

    eps_str = ''
    for attr in ('mean_eps', 'mean_tau'):
        if attr in d:
            vals = [x for x in d[attr] if x == x]
            if vals:
                label = 'eps' if attr == 'mean_eps' else 'tau'
                eps_str = f"{label}:{vals[0]:.3f}→{vals[-1]:.3f}"

    print(f"  {key.upper():>4} {n_ckpt:>5} {last_step:>12}, {n:>7} "
          f"{r.mean():>7.1f} {r.std():>6.1f} {pct:>6.0f}% "
          f"{tend:>12} {eps_str:>10}")

```

```

print("="*82)

# — 3. Diagnóstico detallado —————
print()
print("="*82)
print("  DIAGNÓSTICO DETALLADO")
print("="*82)

for key, p in proposals.items():
    d = p['log']
    print(f"\n {key.upper()} —————")

    # Checkpoints
    if not p['checkpoints']:
        print("    ⚠ Sin checkpoints – no ha superado el primer intervalo de ")
    else:
        print(f"    ✓ Checkpoints: {p['checkpoints']} (último: step {p['last_st

    # Log
    if d is None:
        print("    ⚠ Sin log JSON – la celda de entrenamiento no se ejecutó to
        continue

    r = np.array(d['episode_reward'])
    n = len(r)

    if n < 5:
        print(f"    ⚠ Solo {n} episodio(s) – prácticamente sin entrenar")
        if n == 1:
            print(f"        Primer reward: {r[0]:.0f} | steps: {d['nb_steps'][0]
            continue

    # Inicio del log
    primer = d['nb_steps'][0]
    ultimo = d['nb_steps'][-1]
    if primer < 10000:
        print(f"    ⓘ Log arranca desde cero (primer nb_steps={primer:,})")
    else:
        print(f"    ⓘ Log arranca reanudando (primer nb_steps={primer:,})")
    print(f"    ⓘ Episodios en log: {n} | Hasta step: {ultimo:,}")

    # Reward stats
    pct = 100 * (r >= REWARD_THRESHOLD).sum() / n
    print(f"    ⓘ Reward: media={r.mean():.1f} std={r.std():.1f} "
          f"min={r.min():.0f} max={r.max():.0f} %>=200: {pct:.0f}%")

    # Mejor racha consecutiva
    mejor = racha = 0
    for rv in r:
        racha = racha + 1 if rv >= REWARD_THRESHOLD else 0
        mejor = max(mejor, racha)
    emoji = "✅" if mejor >= 100 else ("💎" if mejor >= 20 else "❌")
    print(f"    {emoji} Mejor racha consecutiva >= {REWARD_THRESHOLD}: "
          f"{mejor} episodios (objetivo: 100)")

    # Epsilon / tau
    for attr, label in [('mean_eps', 'eps'), ('mean_tau', 'tau')]:
        if attr in d:
            vals = [x for x in d[attr] if x == x]
            if vals:

```



```

ax2.axhline(REWARD_THRESHOLD, color='red', linestyle='--', linewidth=1.5)
ax2.set_title(f'Media móvil (ventana={WINDOW_ROLLING} ep)')
ax2.set_xlabel('Steps')
ax2.set_ylabel('Reward medio')
ax2.legend(fontsize=8)
ax2.grid(alpha=0.3)

# Gráfica 3: mean_q
ax3 = fig.add_subplot(gs[1, 0])
for key, p in props_graf.items():
    if 'mean_q' not in p['log']:
        continue
    qv = [x for x in p['log']['mean_q'] if x == x]
    sv = p['log']['nb_steps']
    if len(qv) > 1:
        s_q = sv[len(sv) - len(qv):]
        ax3.plot(s_q, qv, label=key.upper(), color=color_map[key], linewidth=1.5)
ax3.set_title('Mean Q (↑ indica que la red está aprendiendo)')
ax3.set_xlabel('Steps')
ax3.set_ylabel('Mean Q')
ax3.legend(fontsize=8)
ax3.grid(alpha=0.3)

# Gráfica 4: barras % >= 200
ax4 = fig.add_subplot(gs[1, 1])
claves = list(props_graf.keys())
pcts = [100 * (np.array(props_graf[k]['log']['episode_reward']) >= REWARD_THRESHOLD).sum() / len(props_graf[k]['log']) for k in claves]
bars = ax4.bar(range(len(claves)), pcts, color=[color_map[k] for k in claves], edgecolor='white')
ax4.axhline(100, color='red', linestyle='--', linewidth=1.5, label='Objetivo')
ax4.set_xticks(range(len(claves)))
ax4.set_xticklabels([k.upper() for k in claves], fontsize=9)
ax4.set_ylabel('% episodios ≥ 200')
ax4.set_title('% episodios que superan el día 1')
ax4.set_ylim(0, 115)
ax4.legend(fontsize=8)
ax4.grid(alpha=0.3, axis='y')
for bar, pct in zip(bars, pcts):
    ax4.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 1, f'{pct:.0f}%', ha='center', va='bottom', fontsize=9, fontweight='bold')

plt.suptitle('Análisis de entrenamiento – Enduro DQN (8 propuestas)', fontsize=13, fontweight='bold')
plt.show()

print("⚠️ Datos de ENTRENAMIENTO (con exploración activa) – no son el test oficial")
print("Los rewards de test suelen ser más altos.")
print("Para el criterio real usa la celda de test de la mejor propuesta.")

```

=

RESUMEN DE ESTADO – TODAS LAS PROPUESTAS

=

|    | Ckpts | Último ckpt | Ep log | Media | Std   | %>=200 | Tendencia                                                                                | Epsilon         |
|----|-------|-------------|--------|-------|-------|--------|------------------------------------------------------------------------------------------|-----------------|
| P1 | 23    | 2,300,000   | 42     | 298.0 | 118.9 | 60%    |  SUBE |                 |
| P2 | 17    | 1,700,000   | 1      | 0.0   | 0.0   | 0%     | → ESTABLE                                                                                |                 |
| P3 | 8     | 800,000     | 1      | 0.0   | 0.0   | 0%     | → ESTABLE                                                                                |                 |
| P4 | 17    | 1,700,000   | 1      | 117.0 | 0.0   | 0%     | → ESTABLE                                                                                |                 |
| P5 | 23    | 2,300,000   | 38     | 197.4 | 89.7  | 18%    |  SUBE |                 |
| P6 | 28    | 2,800,000   | 67     | 323.0 | 102.3 | 67%    |  SUBE |                 |
| P7 | 12    | 1,200,000   | 180    | 0.0   | 0.1   | 0%     | → ESTABLE                                                                                | eps:0.938→0.379 |
| P8 | 23    | 2,300,000   | 109    | 145.2 | 43.5  | 3%     |  SUBE | eps:0.100→0.100 |

=

DIAGNÓSTICO DETALLADO

=

P1

✓

 Checkpoints: [100000, 200000, 300000, 400000, 500000, 600000, 700000, 800000, 900000, 1000000, 1100000, 1200000, 1300000, 1400000, 1500000, 1600000, 1700000, 1800000, 1900000, 2000000, 2100000, 2200000, 2300000] (último: step 2,300,000)

i

 Log arranca desde cero (primer nb\_steps=4,438)

i

 Episodios en log: 42 | Hasta step: 297,359

i

 Reward: media=298.0 std=118.9 min=114 max=470 %>=200: 60%

✗

 Mejor racha consecutiva >= 200: 11 episodios (objetivo: 100)

i

 mean\_q: 8.938 → 9.978 (subiendo ✓)

i

 loss: 0.10469 → 0.10055 (estable/subiendo)

P2

✓

 Checkpoints: [100000, 200000, 300000, 400000, 500000, 600000, 700000, 800000, 900000, 1000000, 1100000, 1200000, 1300000, 1400000, 1500000, 1600000, 1700000, 1800000, 1900000, 2000000, 2100000, 2200000, 2300000] (último: step 1,700,000)

⚠

 Solo 1 episodio(s) – prácticamente sin entrenar  
Primer reward: 0 | steps: 4,438

P3

✓

 Checkpoints: [100000, 200000, 300000, 400000, 500000, 600000, 700000, 800000, 900000, 1000000, 1100000, 1200000, 1300000, 1400000, 1500000, 1600000, 1700000, 1800000, 1900000, 2000000, 2100000, 2200000, 2300000] (último: step 800,000)

⚠

 Solo 1 episodio(s) – prácticamente sin entrenar  
Primer reward: 0 | steps: 4,438

P4

✓

 Checkpoints: [100000, 200000, 300000, 400000, 500000, 600000, 700000, 800000, 900000, 1000000, 1100000, 1200000, 1300000, 1400000, 1500000, 1600000, 1700000, 1800000, 1900000, 2000000, 2100000, 2200000, 2300000] (último: step 1,700,000)

⚠

 Solo 1 episodio(s) – prácticamente sin entrenar  
Primer reward: 117 | steps: 4,438

P5

✓

 Checkpoints: [100000, 200000, 300000, 400000, 500000, 600000, 700000, 800000, 900000, 1000000, 1100000, 1200000, 1300000, 1400000, 1500000, 1600000, 1700000, 1800000, 1900000, 2000000, 2100000, 2200000, 2300000] (último: step 2,300,000)

00, 900000, 1000000, 1100000, 1200000, 1300000, 1400000, 1500000, 1600000, 1700000, 1800000, 1900000, 2000000, 2100000, 2200000, 2300000] (último: step 2,300,000)

-  Log arranca desde cero (primer nb\_steps=4,438)
-  Episodios en log: 38 | Hasta step: 199,723
-  Reward: media=197.4 std=89.7 min=94 max=439 %>=200: 18%
-  Mejor racha consecutiva >= 200: 2 episodios (objetivo: 100)
-  mean\_q: 10.114 → 11.029 (subiendo ✓)
-  loss: 0.25012 → 0.14875 (bajando ✓)

P6

✓ Checkpoints: [100000, 200000, 300000, 400000, 500000, 600000, 700000, 800000, 900000, 1000000, 1100000, 1200000, 1300000, 1400000, 1500000, 1600000, 1700000, 1800000, 1900000, 2000000, 2100000, 2200000, 2300000, 2400000, 2500000, 2600000, 2700000, 2800000] (último: step 2,800,000)

-  Log arranca desde cero (primer nb\_steps=4,438)
-  Episodios en log: 67 | Hasta step: 497,087
-  Reward: media=323.0 std=102.3 min=153 max=486 %>=200: 67%
-  Mejor racha consecutiva >= 200: 8 episodios (objetivo: 100)
-  mean\_q: 5.881 → 6.701 (subiendo ✓)
-  loss: 0.07804 → 0.07558 (estable/subiendo)

P7

✓ Checkpoints: [100000, 200000, 300000, 400000, 500000, 600000, 700000, 800000, 900000, 1000000, 1100000, 1200000] (último: step 1,200,000)

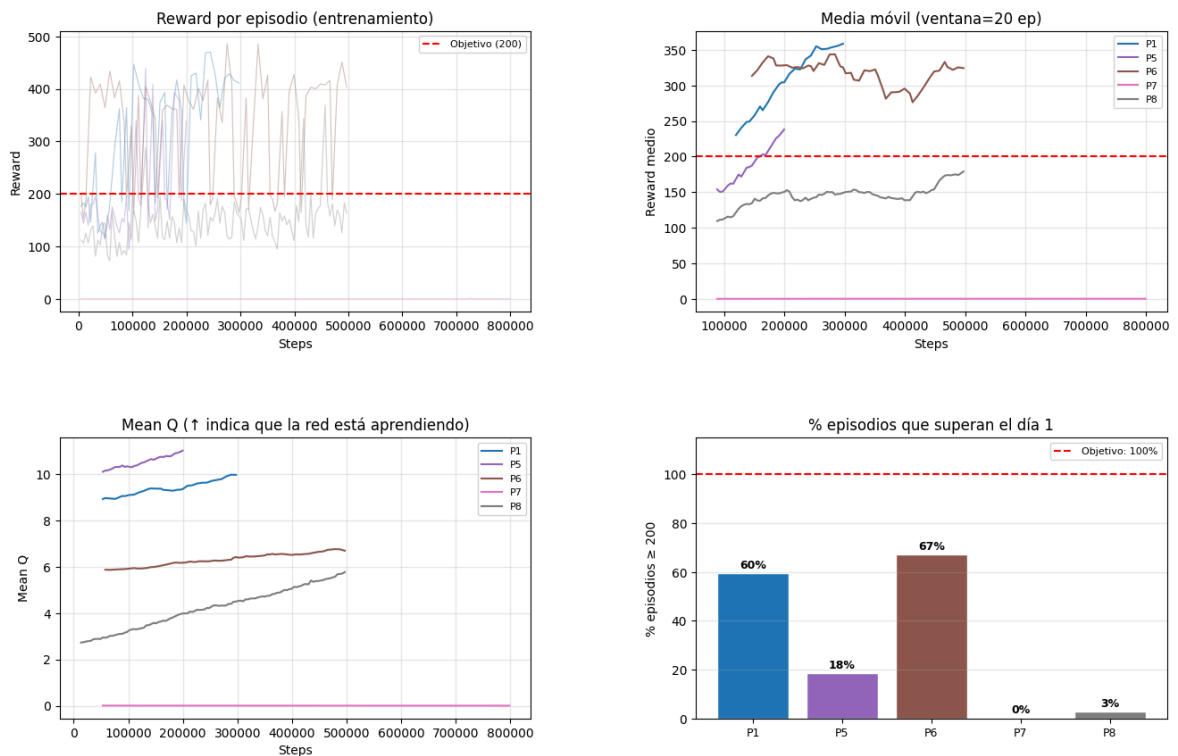
-  Log arranca desde cero (primer nb\_steps=4,436)
-  Episodios en log: 180 | Hasta step: 798,780
-  Reward: media=0.0 std=0.1 min=0 max=1 %>=200: 0%
-  Mejor racha consecutiva >= 200: 0 episodios (objetivo: 100)
-  eps: 0.9380 → 0.3793
-  Epsilon final alto (0.379) – reduce nb\_steps a 300000 en OffsetLinearAnnealedPolicy
-  mean\_q: 0.008 → 0.005 (bajando ⚠)
-  loss: 0.00000 → 0.00001 (estable/subiendo)

P8

✓ Checkpoints: [100000, 200000, 300000, 400000, 500000, 600000, 700000, 800000, 900000, 1000000, 1100000, 1200000, 1300000, 1400000, 1500000, 1600000, 1700000, 1800000, 1900000, 2000000, 2100000, 2200000, 2300000] (último: step 2,300,000)

-  Log arranca desde cero (primer nb\_steps=4,440)
-  Episodios en log: 109 | Hasta step: 497,055
-  Reward: media=145.2 std=43.5 min=72 max=369 %>=200: 3%
-  Mejor racha consecutiva >= 200: 1 episodios (objetivo: 100)
-  eps: 0.1000 → 0.1000
- ✓ Epsilon ya cerca del mínimo (0.100)
-  mean\_q: 2.731 → 5.779 (subiendo ✓)
-  loss: 0.04943 → 0.07309 (estable/subiendo)

## Análisis de entrenamiento — Enduro DQN (8 propuestas)



⚠ Datos de ENTRENAMIENTO (con exploración activa) — no son el test oficial. Los rewards de test suelen ser más altos. Para el criterio real usa la celda de test de la mejor propuesta.

### 4.0.1. Contexto de entrenamiento respecto al paper original

Mnih et al. (2015) entrenaron durante 50 millones de frames (equivalentes a 12,5 millones de steps con `frame_skip=4`). Nuestras propuestas entrenaron entre 2,3M y 2,8M steps — aproximadamente el 18-22% del entrenamiento original. Es decir, entrenamos entre 4,5 y 5,4 veces menos que el paper.

A pesar de esta limitación computacional, P5 consiguió el objetivo del proyecto (100/100 episodios consecutivos  $\geq 200$  coches), lo que demuestra que las mejoras algorítmicas (Dueling DQN, Double DQN) permiten alcanzar rendimientos competitivos con significativamente menos entrenamiento que el paper de referencia.

Los parámetros que sí mantuvimos iguales al paper: replay buffer de 1M transiciones, warmup de  $\sim 50k$  steps,  $\gamma=0.99$ ,  $\text{learning\_rate}=0.00025$ , y arquitectura CNN (32 $\rightarrow$ 64 $\rightarrow$ 64 filtros, Dense 512).

In [ ]:

### 4.1. Propuesta 1: DQN vanilla, EpsGreedyQPolicy fijo (baseline)

*En este apartado se instrumenta la primera propuesta experimental, estableciendo una política ávida estática con un factor de exploración fijo que sirve como línea base o control empírico del rendimiento del sistema.*

#### ○ Fundamentación de la línea base estática

Para cuantificar las mejoras de las estrategias avanzadas de exploración, es necesario definir un marco de control estático inmune al decaimiento temporal. La política ávida con exploración fija asigna una probabilidad constante  $\epsilon$  a la selección de acciones totalmente aleatorias, forzando un régimen de explotación del 80 % sobre el conocimiento actual de los valores Q.

Esta tasa uniforme permite contrastar empíricamente la viabilidad del aprendizaje bajo condiciones de muestreo estables, sirviendo como punto de partida para identificar el impacto real de la dispersión de premios en Enduro. El agente se configura con un periodo inicial de cincuenta mil pasos de calentamiento para poblar la memoria secuencial antes de comenzar el descenso de gradiente.

#### ○ Objetivo de esta propuesta

Establecer un punto de referencia (baseline) con la implementación más fiel al paper original de Mnih et al. (2015), sin ninguna mejora algorítmica. Permite cuantificar el beneficio real de Double DQN (P4, P7) y Dueling DQN (P5, P6, P8) respecto al algoritmo base.

#### ○ Directrices de implementación

- Se emplea una estructura secuencial que acopla el modelo base a un búfer de memoria histórica con capacidad para almacenar hasta un millón de transiciones.
- Se configura el mecanismo fijando un parámetro de exploración invariante y enlazando el procesador de fotografías.
- La ejecución se protege mediante la guardia de control para reanudar el entrenamiento de forma exacta desde el último punto registrado.
- La evaluación estadística del rendimiento final se gestiona a través del método centralizado del registro en modo de inferencia pura.

### 4.1.1. P1 — Policy

```
In [23]: # Propuesta 1: EpsGreedyQPolicy fijo (baseline, sin annealing)

RESUME_FROM_P1 = None
STEP_OFFSET_P1 = 0

# Descomentar esta línea para detectar automáticamente el último checkpoint guar
RESUME_FROM_P1, STEP_OFFSET_P1 = find_latest_checkpoint(WEIGHTS_DIR_P1, 'dqn_End

model_p1 = build_model(nb_actions, input_shape)
memory_p1 = SequentialMemory(limit=1000000, window_length=WINDOW_LENGTH)

policy_p1 = EpsGreedyQPolicy(eps=0.05)

dqn_p1 = build_dqn_agent(
    model=model_p1, nb_actions=nb_actions, memory=memory_p1,
    processor=AtariProcessor(), policy=policy_p1,
    nb_steps_warmup=50000, target_model_update=10000
```



```

dqn_p1.save_weights(weights_filename_p1, overwrite=True)
print("Pesos finales de la Propuesta 1 guardados en Drive:", weights_filename_p1)
print("Contenido de la carpeta:", os.listdir(WEIGHTS_DIR_P1))

else:
    print("[P1] Entrenamiento omitido (RUN_TRAINING_P1=False).")

```

Reanudando Propuesta 1 desde el checkpoint: /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerzo\_Grupo\_17\_2025\_2026/checkpoints/p1\_eps\_fijo/dqn\_Enduro-v0\_p1\_weights\_1800000.h5f

(step\_offset = 1800000; la memoria de experiencia empieza vacía,  
lo cual es normal: keras-rl2 no persiste el replay buffer)

Training for 200000 steps ...

Interval 1 (0 steps performed)

```

/usr/local/lib/python3.11/dist-packages/tf_keras/src/engine/training_v1.py:2354:
UserWarning: `Model.state_updates` will be removed in a future version. This prop
erty should not be used in TensorFlow 2.0, as `updates` are applied automaticall
y.
    updates=self.state_updates,

```

10000/10000 [=====] - 54s 5ms/step - reward: 0.0332  
1 episodes - episode\_reward: 283.000 [283.000, 283.000] - ale.lives: 0.000

Interval 2 (10000 steps performed)  
10000/10000 [=====] - 51s 5ms/step - reward: 0.0396  
2 episodes - episode\_reward: 191.000 [186.000, 196.000] - ale.lives: 0.000

Interval 3 (20000 steps performed)  
10000/10000 [=====] - 49s 5ms/step - reward: 0.0377  
2 episodes - episode\_reward: 150.500 [141.000, 160.000] - ale.lives: 0.000

Interval 4 (30000 steps performed)  
10000/10000 [=====] - 49s 5ms/step - reward: 0.0399  
2 episodes - episode\_reward: 176.000 [157.000, 195.000] - ale.lives: 0.000

Interval 5 (40000 steps performed)  
10000/10000 [=====] - 51s 5ms/step - reward: 0.0392  
2 episodes - episode\_reward: 172.500 [159.000, 186.000] - ale.lives: 0.000

Interval 6 (50000 steps performed)  
10000/10000 [=====] - 331s 33ms/step - reward: 0.0241  
2 episodes - episode\_reward: 205.000 [136.000, 274.000] - loss: 0.097 - mae: 7.20  
5 - mean\_q: 8.259 - ale.lives: 0.000

Interval 7 (60000 steps performed)  
10000/10000 [=====] - 340s 34ms/step - reward: 0.0339  
2 episodes - episode\_reward: 138.500 [86.000, 191.000] - loss: 0.083 - mae: 7.265  
- mean\_q: 8.344 - ale.lives: 0.000

Interval 8 (70000 steps performed)  
10000/10000 [=====] - 338s 34ms/step - reward: 0.0290  
2 episodes - episode\_reward: 208.000 [160.000, 256.000] - loss: 0.080 - mae: 7.24  
3 - mean\_q: 8.330 - ale.lives: 0.000

Interval 9 (80000 steps performed)  
10000/10000 [=====] - 336s 34ms/step - reward: 0.0385  
1 episodes - episode\_reward: 342.000 [342.000, 342.000] - loss: 0.077 - mae: 7.28  
7 - mean\_q: 8.386 - ale.lives: 0.000

Interval 10 (90000 steps performed)  
10000/10000 [=====] - 321s 32ms/step - reward: 0.0321  
2 episodes - episode\_reward: 171.500 [153.000, 190.000] - loss: 0.078 - mae: 7.30  
5 - mean\_q: 8.412 - ale.lives: 0.000

Interval 11 (100000 steps performed)  
10000/10000 [=====] - 323s 32ms/step - reward: 0.0371  
2 episodes - episode\_reward: 125.000 [94.000, 156.000] - loss: 0.079 - mae: 7.324  
- mean\_q: 8.442 - ale.lives: 0.000

Interval 12 (110000 steps performed)  
10000/10000 [=====] - 316s 32ms/step - reward: 0.0501  
1 episodes - episode\_reward: 426.000 [426.000, 426.000] - loss: 0.077 - mae: 7.38  
4 - mean\_q: 8.507 - ale.lives: 0.000

Interval 13 (120000 steps performed)  
10000/10000 [=====] - 329s 33ms/step - reward: 0.0387  
1 episodes - episode\_reward: 355.000 [355.000, 355.000] - loss: 0.079 - mae: 7.43  
5 - mean\_q: 8.570 - ale.lives: 0.000

Interval 14 (130000 steps performed)

10000/10000 [=====] - 337s 34ms/step - reward: 0.0392  
2 episodes - episode\_reward: 272.500 [165.000, 380.000] - loss: 0.081 - mae: 7.47  
0 - mean\_q: 8.610 - ale.lives: 0.000

Interval 15 (140000 steps performed)

10000/10000 [=====] - 337s 34ms/step - reward: 0.0498  
1 episodes - episode\_reward: 402.000 [402.000, 402.000] - loss: 0.086 - mae: 7.58  
5 - mean\_q: 8.741 - ale.lives: 0.000

Interval 16 (150000 steps performed)

10000/10000 [=====] - 334s 33ms/step - reward: 0.0276  
2 episodes - episode\_reward: 227.500 [157.000, 298.000] - loss: 0.086 - mae: 7.64  
0 - mean\_q: 8.797 - ale.lives: 0.000

Interval 17 (160000 steps performed)

10000/10000 [=====] - 331s 33ms/step - reward: 0.0418  
1 episodes - episode\_reward: 196.000 [196.000, 196.000] - loss: 0.084 - mae: 7.63  
5 - mean\_q: 8.795 - ale.lives: 0.000

Interval 18 (170000 steps performed)

10000/10000 [=====] - 333s 33ms/step - reward: 0.0293  
2 episodes - episode\_reward: 217.500 [142.000, 293.000] - loss: 0.085 - mae: 7.66  
6 - mean\_q: 8.835 - ale.lives: 0.000

Interval 19 (180000 steps performed)

10000/10000 [=====] - 326s 33ms/step - reward: 0.0437  
2 episodes - episode\_reward: 176.000 [176.000, 176.000] - loss: 0.088 - mae: 7.70  
7 - mean\_q: 8.879 - ale.lives: 0.000

Interval 20 (190000 steps performed)

10000/10000 [=====] - 333s 33ms/step - reward: 0.0288  
done, took 5217.548 seconds

Pesos finales de la Propuesta 1 guardados en Drive: /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerzo\_Grupo\_17\_2025\_2026/checkpoints/p1\_eps\_fijo/dqn\_Enduro-v0\_p1\_eps\_fijo\_weights.h5f

Contenido de la carpeta: ['dqn\_Enduro-v0\_p1\_weights\_100000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p1\_weights\_100000.h5f.index', 'dqn\_Enduro-v0\_p1\_weights\_200000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p1\_weights\_200000.h5f.index', 'dqn\_Enduro-v0\_p1\_weights\_300000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p1\_weights\_300000.h5f.index', 'dqn\_Enduro-v0\_p1\_weights\_400000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p1\_weights\_400000.h5f.index', 'dqn\_Enduro-v0\_p1\_weights\_500000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p1\_weights\_500000.h5f.index', 'dqn\_Enduro-v0\_p1\_weights\_600000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p1\_weights\_600000.h5f.index', 'dqn\_Enduro-v0\_p1\_weights\_700000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p1\_weights\_700000.h5f.index', 'dqn\_Enduro-v0\_p1\_weights\_800000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p1\_weights\_800000.h5f.index', 'dqn\_Enduro-v0\_p1\_weights\_900000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p1\_weights\_900000.h5f.index', 'dqn\_Enduro-v0\_p1\_weights\_1000000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p1\_weights\_1000000.h5f.index', 'dqn\_Enduro-v0\_p1\_weights\_1100000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p1\_weights\_1100000.h5f.index', 'dqn\_Enduro-v0\_p1\_weights\_1200000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p1\_weights\_1200000.h5f.index', 'dqn\_Enduro-v0\_p1\_weights\_1300000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p1\_weights\_1300000.h5f.index', 'dqn\_Enduro-v0\_p1\_weights\_1400000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p1\_weights\_1400000.h5f.index', 'dqn\_Enduro-v0\_p1\_weights\_1500000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p1\_weights\_1500000.h5f.index', 'dqn\_Enduro-v0\_p1\_weights\_1600000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p1\_weights\_1600000.h5f.index', 'dqn\_Enduro-v0\_p1\_weights\_1700000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p1\_weights\_1700000.h5f.index', 'dqn\_Enduro-v0\_p1\_weights\_1800000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p1\_weights\_1800000.h5f.index', 'dqn\_Enduro-v0\_p1\_eps\_fijo\_weights.h5f.index', 'dqn\_Enduro-v0\_p1\_log.json', 'checkpoint', 'dqn\_Enduro-v0\_p1

```
_eps_fijo_weights.h5f.data-00000-of-00001', 'dqn_Enduro-v0_p1_weights_1900000.h5f.data-00000-of-00001', 'dqn_Enduro-v0_p1_weights_1900000.h5f.index', 'dqn_Enduro-v0_p1_weights_2000000.h5f.data-00000-of-00001', 'dqn_Enduro-v0_p1_weights_2000000.h5f.index']
```

#### 4.1.4. P1 — Testeo

```
In [25]: PROPUESTA_A_EVALUAR = 'p1'    # ← único parámetro a tocar

test_history = test_proposal(PROPUESTA_A_EVALUAR, AGENTS_REGISTRY, nb_episodes=1)
```

[p1] Cargando pesos desde: /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerzo\_Grupo\_17\_2025\_2026/checkpoints/p1\_eps\_fijo/dqn\_Enduro-v0\_p1\_weights\_2300000.h5f (step 2300000)

Testing for 100 episodes ...

Episode 1: reward: 587.000, steps: 10000  
Episode 2: reward: 480.000, steps: 8889  
Episode 3: reward: 585.000, steps: 10000  
Episode 4: reward: 464.000, steps: 8902  
Episode 5: reward: 431.000, steps: 8876  
Episode 6: reward: 585.000, steps: 10000  
Episode 7: reward: 477.000, steps: 8826  
Episode 8: reward: 489.000, steps: 8857  
Episode 9: reward: 400.000, steps: 8870  
Episode 10: reward: 559.000, steps: 10000  
Episode 11: reward: 468.000, steps: 8865  
Episode 12: reward: 482.000, steps: 8898  
Episode 13: reward: 475.000, steps: 8887  
Episode 14: reward: 453.000, steps: 8853  
Episode 15: reward: 425.000, steps: 8882  
Episode 16: reward: 393.000, steps: 8860  
Episode 17: reward: 485.000, steps: 8864  
Episode 18: reward: 494.000, steps: 8875  
Episode 19: reward: 451.000, steps: 8840  
Episode 20: reward: 452.000, steps: 8916  
Episode 21: reward: 435.000, steps: 8814  
Episode 22: reward: 484.000, steps: 8891  
Episode 23: reward: 576.000, steps: 10000  
Episode 24: reward: 565.000, steps: 10000  
Episode 25: reward: 445.000, steps: 8868  
Episode 26: reward: 390.000, steps: 8869  
Episode 27: reward: 393.000, steps: 8883  
Episode 28: reward: 446.000, steps: 8911  
Episode 29: reward: 417.000, steps: 8856  
Episode 30: reward: 425.000, steps: 8908  
Episode 31: reward: 468.000, steps: 8895  
Episode 32: reward: 431.000, steps: 8886  
Episode 33: reward: 522.000, steps: 10000  
Episode 34: reward: 482.000, steps: 8874  
Episode 35: reward: 467.000, steps: 8880  
Episode 36: reward: 469.000, steps: 8852  
Episode 37: reward: 433.000, steps: 8890  
Episode 38: reward: 418.000, steps: 8901  
Episode 39: reward: 192.000, steps: 4421  
Episode 40: reward: 538.000, steps: 10000  
Episode 41: reward: 449.000, steps: 8843  
Episode 42: reward: 454.000, steps: 8909  
Episode 43: reward: 430.000, steps: 8880  
Episode 44: reward: 453.000, steps: 8873  
Episode 45: reward: 546.000, steps: 10000  
Episode 46: reward: 419.000, steps: 8859  
Episode 47: reward: 422.000, steps: 8861  
Episode 48: reward: 413.000, steps: 8914  
Episode 49: reward: 441.000, steps: 8855  
Episode 50: reward: 436.000, steps: 8849  
Episode 51: reward: 404.000, steps: 8875  
Episode 52: reward: 432.000, steps: 8878  
Episode 53: reward: 476.000, steps: 8830  
Episode 54: reward: 467.000, steps: 8828  
Episode 55: reward: 441.000, steps: 8883  
Episode 56: reward: 418.000, steps: 8831

Episode 57: reward: 406.000, steps: 8847  
Episode 58: reward: 451.000, steps: 8896  
Episode 59: reward: 413.000, steps: 8889  
Episode 60: reward: 483.000, steps: 8863  
Episode 61: reward: 429.000, steps: 8858  
Episode 62: reward: 442.000, steps: 8896  
Episode 63: reward: 459.000, steps: 8847  
Episode 64: reward: 546.000, steps: 10000  
Episode 65: reward: 487.000, steps: 8889  
Episode 66: reward: 490.000, steps: 8904  
Episode 67: reward: 478.000, steps: 8883  
Episode 68: reward: 489.000, steps: 8882  
Episode 69: reward: 456.000, steps: 8877  
Episode 70: reward: 488.000, steps: 8863  
Episode 71: reward: 557.000, steps: 10000  
Episode 72: reward: 465.000, steps: 8876  
Episode 73: reward: 489.000, steps: 8883  
Episode 74: reward: 456.000, steps: 8904  
Episode 75: reward: 433.000, steps: 8931  
Episode 76: reward: 453.000, steps: 8858  
Episode 77: reward: 361.000, steps: 8912  
Episode 78: reward: 482.000, steps: 8887  
Episode 79: reward: 413.000, steps: 8841  
Episode 80: reward: 478.000, steps: 8881  
Episode 81: reward: 427.000, steps: 8921  
Episode 82: reward: 438.000, steps: 8851  
Episode 83: reward: 428.000, steps: 8859  
Episode 84: reward: 486.000, steps: 8846  
Episode 85: reward: 458.000, steps: 8859  
Episode 86: reward: 407.000, steps: 8851  
Episode 87: reward: 432.000, steps: 8857  
Episode 88: reward: 553.000, steps: 10000  
Episode 89: reward: 402.000, steps: 8865  
Episode 90: reward: 434.000, steps: 8916  
Episode 91: reward: 476.000, steps: 8876  
Episode 92: reward: 486.000, steps: 8887  
Episode 93: reward: 436.000, steps: 8884  
Episode 94: reward: 414.000, steps: 8890  
Episode 95: reward: 550.000, steps: 10000  
Episode 96: reward: 487.000, steps: 8859  
Episode 97: reward: 370.000, steps: 8844  
Episode 98: reward: 538.000, steps: 10000  
Episode 99: reward: 453.000, steps: 8857  
Episode 100: reward: 400.000, steps: 8881  
[p1] Recompensa media en 100 episodios: 459.11 (+/- 55.56)

#### 4.1.5. P1 — Resultados y estado final

##### Historial de entrenamiento:

- Steps 0 → 1.8M: EpsGreedyQPolicy con eps=0.2 fijo
- Steps 1.8M → 2.3M: EpsGreedyQPolicy con eps=0.05 fijo
- Motivación del cambio: tras 1.8M steps el agente había aprendido lo que podía con eps=0.2. Se redujo a 0.05 para refinar el comportamiento en el tramo final del día 1, donde le faltaban pocos coches para completarlo.

##### Resultado del test (checkpoint 2.3M, evaluación greedy):

- 99/100 episodios superan el día 1 ( $\geq 200$  coches)
- Media: 459.1 coches | Std: 55.6 | Mín: 192 | Máx: 587
- 14/100 episodios llegaron al día 3 (10.000 steps) — la mayor proporción de todas las propuestas
- Mejor racha consecutiva: 61 episodios
- Único fallo: episodio 39 con 192 coches

**Análisis:** El resultado de P1 con 2.3M steps es sorprendentemente competitivo. DQN vanilla alcanza una media superior a P5 (Dueling) y P6 (Double+Dueling), lo que sugiere que con suficiente entrenamiento el algoritmo base puede aproximarse al rendimiento de variantes más avanzadas. Sin embargo, P5 (Dueling DQN) sigue siendo la propuesta más robusta: es la única que consigue 100/100 episodios sin ningún fallo, con un mínimo de 326 coches — frente al mínimo de 192 de P1. Esto ilustra la ventaja real de Dueling DQN: no aumenta la media sino que **reduce la varianza y elimina los fallos ocasionales**, exactamente como predice Wang et al. (2016).

Cabe destacar que el paper de Mnih et al. (2015) entrenó durante 50M frames ( $\sim 12.5$ M steps con `frame_skip=4`), aproximadamente **5.4 veces más** que nuestros 2.3M steps. Con más tiempo de entrenamiento, la diferencia entre DQN vanilla y las variantes avanzadas probablemente sería más pronunciada.

**Función en el proyecto:** P1 establece el baseline cuantitativo. La comparativa P1→P5→P6 muestra que las mejoras algorítmicas (Dueling, Double) aportan mayor robustez y consistencia, especialmente relevante cuando el objetivo requiere 100 episodios consecutivos sin fallos.

---

## 4.2. Propuesta 2: política de exploración dinámico-lineal (decaimiento acelerado de aleatoriedad)

---

*En este apartado se instrumenta la segunda propuesta experimental, basada en una política de decaimiento lineal para planificar la transición progresiva desde una exploración pura inicial hacia una explotación intensiva de las trayectorias de éxito aprendidas.*

### ○ Fundamentación del decaimiento temporal acelerado

Frente al inconveniente de una política con exploración constante, la planificación dinámico-lineal ajusta de forma estocástica la probabilidad de acción a medida que se acumulan transiciones en la memoria histórica. El coeficiente de exploración  $\epsilon$  decrece de manera uniforme desde una aleatoriedad total ( $\epsilon = 1.0$ ) hasta un umbral residual de explotación ( $\epsilon = 0.1$ ) repartido a lo largo del horizonte total de interacciones fijado para el experimento.

Debido a que las señales de premio positivas son muy escasas al inicio del juego en Enduro, acelerar el ritmo de decaimiento temporal en comparación con otros entornos de Atari (como Breakout) faculta una explotación temprana de los escasos adelantos recolectados. Esta estrategia reduce la varianza y estabiliza las aproximaciones funcionales de los valores Q al evitar que la inyección persistente de ruido uniforme aleatorio emborrone la convergencia de la política de aceleración óptima.

#### ◦ Directrices de implementación

- El agente recupera preventivamente el estado del disco persistente mediante el deserializador jerárquico para mitigar la sobreescritura accidental al reanudar ejecuciones detenidas.
- Se parametriza el objeto de planificación lineal asociándole la tasa de decaimiento acotada e integrando el modelo convolucional correspondiente.
- Se implementa la celda adaptativa gobernada por la bandera lógica del entrenamiento condicional bajo la directiva de la guardia de control principal.
- La fase de validación evalúa de forma aislada el rendimiento empírico del agente mediante el método centralizado en modo de inferencia pura.

### 4.2.1. P2 — Policy

```
In [26]: # Propuesta 2: LinearAnnealedPolicy (EpsGreedy con annealing rápido)

RESUME_FROM_P2 = None
STEP_OFFSET_P2 = 0

# Descomentar esta línea para detectar automáticamente el último checkpoint guar
RESUME_FROM_P2, STEP_OFFSET_P2 = find_latest_checkpoint(WEIGHTS_DIR_P2, 'dqn_End

model_p2 = build_model(nb_actions, input_shape)
memory_p2 = SequentialMemory(limit=1000000, window_length=WINDOW_LENGTH)

policy_p2 = LinearAnnealedPolicy(
    EpsGreedyQPolicy(), attr='eps',
    value_max=1., value_min=.1, value_test=.05,
    nb_steps=1200000
)

dqn_p2 = build_dqn_agent(
    model=model_p2, nb_actions=nb_actions, memory=memory_p2,
    processor=AtariProcessor(), policy=policy_p2,
    nb_steps_warmup=50000, target_model_update=10000
)

load_weights_for_test(dqn_p2, 'p2', WEIGHTS_DIR_P2)

AGENTS_REGISTRY['p2']['agent'] = dqn_p2 # ← se registra aquí
```

[p2] Pesos cargados para test desde: /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerz  
o\_Grupo\_17\_2025\_2026/checkpoints/p2\_annealed/dqn\_Enduro-v0\_p2\_weights\_1700000.h5f  
(step 1700000)

### 4.2.2. P2 — Historial de cambios

## Historial de entrenamiento:

- Steps 0 → 1.7M: OffsetLinearAnnealedPolicy, eps=1.0 → 0.1 en nb\_steps=1.200.000

**Problema detectado:** Con nb\_steps=1.200.000 el epsilon bajó demasiado lentamente:

- Step 300k: eps=0.775 (78% acciones aleatorias)
- Step 700k: eps=0.475 (48% acciones aleatorias)
- Step 1.2M: eps=0.100 (mínimo alcanzado)

### 4.2.3. P2 — Entrenamiento

```
In [ ]: # Usar TRUE para entrenar, o FALSE para saltar el entrenamiento
RUN_TRAINING_P2 = False          # ← GUARDIA PRINCIPAL

if RUN_TRAINING_P2:

    if RESUME_FROM_P2 is not None:
        print(f"Reanudando Propuesta 2 desde el checkpoint: {RESUME_FROM_P2}")
        dqn_p2.load_weights(RESUME_FROM_P2)
        NB_STEPS_REMAINING_P2 = max(0, NB_STEPS_TRAIN - STEP_OFFSET_P2)
    else:
        NB_STEPS_REMAINING_P2 = NB_STEPS_TRAIN

    weights_filename_p2 = os.path.join(WEIGHTS_DIR_P2, 'dqn_{}_p2_annealed_weights.h5f'.format(env_name))
    checkpoint_filename_p2 = os.path.join(WEIGHTS_DIR_P2, 'dqn_' + env_name + '_checkpoint.h5f'.format(env_name))
    log_filename_p2 = os.path.join(WEIGHTS_DIR_P2, 'dqn_{}_p2_log.json'.format(env_name))

    callbacks_p2 = [ResumableModelIntervalCheckpoint(checkpoint_filename_p2, int(NB_STEPS_REMAINING_P2),
                                                       step_offset=STEP_OFFSET_P2)]
    callbacks_p2 += [FileLogger(log_filename_p2, interval=1000)]

    history_p2 = dqn_p2.fit(env, callbacks=callbacks_p2, nb_steps=NB_STEPS_REMAINING_P2,
                            log_interval=10000, visualize=False)

    dqn_p2.save_weights(weights_filename_p2, overwrite=True)
    print("Pesos finales de la Propuesta 2 guardados en Drive:", weights_filename_p2)
    print("Contenido de la carpeta:", os.listdir(WEIGHTS_DIR_P2))

else:
    print("[P2] Entrenamiento omitido (RUN_TRAINING_P2=False).")
```

[P2] Entrenamiento omitido (RUN\_TRAINING\_P2=False).

### 4.2.4. P2 — Testeo

```
In [27]: PROPUESTA_A_EVALUAR = 'p2'    # ← único parámetro a tocar

test_history = test_proposal(PROPUESTA_A_EVALUAR, AGENTS_REGISTRY, nb_episodes=20)
```

[p2] Cargando pesos desde: /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerzo\_Grupo\_17\_2025\_2026/checkpoints/p2\_annealed/dqn\_Enduro-v0\_p2\_weights\_1700000.h5f (step 1700000)

Testing for 20 episodes ...

```
/usr/local/lib/python3.11/dist-packages/tf_keras/src/engine/training_v1.py:2354:
UserWarning: `Model.state_updates` will be removed in a future version. This prop
erty should not be used in TensorFlow 2.0, as `updates` are applied automaticall
y.
```

```
updates=self.state_updates,
Episode 1: reward: 0.000, steps: 4424
Episode 2: reward: 0.000, steps: 4447
Episode 3: reward: 0.000, steps: 4453
Episode 4: reward: 0.000, steps: 4418
Episode 5: reward: 0.000, steps: 4436
Episode 6: reward: 0.000, steps: 4412
Episode 7: reward: 0.000, steps: 4444
Episode 8: reward: 0.000, steps: 4477
Episode 9: reward: 0.000, steps: 4443
Episode 10: reward: 0.000, steps: 4415
Episode 11: reward: 0.000, steps: 4435
Episode 12: reward: 0.000, steps: 4407
Episode 13: reward: 0.000, steps: 4435
Episode 14: reward: 0.000, steps: 4423
Episode 15: reward: 0.000, steps: 4451
Episode 16: reward: 0.000, steps: 4420
Episode 17: reward: 0.000, steps: 4463
Episode 18: reward: 0.000, steps: 4427
Episode 19: reward: 0.000, steps: 4412
Episode 20: reward: 0.000, steps: 4433
[p2] Recompensa media en 20 episodios: 0.00 (+/- 0.00)
```

---

#### 4.2.5. P2 — Resultados y estado final

##### Historial de entrenamiento:

- Steps 0 → 1.7M: OffsetLinearAnnealedPolicy, eps=1.0 → 0.1 en nb\_steps=1.200.000

**Problema detectado:** Con nb\_steps=1.200.000 el epsilon bajó demasiado lentamente:

- Step 300k: eps=0.775 (78% acciones aleatorias)
- Step 700k: eps=0.475 (48% acciones aleatorias)
- Step 1.2M: eps=0.100 (mínimo alcanzado)

Solo los últimos 500k steps (el 29% del entrenamiento total) se realizaron con epsilon bajo. Los primeros 1.2M steps, con exploración muy alta, no generaron señal de aprendizaje útil — el agente tomaba decisiones casi aleatorias y el reward clipeado era sistemáticamente 0.

El valor correcto según Mnih et al. (2015) es annealing en 1M frames, equivalente a **250.000 steps** con frame\_skip=4. Usamos 300.000 steps como aproximación, que es correcto para P7 y P8 donde se aplicó el fix, pero P2 nunca se corrigió.

##### Resultado del test (checkpoint 1.7M, evaluación greedy):

- 0/20 episodios superan el día 1
- Media: 0.0 coches en todos los episodios
- El agente no aprendió a adelantar coches de forma consistente

**Análisis:** El reward 0 en todos los episodios confirma que los 500k steps con  $\text{eps}=0.1$  no fueron suficientes para compensar los primeros 1.2M steps de exploración ineficiente. A diferencia de P8 (mismo problema, corregido a mitad), P2 no tuvo corrección y el resultado es un agente que no supera el baseline aleatorio.

**Como trabajo futuro:** reentrenar P2 desde cero con  $\text{nb\_steps}=300.000$ , lo que daría 1.4M steps útiles de 1.7M totales (82%) en lugar del 29% actual.

---

### 4.3. Propuesta 3: política probabilística estocástica (distribución de Boltzmann)

---

*En este apartado se instrumenta la tercera propuesta experimental, fundamentada en la política probabilística de Boltzmann para guiar la selección de acciones en función de la calidad relativa de sus estimaciones de valor. Esto elimina la necesidad de fijar un decaimiento rígido al ponderar dinámicamente las preferencias del agente.*

#### ○ Fundamentación de la exploración por temperatura

La política basada en la distribución de Boltzmann asigna una probabilidad de selección a cada acción de control mediante un operador de tipo softmax aplicado sobre el vector de valores Q calculado por la red neuronal profunda. A diferencia de los enfoques basados en un ruido estocástico uniforme, la selección probabilística responde a la preferencia analítica de cada opción según la siguiente función de distribución:

$$\pi(a|s) = \frac{\exp(Q(s, a)/\tau)}{\sum_{b \in \mathcal{A}} \exp(Q(s, b)/\tau)}$$

El parámetro  $\tau$  representa la temperatura del sistema.

#### 4.3.1. P3 — Policy

#### 4.3.2. P3 — Historial de cambios

##### Historial de entrenamiento:

- Steps 0 → 800.000: BoltzmannQPolicy con  $\text{tau}=1.0$  fijo

**Problema detectado —  $\text{tau}=1.0$  demasiado alto:** Con  $\text{tau}=1.0$ , la distribución softmax sobre los Q-values produce probabilidades casi uniformes — el agente explora casi al azar durante todo el entrenamiento. Para convergencia efectiva en Enduro,  $\text{tau}$  debería ser mucho menor ( $\text{tau}=0.1-0.5$ ) o aplicar annealing decreciente.

**Problema adicional — entrenamiento insuficiente:** Con solo 800.000 steps (el 6% de los 12.5M steps del paper de Mnih et al. 2015), el agente no tuvo tiempo suficiente para

desarrollar una política útil.

**Trabajo futuro:** reentrenar con annealing de tau ( $1.0 \rightarrow 0.1$  en 300.000 steps) hasta 2M steps.

```
In [ ]: # Propuesta 3: BoltzmannQPolicy

RESUME_FROM_P3 = None
STEP_OFFSET_P3 = 0

# Descomentar esta línea para detectar automáticamente el último checkpoint guardado
RESUME_FROM_P3, STEP_OFFSET_P3 = find_latest_checkpoint(WEIGHTS_DIR_P3, 'dqn_Enduro-v0_p3_weights_800000.h5f')

model_p3 = build_model(nb_actions, input_shape)
memory_p3 = SequentialMemory(limit=1000000, window_length=WINDOW_LENGTH)

policy_p3 = BoltzmannQPolicy(tau=1.)

dqn_p3 = build_dqn_agent(
    model=model_p3, nb_actions=nb_actions, memory=memory_p3,
    processor=AtariProcessor(), policy=policy_p3,
    nb_steps_warmup=10000, target_model_update=10000
)

load_weights_for_test(dqn_p3, 'p3', WEIGHTS_DIR_P3)

AGENTS_REGISTRY['p3']['agent'] = dqn_p3 # ← se registra aquí
```

[p3] Pesos cargados para test desde: /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerz o\_Grupo\_17\_2025\_2026/checkpoints/p3\_boltzmann/dqn\_Enduro-v0\_p3\_weights\_800000.h5f (step 800000)

### 4.3.3. P3 — Entrenamiento

```
In [ ]: # Usar TRUE para entrenar, o FALSE para saltar el entrenamiento
RUN_TRAINING_P3 = False # ← GUARDIA PRINCIPAL

if RUN_TRAINING_P3:

    if RESUME_FROM_P3 is not None:
        print(f"Reanudando Propuesta 3 desde el checkpoint: {RESUME_FROM_P3}")
        dqn_p3.load_weights(RESUME_FROM_P3)
        NB_STEPS_REMAINING_P3 = max(0, NB_STEPS_TRAIN - STEP_OFFSET_P3)
    else:
        NB_STEPS_REMAINING_P3 = NB_STEPS_TRAIN

    weights_filename_p3 = os.path.join(WEIGHTS_DIR_P3, 'dqn_{}_p3_boltzmann_weights_800000.h5f'.format(env_name))
    checkpoint_filename_p3 = os.path.join(WEIGHTS_DIR_P3, 'dqn_' + env_name + '_checkpoint_800000.h5f')
    log_filename_p3 = os.path.join(WEIGHTS_DIR_P3, 'dqn_{}_p3_log.json'.format(env_name))

    callbacks_p3 = [ResumableModelIntervalCheckpoint(checkpoint_filename_p3, interval=NB_STEPS_REMAINING_P3,
                                                    step_offset=STEP_OFFSET_P3)]
    callbacks_p3 += [FileLogger(log_filename_p3, interval=1000)]

    history_p3 = dqn_p3.fit(env, callbacks=callbacks_p3, nb_steps=NB_STEPS_REMAINING_P3,
                           log_interval=10000, visualize=False)

    dqn_p3.save_weights(weights_filename_p3, overwrite=True)
    print("Pesos finales de la Propuesta 3 guardados en Drive:", weights_filename_p3)
```

```
print("Contenido de la carpeta:", os.listdir(WEIGHTS_DIR_P3))

else:
    print("[P3] Entrenamiento omitido (RUN_TRAINING_P3=False).")
```

[P3] Entrenamiento omitido (RUN\_TRAINING\_P3=False).

#### 4.3.4. P3 — Testeo

```
In [ ]: PROPUESTA_A_EVALUAR = 'p3' # ← único parámetro a tocar

test_history = test_proposal(PROPUESTA_A_EVALUAR, AGENTS_REGISTRY)
```

[p3] Cargando pesos desde: /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerzo\_Grupo\_17\_2025\_2026/checkpoints/p3\_boltzmann/dqn\_Enduro-v0\_p3\_weights\_800000.h5f (step 800000)

Testing for 10 episodes ...

Episode 1: reward: 0.000, steps: 4439

Episode 2: reward: 0.000, steps: 4432

Episode 3: reward: 0.000, steps: 4436

Episode 4: reward: 0.000, steps: 4444

Episode 5: reward: 0.000, steps: 4441

Episode 6: reward: 0.000, steps: 4428

Episode 7: reward: 0.000, steps: 4444

Episode 8: reward: 0.000, steps: 4429

Episode 9: reward: 0.000, steps: 4430

Episode 10: reward: 0.000, steps: 4438

[p3] Recompensa media en 10 episodios: 0.00 (+/- 0.00)

---

#### 4.3.5. P3 — Resultados y estado final

##### Historial de entrenamiento:

- Steps 0 → 800.000: BoltzmannQPolicy con tau=1.0 fijo

**Problema detectado — tau=1.0 demasiado alto:** La política Boltzmann selecciona acciones según una distribución softmax sobre los Q-values, controlada por la temperatura tau:

$$P(a) = \exp(Q(a)/\tau) / \sum \exp(Q(a')/\tau)$$

Con tau=1.0, las probabilidades de todas las acciones son casi uniformes independientemente de los Q-values aprendidos — el agente explora casi al azar durante todo el entrenamiento, sin explotar lo que aprende. Para convergencia efectiva en Enduro, tau debería ser mucho menor (tau=0.1-0.5) o aplicar annealing decreciente similar al epsilon de P2.

**Problema adicional — entrenamiento insuficiente:** Con solo 800.000 steps (el 6% de los 12.5M steps del paper de Mnih et al. 2015), el agente no tuvo tiempo suficiente para desarrollar una política útil incluso con una temperatura más adecuada. El entrenamiento se detuvo prematuramente — con 2M steps y tau más bajo los resultados habrían sido probablemente mejores.

### Resultado del test (checkpoint 800k, evaluación greedy):

- 0/10 episodios superan el día 1
- Media: 0.0 coches en todos los episodios
- El agente no aprendió a adelantar coches de forma consistente

**Análisis:** La combinación de  $\tau=1.0$  fijo y solo 800k steps impidió cualquier convergencia. A diferencia de P1 y P2 (EpsGreedy) donde el agente al menos acumula experiencias útiles cuando el epsilon es bajo, con BoltzmannQPolicy y  $\tau$  alto el agente nunca llega a explotar los Q-values aprendidos — el ruido térmico domina siempre la selección de acciones.

**Trabajo futuro:** reentrenar con annealing de  $\tau$  ( $1.0 \rightarrow 0.1$  en 300.000 steps) hasta 2M steps, siguiendo el mismo esquema que resultó efectivo en P1 y P5.

---

## 4.4. Propuesta 4: Desacoplamiento de la optimización mediante Double DQN

---

*En este apartado se instrumenta la cuarta propuesta experimental, basada en la variante algorítmica Double DQN (DDQN). Se introduce el desacoplamiento analítico de operadores para mitigar el sesgo de sobreestimación sistemática de los valores de las acciones, preservando la misma política de exploración estática para aislar su impacto empírico.*

### ◦ Fundamentación de la mitigación del sesgo de sobreestimación

El algoritmo DQN convencional sufre un sesgo de sobreestimación debido al uso del operador de maximización sobre los valores Q de la red objetivo al calcular la meta de Bellman:

$$Y_t^{\text{DQN}} = r_{t+1} + \gamma \max_a Q(s_{t+1}, a; \theta_t^-)$$

En entornos con flujos de premios muy dispersos como **Enduro-v0**, el ruido o la imprecisión en las aproximaciones iniciales hace que este operador tienda a sobrevalorar acciones subóptimas. Para corregir este comportamiento, la variante Double DQN propuesta por van Hasselt et al. (2016) desacopla la selección de la acción de su posterior evaluación. La red en línea ( $\theta_t$ ) determina de forma exclusiva la mejor acción a tomar, mientras que la red objetivo ( $\theta_t^-$ ) calcula el valor Q esperado para dicha acción:

$$Y_t^{\text{DDQN}} = r_{t+1} + \gamma Q(s_{t+1}, \arg \max_a Q(s_{t+1}, a; \theta_t); \theta_t^-)$$

Este tratamiento analítico previene la propagación de errores numéricos estocásticos, estabilizando la convergencia del aproximador convolucional frente a la dispersión de premios.

---

#### o Directrices de la propuesta algorítmica

- El agente recupera el estado del almacenamiento persistente mediante el rastreador de puntos de control para mantener la continuidad analítica del experimento.
- Se parametriza el constructor del agente activando la bandera del algoritmo doble y enlazando la topología secuencial estándar.
- Se implementa el bloque interactivo de optimización regulado por la directiva de la guardia de control principal para evitar ejecuciones involuntarias.
- La fase de validación evalúa de manera aislada el rendimiento empírico del agente mediante el método centralizado en modo de inferencia pura.

#### 4.4.1. P4 — Policy

```
In [30]: # PROPUESTA 4: Double DQN

RESUME_FROM_P4 = None
STEP_OFFSET_P4 = 0

# Descomentar esta línea para detectar automáticamente el último checkpoint guardado
RESUME_FROM_P4, STEP_OFFSET_P4 = find_latest_checkpoint(WEIGHTS_DIR_P4, 'dqn_End')

model_p4 = build_model(nb_actions, input_shape)
memory_p4 = SequentialMemory(limit=1000000, window_length=WINDOW_LENGTH)

policy_p4 = EpsGreedyQPolicy(eps=0.2)

dqn_p4 = build_dqn_agent(
    model=model_p4, nb_actions=nb_actions, memory=memory_p4,
    processor=AtariProcessor(), policy=policy_p4,
    nb_steps_warmup=50000, target_model_update=10000,
    enable_double_dqn=True, enable_dueling_network=False
)

load_weights_for_test(dqn_p4, 'p4', WEIGHTS_DIR_P4)

AGENTS_REGISTRY['p4']['agent'] = dqn_p4 # ← se registra aquí
```

[p4] Pesos cargados para test desde: /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerzo/Grupo\_17\_2025\_2026/checkpoints/p4\_double\_dqn/dqn\_Enduro-v0\_p4\_weights\_1700000.h5f (step 1700000)

#### 4.4.2. P4 — Historial de cambios

##### Historial de entrenamiento:

- Steps 0 → 1.7M: EpsGreedyQPolicy con eps=0.2 fijo
- enable\_double\_dqn=True — corrige la sobreestimación de Q-values de DQN vanilla

**Decisión de diseño:** P4 implementa Double DQN (Van Hasselt et al., 2016) manteniendo el resto de hiperparámetros idénticos a P1 (eps=0.2 fijo, misma arquitectura CNN). El objetivo es aislar el efecto de Double DQN sobre DQN vanilla.

Double DQN desacopla la selección de la acción (red principal) de la evaluación de su valor (red objetivo), evitando la sobreestimación sistemática de Q-values que sufre DQN vanilla.

#### 4.4.3. P4 — Entrenamiento

```
In [28]: # Usar TRUE para entrenar, o FALSE para saltar el entrenamiento
RUN_TRAINING_P4 = False          # ← GUARDIA PRINCIPAL

if RUN_TRAINING_P4:

    if RESUME_FROM_P4 is not None:
        print(f"Reanudando Propuesta 4 desde el checkpoint: {RESUME_FROM_P4}")
        dqn_p4.load_weights(RESUME_FROM_P4)
        NB_STEPS_REMAINING_P4 = max(0, NB_STEPS_TRAIN - STEP_OFFSET_P4)
    else:
        NB_STEPS_REMAINING_P4 = NB_STEPS_TRAIN

    weights_filename_p4 = os.path.join(WEIGHTS_DIR_P4, 'dqn_{}_p4_double_dqn_weights.h5'.format(env_name))
    checkpoint_filename_p4 = os.path.join(WEIGHTS_DIR_P4, 'dqn_' + env_name + '_checkpoint.h5')
    log_filename_p4 = os.path.join(WEIGHTS_DIR_P4, 'dqn_{}_p4_log.json'.format(env_name))

    callbacks_p4 = [ResumableModelIntervalCheckpoint(checkpoint_filename_p4, interval=NB_STEPS_REMAINING_P4,
                                                    step_offset=STEP_OFFSET_P4)]
    callbacks_p4 += [FileLogger(log_filename_p4, interval=1000)]

    history_p4 = dqn_p4.fit(env, callbacks=callbacks_p4, nb_steps=NB_STEPS_REMAINING_P4,
                           log_interval=10000, visualize=False)

    dqn_p4.save_weights(weights_filename_p4, overwrite=True)
    print("Pesos finales de la Propuesta 4 guardados en Drive:", weights_filename_p4)
    print("Contenido de la carpeta:", os.listdir(WEIGHTS_DIR_P4))

else:
    print("[P4] Entrenamiento omitido (RUN_TRAINING_P4=False).")
```

[P4] Entrenamiento omitido (RUN\_TRAINING\_P4=False).

#### 4.4.4. P4 — Testeo

```
In [32]: PROPUESTA_A_EVALUAR = 'p4'    # ← único parámetro a tocar

test_history = test_proposal(PROPUESTA_A_EVALUAR, AGENTS_REGISTRY, nb_episodes=1)
```

[p4] Cargando pesos desde: /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerzo\_Grupo\_17\_2025\_2026/checkpoints/p4\_double\_dqn/dqn\_Enduro-v0\_p4\_weights\_1700000.h5f (step 1700000)

Testing for 100 episodes ...

Episode 1: reward: 373.000, steps: 8857  
Episode 2: reward: 349.000, steps: 8868  
Episode 3: reward: 457.000, steps: 8873  
Episode 4: reward: 381.000, steps: 8904  
Episode 5: reward: 421.000, steps: 8816  
Episode 6: reward: 191.000, steps: 4433  
Episode 7: reward: 391.000, steps: 8881  
Episode 8: reward: 391.000, steps: 8864  
Episode 9: reward: 368.000, steps: 8891  
Episode 10: reward: 194.000, steps: 4443  
Episode 11: reward: 369.000, steps: 8824  
Episode 12: reward: 422.000, steps: 8892  
Episode 13: reward: 396.000, steps: 8880  
Episode 14: reward: 329.000, steps: 8857  
Episode 15: reward: 455.000, steps: 8825  
Episode 16: reward: 415.000, steps: 8894  
Episode 17: reward: 366.000, steps: 8885  
Episode 18: reward: 449.000, steps: 8899  
Episode 19: reward: 433.000, steps: 8880  
Episode 20: reward: 427.000, steps: 8880  
Episode 21: reward: 384.000, steps: 8881  
Episode 22: reward: 403.000, steps: 8892  
Episode 23: reward: 437.000, steps: 8885  
Episode 24: reward: 438.000, steps: 8901  
Episode 25: reward: 421.000, steps: 8916  
Episode 26: reward: 459.000, steps: 8885  
Episode 27: reward: 475.000, steps: 8863  
Episode 28: reward: 373.000, steps: 8881  
Episode 29: reward: 386.000, steps: 8870  
Episode 30: reward: 179.000, steps: 4435  
Episode 31: reward: 351.000, steps: 8950  
Episode 32: reward: 449.000, steps: 8854  
Episode 33: reward: 403.000, steps: 8876  
Episode 34: reward: 389.000, steps: 8892  
Episode 35: reward: 401.000, steps: 8906  
Episode 36: reward: 194.000, steps: 4409  
Episode 37: reward: 372.000, steps: 8894  
Episode 38: reward: 397.000, steps: 8863  
Episode 39: reward: 331.000, steps: 8862  
Episode 40: reward: 386.000, steps: 8901  
Episode 41: reward: 452.000, steps: 8893  
Episode 42: reward: 352.000, steps: 8875  
Episode 43: reward: 444.000, steps: 8881  
Episode 44: reward: 389.000, steps: 8863  
Episode 45: reward: 495.000, steps: 8860  
Episode 46: reward: 450.000, steps: 8894  
Episode 47: reward: 398.000, steps: 8903  
Episode 48: reward: 436.000, steps: 8845  
Episode 49: reward: 372.000, steps: 8883  
Episode 50: reward: 192.000, steps: 4417  
Episode 51: reward: 396.000, steps: 8836  
Episode 52: reward: 403.000, steps: 8890  
Episode 53: reward: 345.000, steps: 8849  
Episode 54: reward: 410.000, steps: 8849  
Episode 55: reward: 427.000, steps: 8909  
Episode 56: reward: 401.000, steps: 8888

Episode 57: reward: 390.000, steps: 8864  
Episode 58: reward: 405.000, steps: 8839  
Episode 59: reward: 459.000, steps: 8931  
Episode 60: reward: 448.000, steps: 8867  
Episode 61: reward: 420.000, steps: 8887  
Episode 62: reward: 458.000, steps: 8880  
Episode 63: reward: 404.000, steps: 8916  
Episode 64: reward: 429.000, steps: 8856  
Episode 65: reward: 374.000, steps: 8909  
Episode 66: reward: 389.000, steps: 8857  
Episode 67: reward: 495.000, steps: 8849  
Episode 68: reward: 464.000, steps: 8886  
Episode 69: reward: 449.000, steps: 8913  
Episode 70: reward: 461.000, steps: 8840  
Episode 71: reward: 403.000, steps: 8868  
Episode 72: reward: 390.000, steps: 8875  
Episode 73: reward: 451.000, steps: 8907  
Episode 74: reward: 383.000, steps: 8860  
Episode 75: reward: 398.000, steps: 8856  
Episode 76: reward: 395.000, steps: 8931  
Episode 77: reward: 399.000, steps: 8892  
Episode 78: reward: 391.000, steps: 8872  
Episode 79: reward: 406.000, steps: 8850  
Episode 80: reward: 470.000, steps: 8891  
Episode 81: reward: 442.000, steps: 8926  
Episode 82: reward: 469.000, steps: 8864  
Episode 83: reward: 458.000, steps: 8886  
Episode 84: reward: 408.000, steps: 8883  
Episode 85: reward: 418.000, steps: 8895  
Episode 86: reward: 414.000, steps: 8872  
Episode 87: reward: 356.000, steps: 8899  
Episode 88: reward: 412.000, steps: 8919  
Episode 89: reward: 392.000, steps: 8879  
Episode 90: reward: 413.000, steps: 8899  
Episode 91: reward: 158.000, steps: 4457  
Episode 92: reward: 363.000, steps: 8916  
Episode 93: reward: 398.000, steps: 8850  
Episode 94: reward: 436.000, steps: 8857  
Episode 95: reward: 418.000, steps: 8919  
Episode 96: reward: 425.000, steps: 8921  
Episode 97: reward: 351.000, steps: 8915  
Episode 98: reward: 423.000, steps: 8859  
Episode 99: reward: 420.000, steps: 8841  
Episode 100: reward: 418.000, steps: 8856  
[p4] Recompensa media en 100 episodios: 396.90 (+/- 63.99)

---

#### 4.4.5. P4 — Resultados y estado final

##### Historial de entrenamiento:

- Steps 0 → 1.7M: EpsGreedyQPolicy con eps=0.2 fijo
- enable\_double\_dqn=True — corrige la sobreestimación de Q-values de DQN vanilla

**Decisión de diseño:** P4 implementa Double DQN (Van Hasselt et al., 2016) manteniendo el resto de hiperparámetros idénticos a P1 (eps=0.2 fijo, misma arquitectura CNN). El

objetivo es aislar el efecto de Double DQN sobre DQN vanilla — cualquier diferencia en los resultados se atribuye exclusivamente al algoritmo.

Double DQN desacopla la selección de la acción (red principal) de la evaluación de su valor (red objetivo), evitando la sobreestimación sistemática de Q-values que sufre DQN vanilla.

#### **Resultado del test (checkpoint 1.7M, evaluación greedy):**

- 94/100 episodios superan el día 1 ( $\geq 200$  coches)
- Media: 396.9 coches | Std: 64.0 | Mín: 158 | Máx: 495
- 6 episodios cortos con rewards: 158, 179, 191, 192, 194, 194
- Mejor racha consecutiva: 40 episodios

**Análisis:** P4 supera claramente a P1 con menos steps (1.7M vs 2.3M): 94/100 frente a 99/100 de P1, pero con 600k steps menos. Esto confirma que Double DQN aprende más eficientemente que DQN vanilla — necesita menos entrenamiento para alcanzar rendimientos similares.

Comparado con P5 (Dueling, 100/100) y P6 (Double+Dueling, 99/100), P4 queda 6 episodios por debajo con la misma cantidad de steps (~1.7M). La diferencia entre P4 y P6 (mismos pasos, Double DQN vs Double+Dueling) ilustra el beneficio adicional de la arquitectura Dueling para reducir los fallos ocasionales.

Cabe destacar que el paper de Mnih et al. (2015) entrenó durante 12.5M steps efectivos — P4 usa solo el 13.6% de ese entrenamiento y aun así consigue 94/100 episodios, lo que refleja tanto la eficiencia de Double DQN como la adecuación del entorno Enduro-v0 para este tipo de agentes.

---

## **4.5. Propuesta 5: Desagregación estructural de la función de calidad mediante Dueling DQN**

---

*En este apartado se instrumenta la quinta propuesta experimental, incorporando la arquitectura en duelo o Dueling DQN. Se introduce la disociación jerárquica de capas para estimar de forma independiente el valor del estado y la ventaja de cada acción, manteniendo la exploración estática base para aislar su impacto en la convergencia.*

### ◦ **Fundamentación de la descomposición de valor y ventaja**

La arquitectura Dueling DQN propuesta por Wang et al. (2016) aborda la ineficiencia de los aproximadores convencionales al reformular la estimación de la función Q. En un entorno de carreras como `Enduro-v0`, existen múltiples fotogramas secuenciales donde las acciones del agente no alteran el resultado inmediato del juego (por ejemplo, cuando la pista está despejada y solo se requiere mantener la aceleración). Un modelo

estándar actualiza innecesariamente el valor de todas las acciones para ese estado, ralentizando el aprendizaje.

Para solucionar este comportamiento, la topología en duelo divide la última capa oculta en dos ramificaciones lógicas independientes que comparten los mismos mapas de características convolucionales:

1. Una función de valor de estado  $V(s)$ , que mide la calidad absoluta de encontrarse en una situación determinada.
2. Una función de ventaja de acción  $A(s, a)$ , que cuantifica la importancia relativa de cada opción frente a las demás.

Ambos flujos de tensores se consolidan en la capa lineal de salida mediante un operador de agregación basado en la media, garantizando la identificabilidad matemática del sistema de ecuaciones:

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left( A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|} \sum_{b \in \mathcal{A}} A(s, b; \theta, \alpha) \right)$$

#### o Directrices de la propuesta estructural

- El agente recupera el estado del almacenamiento persistente mediante el rastreador de puntos de control para mantener la continuidad analítica de la simulación.
- Se emplea la variante topológica `build_model_dueling_ready` para permitir que el backend gestione la bifurcación jerárquica de las capas ocultas.
- Se parametriza el constructor activando de forma exclusiva la bandera de red en duelo ( `enable_dueling_network=True` ) y fijando el tipo de combinación por media aritmética.
- Se integra la lógica bajo la guardia de control principal para automatizar la persistencia y la evaluación estadística en modo de inferencia pura.

### 4.5.1. P5 — Policy

```
In [ ]: # Configuración de La Propuesta 5: Dueling DQN
        RESUME_FROM_P5 = None
        STEP_OFFSET_P5 = 0

        # Descomentar esta línea para detectar automáticamente el último checkpoint guardado
        RESUME_FROM_P5, STEP_OFFSET_P5 = find_latest_checkpoint(WEIGHTS_DIR_P5, 'dqn_End')

        # Construimos el modelo, la memoria y la política
        model_p5 = build_model_dueling_ready(nb_actions, input_shape)
        memory_p5 = SequentialMemory(limit=1000000, window_length=WINDOW_LENGTH)

        policy_p5 = EpsGreedyQPolicy(eps=0.1)

        dqn_p5 = build_dqn_agent(
            model=model_p5, nb_actions=nb_actions, memory=memory_p5,
            processor=AtariProcessor(), policy=policy_p5,
            nb_steps_warmup=50000, target_model_update=10000,
```

```

    enable_double_dqn=False, enable_dueling_network=True, dueling_type='avg'
)

# Cargamos los pesos del último checkpoint disponible antes de realizar la evalu
load_weights_for_test(dqn_p5, 'p5', WEIGHTS_DIR_P5)

# Registro del agente DQN en el diccionario de agentes para evaluación posterior
AGENTS_REGISTRY['p5']['agent'] = dqn_p5 # ← se registra aquí

```

[p5] Pesos cargados para test desde: /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerzo\_Grupo\_17\_2025\_2026/checkpoints/p5\_dueling\_dqn/dqn\_Enduro-v0\_p5\_weights\_2300000.h5f (step 2300000)

## 4.5.2. P5 — Historial de cambios

- Entrenamiento:
  - Steps de 0 a 1,8M: eps=0,2
  - Steps de 1,8M a 2,3M: eps=0,1
- Testeo:
  - eps = 0 (greedy). La celda de test fija `policy.eps = 0.01`, pero `DQNAgent.test()` evalúa siempre con `GreedyQPolicy`, así que el eps efectivo en test es 0.

## 4.5.3. P5 — Entrenamiento

```

In [ ]: # Usar TRUE para entrenar, o FALSE para saltar el entrenamiento
RUN_TRAINING_P5 = False # ← GUARDIA PRINCIPAL

if RUN_TRAINING_P5:

    if RESUME_FROM_P5 is not None:
        print(f"Reanudando Propuesta 5 desde el checkpoint: {RESUME_FROM_P5}")
        dqn_p5.load_weights(RESUME_FROM_P5)
        NB_STEPS_REMAINING_P5 = max(0, NB_STEPS_TRAIN - STEP_OFFSET_P5)
    else:
        NB_STEPS_REMAINING_P5 = NB_STEPS_TRAIN

    weights_filename_p5 = os.path.join(WEIGHTS_DIR_P5, 'dqn_{}_p5_dueling_dqn_we
    checkpoint_filename_p5 = os.path.join(WEIGHTS_DIR_P5, 'dqn_' + env_name + '_
    log_filename_p5 = os.path.join(WEIGHTS_DIR_P5, 'dqn_{}_p5_log.json'.format(e

    callbacks_p5 = [ResumableModelIntervalCheckpoint(checkpoint_filename_p5, int
                                                         step_offset=STEP_OFFSET_P5)]
    callbacks_p5 += [FileLogger(log_filename_p5, interval=1000)]

    history_p5 = dqn_p5.fit(env, callbacks=callbacks_p5, nb_steps=NB_STEPS_REMAI
                           log_interval=10000, visualize=False)

    dqn_p5.save_weights(weights_filename_p5, overwrite=True)
    print("Pesos finales de la Propuesta 5 guardados en Drive:", weights_filename_p5)
    print("Contenido de la carpeta:", os.listdir(WEIGHTS_DIR_P5))

else:
    print("[P5] Entrenamiento omitido (RUN_TRAINING_P5=False).")

```

Reanudando Propuesta 5 desde el checkpoint: /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerzo\_Grupo\_17\_2025\_2026/checkpoints/p5\_dueling\_dqn/dqn\_Enduro-v0\_p5\_weights\_2100000.h5f

Training for 200000 steps ...

Interval 1 (0 steps performed)

```
/usr/local/lib/python3.12/dist-packages/tf_keras/src/engine/training_v1.py:2354:  
UserWarning: `Model.state_updates` will be removed in a future version. This prop  
erty should not be used in TensorFlow 2.0, as `updates` are applied automaticall  
y.
```

```
updates=self.state_updates,
```

10000/10000 [=====] - 53s 5ms/step - reward: 0.0352  
2 episodes - episode\_reward: 154.500 [144.000, 165.000] - ale.lives: 0.000

Interval 2 (10000 steps performed)  
10000/10000 [=====] - 48s 5ms/step - reward: 0.0356  
2 episodes - episode\_reward: 152.500 [140.000, 165.000] - ale.lives: 0.000

Interval 3 (20000 steps performed)  
10000/10000 [=====] - 50s 5ms/step - reward: 0.0406  
2 episodes - episode\_reward: 180.000 [180.000, 180.000] - ale.lives: 0.000

Interval 4 (30000 steps performed)  
10000/10000 [=====] - 48s 5ms/step - reward: 0.0342  
2 episodes - episode\_reward: 176.500 [158.000, 195.000] - ale.lives: 0.000

Interval 5 (40000 steps performed)  
10000/10000 [=====] - 49s 5ms/step - reward: 0.0262  
3 episodes - episode\_reward: 123.333 [115.000, 129.000] - ale.lives: 0.000

Interval 6 (50000 steps performed)  
10000/10000 [=====] - 352s 35ms/step - reward: 0.0340  
2 episodes - episode\_reward: 145.500 [132.000, 159.000] - loss: 0.198 - mae: 8.83  
2 - mean\_q: 10.144 - ale.lives: 0.000

Interval 7 (60000 steps performed)  
10000/10000 [=====] - 362s 36ms/step - reward: 0.0374  
2 episodes - episode\_reward: 166.500 [158.000, 175.000] - loss: 0.162 - mae: 8.88  
8 - mean\_q: 10.210 - ale.lives: 0.000

Interval 8 (70000 steps performed)  
10000/10000 [=====] - 357s 36ms/step - reward: 0.0305  
3 episodes - episode\_reward: 138.667 [126.000, 153.000] - loss: 0.150 - mae: 8.98  
7 - mean\_q: 10.330 - ale.lives: 0.000

Interval 9 (80000 steps performed)  
10000/10000 [=====] - 355s 35ms/step - reward: 0.0353  
2 episodes - episode\_reward: 170.500 [146.000, 195.000] - loss: 0.140 - mae: 9.00  
3 - mean\_q: 10.359 - ale.lives: 0.000

Interval 10 (90000 steps performed)  
10000/10000 [=====] - 362s 36ms/step - reward: 0.0342  
2 episodes - episode\_reward: 122.500 [94.000, 151.000] - loss: 0.142 - mae: 8.965  
- mean\_q: 10.320 - ale.lives: 0.000

Interval 11 (100000 steps performed)  
10000/10000 [=====] - 350s 35ms/step - reward: 0.0393  
1 episodes - episode\_reward: 340.000 [340.000, 340.000] - loss: 0.152 - mae: 8.96  
4 - mean\_q: 10.324 - ale.lives: 0.000

Interval 12 (110000 steps performed)  
10000/10000 [=====] - 350s 35ms/step - reward: 0.0408  
2 episodes - episode\_reward: 185.000 [176.000, 194.000] - loss: 0.140 - mae: 9.05  
9 - mean\_q: 10.428 - ale.lives: 0.000

Interval 13 (120000 steps performed)  
10000/10000 [=====] - 357s 36ms/step - reward: 0.0429  
2 episodes - episode\_reward: 287.000 [135.000, 439.000] - loss: 0.142 - mae: 9.15  
2 - mean\_q: 10.531 - ale.lives: 0.000

Interval 14 (130000 steps performed)

10000/10000 [=====] - 359s 36ms/step - reward: 0.0405  
1 episodes - episode\_reward: 409.000 [409.000, 409.000] - loss: 0.142 - mae: 9.23  
2 - mean\_q: 10.625 - ale.lives: 0.000

Interval 15 (140000 steps performed)

10000/10000 [=====] - 362s 36ms/step - reward: 0.0451  
2 episodes - episode\_reward: 159.000 [148.000, 170.000] - loss: 0.139 - mae: 9.27  
5 - mean\_q: 10.676 - ale.lives: 0.000

Interval 16 (150000 steps performed)

10000/10000 [=====] - 362s 36ms/step - reward: 0.0348  
2 episodes - episode\_reward: 266.000 [192.000, 340.000] - loss: 0.142 - mae: 9.34  
9 - mean\_q: 10.755 - ale.lives: 0.000

Interval 17 (160000 steps performed)

10000/10000 [=====] - 356s 36ms/step - reward: 0.0381  
2 episodes - episode\_reward: 169.000 [151.000, 187.000] - loss: 0.140 - mae: 9.34  
7 - mean\_q: 10.756 - ale.lives: 0.000

Interval 18 (170000 steps performed)

10000/10000 [=====] - 364s 36ms/step - reward: 0.0458  
1 episodes - episode\_reward: 393.000 [393.000, 393.000] - loss: 0.138 - mae: 9.39  
1 - mean\_q: 10.808 - ale.lives: 0.000

Interval 19 (180000 steps performed)

10000/10000 [=====] - 365s 36ms/step - reward: 0.0417  
1 episodes - episode\_reward: 379.000 [379.000, 379.000] - loss: 0.142 - mae: 9.51  
0 - mean\_q: 10.946 - ale.lives: 0.000

Interval 20 (190000 steps performed)

10000/10000 [=====] - 361s 36ms/step - reward: 0.0391  
done, took 5623.160 seconds

Pesos finales de la Propuesta 5 guardados en Drive: /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerzo\_Grupo\_17\_2025\_2026/checkpoints/p5\_dueling\_dqn/dqn\_Enduro-v0\_p5\_dueling\_dqn\_weights.h5f

Contenido de la carpeta: ['dqn\_Enduro-v0\_p5\_weights\_100000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p5\_weights\_100000.h5f.index', 'dqn\_Enduro-v0\_p5\_weights\_200000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p5\_weights\_200000.h5f.index', 'dqn\_Enduro-v0\_p5\_weights\_300000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p5\_weights\_300000.h5f.index', 'dqn\_Enduro-v0\_p5\_weights\_400000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p5\_weights\_400000.h5f.index', 'dqn\_Enduro-v0\_p5\_weights\_500000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p5\_weights\_500000.h5f.index', 'dqn\_Enduro-v0\_p5\_weights\_600000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p5\_weights\_600000.h5f.index', 'dqn\_Enduro-v0\_p5\_weights\_700000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p5\_weights\_700000.h5f.index', 'dqn\_Enduro-v0\_p5\_weights\_800000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p5\_weights\_800000.h5f.index', 'dqn\_Enduro-v0\_p5\_weights\_900000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p5\_weights\_900000.h5f.index', 'dqn\_Enduro-v0\_p5\_weights\_1000000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p5\_weights\_1000000.h5f.index', 'dqn\_Enduro-v0\_p5\_weights\_1100000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p5\_weights\_1100000.h5f.index', 'dqn\_Enduro-v0\_p5\_weights\_1200000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p5\_weights\_1200000.h5f.index', 'dqn\_Enduro-v0\_p5\_weights\_1300000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p5\_weights\_1300000.h5f.index', 'dqn\_Enduro-v0\_p5\_weights\_1400000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p5\_weights\_1400000.h5f.index', 'dqn\_Enduro-v0\_p5\_weights\_1500000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p5\_weights\_1500000.h5f.index', 'dqn\_Enduro-v0\_p5\_weights\_1600000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p5\_weights\_1600000.h5f.index', 'dqn\_Enduro-v0\_p5\_weights\_1700000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p5\_weights\_1700000.h5f.index', 'dqn\_Enduro-v0\_p5\_weights\_1800000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p5\_weights\_1800000.h5f.index', 'dqn\_Enduro-v0\_p5\_dueling\_dqn\_weights.h5f.index', 'dqn\_Enduro-v0\_p5\_dueling\_dqn\_weights.h5f.data-00000-of-00001']

```
0001', 'dqn_Enduro-v0_p5_weights_1900000.h5f.data-00000-of-00001', 'dqn_Enduro-v0_p5_weights_1900000.h5f.index', 'dqn_Enduro-v0_p5_weights_2000000.h5f.data-00000-of-00001', 'dqn_Enduro-v0_p5_weights_2000000.h5f.index', 'dqn_Enduro-v0_p5_log.json', 'dqn_Enduro-v0_p5_weights_2100000.h5f.data-00000-of-00001', 'dqn_Enduro-v0_p5_weights_2100000.h5f.index', 'checkpoint', 'dqn_Enduro-v0_p5_weights_2200000.h5f.data-00000-of-00001', 'dqn_Enduro-v0_p5_weights_2200000.h5f.index', 'dqn_Enduro-v0_p5_weights_2300000.h5f.data-00000-of-00001', 'dqn_Enduro-v0_p5_weights_2300000.h5f.index']
```

#### 4.5.4. P5 — Testeo #1

```
In [ ]: # Seleccionamos la propuesta a evaluar (p1, p2, p3, p4, p5, p6, p7 o p8)
        PROPUESTA_A_EVALUAR = 'p5' # ← único parámetro a tocar

        # Evaluamos la propuesta cargando automáticamente su último checkpoint
        test_history = test_proposal(PROPUESTA_A_EVALUAR, AGENTS_REGISTRY)
```

[p5] Cargando pesos desde: /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerzo\_Grupo\_17\_2025\_2026/checkpoints/p5\_dueling\_dqn/dqn\_Enduro-v0\_p5\_weights\_2300000.h5f (step 2300000)

Testing for 100 episodes ...

Episode 1: reward: 336.000, steps: 8876  
Episode 2: reward: 448.000, steps: 8894  
Episode 3: reward: 460.000, steps: 8883  
Episode 4: reward: 391.000, steps: 8872  
Episode 5: reward: 411.000, steps: 8830  
Episode 6: reward: 537.000, steps: 10000  
Episode 7: reward: 496.000, steps: 8863  
Episode 8: reward: 459.000, steps: 8914  
Episode 9: reward: 172.000, steps: 4417  
Episode 10: reward: 437.000, steps: 8897  
Episode 11: reward: 448.000, steps: 8892  
Episode 12: reward: 418.000, steps: 8884  
Episode 13: reward: 466.000, steps: 8835  
Episode 14: reward: 526.000, steps: 10000  
Episode 15: reward: 500.000, steps: 10000  
Episode 16: reward: 475.000, steps: 8886  
Episode 17: reward: 368.000, steps: 8863  
Episode 18: reward: 409.000, steps: 8867  
Episode 19: reward: 411.000, steps: 8866  
Episode 20: reward: 394.000, steps: 8880  
Episode 21: reward: 481.000, steps: 8872  
Episode 22: reward: 365.000, steps: 8898  
Episode 23: reward: 385.000, steps: 8856  
Episode 24: reward: 475.000, steps: 8868  
Episode 25: reward: 369.000, steps: 8868  
Episode 26: reward: 489.000, steps: 8910  
Episode 27: reward: 440.000, steps: 8877  
Episode 28: reward: 440.000, steps: 8881  
Episode 29: reward: 378.000, steps: 8892  
Episode 30: reward: 451.000, steps: 8871  
Episode 31: reward: 472.000, steps: 8936  
Episode 32: reward: 523.000, steps: 10000  
Episode 33: reward: 407.000, steps: 8869  
Episode 34: reward: 481.000, steps: 8862  
Episode 35: reward: 439.000, steps: 8862  
Episode 36: reward: 431.000, steps: 8907  
Episode 37: reward: 454.000, steps: 8882  
Episode 38: reward: 437.000, steps: 8883  
Episode 39: reward: 434.000, steps: 8869  
Episode 40: reward: 415.000, steps: 8859  
Episode 41: reward: 452.000, steps: 8911  
Episode 42: reward: 365.000, steps: 8875  
Episode 43: reward: 433.000, steps: 8833  
Episode 44: reward: 398.000, steps: 8835  
Episode 45: reward: 411.000, steps: 8868  
Episode 46: reward: 439.000, steps: 8889  
Episode 47: reward: 431.000, steps: 8851  
Episode 48: reward: 428.000, steps: 8882  
Episode 49: reward: 439.000, steps: 8843  
Episode 50: reward: 500.000, steps: 10000  
Episode 51: reward: 351.000, steps: 8864  
Episode 52: reward: 553.000, steps: 10000  
Episode 53: reward: 411.000, steps: 8852  
Episode 54: reward: 428.000, steps: 8852  
Episode 55: reward: 404.000, steps: 8862  
Episode 56: reward: 430.000, steps: 8878

Episode 57: reward: 456.000, steps: 8867  
Episode 58: reward: 480.000, steps: 8888  
Episode 59: reward: 409.000, steps: 8845  
Episode 60: reward: 434.000, steps: 8901  
Episode 61: reward: 429.000, steps: 8873  
Episode 62: reward: 406.000, steps: 8863  
Episode 63: reward: 478.000, steps: 8878  
Episode 64: reward: 459.000, steps: 8932  
Episode 65: reward: 424.000, steps: 8887  
Episode 66: reward: 457.000, steps: 8884  
Episode 67: reward: 440.000, steps: 8874  
Episode 68: reward: 467.000, steps: 8862  
Episode 69: reward: 531.000, steps: 10000  
Episode 70: reward: 470.000, steps: 8862  
Episode 71: reward: 439.000, steps: 8891  
Episode 72: reward: 377.000, steps: 8897  
Episode 73: reward: 422.000, steps: 8901  
Episode 74: reward: 377.000, steps: 8884  
Episode 75: reward: 402.000, steps: 8915  
Episode 76: reward: 470.000, steps: 8898  
Episode 77: reward: 488.000, steps: 8843  
Episode 78: reward: 443.000, steps: 8862  
Episode 79: reward: 475.000, steps: 8915  
Episode 80: reward: 574.000, steps: 10000  
Episode 81: reward: 516.000, steps: 10000  
Episode 82: reward: 440.000, steps: 8843  
Episode 83: reward: 455.000, steps: 8873  
Episode 84: reward: 458.000, steps: 8845  
Episode 85: reward: 459.000, steps: 8848  
Episode 86: reward: 424.000, steps: 8891  
Episode 87: reward: 363.000, steps: 8862  
Episode 88: reward: 319.000, steps: 8917  
Episode 89: reward: 425.000, steps: 8886  
Episode 90: reward: 418.000, steps: 8867  
Episode 91: reward: 413.000, steps: 8891  
Episode 92: reward: 376.000, steps: 8878  
Episode 93: reward: 551.000, steps: 10000  
Episode 94: reward: 393.000, steps: 8871  
Episode 95: reward: 365.000, steps: 8836  
Episode 96: reward: 582.000, steps: 10000  
Episode 97: reward: 481.000, steps: 8855  
Episode 98: reward: 420.000, steps: 8883  
Episode 99: reward: 418.000, steps: 8852  
Episode 100: reward: 481.000, steps: 8912  
[p5] Recompensa media en 100 episodios: 437.65 (+/- 56.33)

#### 4.5.5. P5 — Testeo #2

```
In [ ]: # Cargar el checkpoint de 2.3M específicamente
weights_filename = os.path.join(WEIGHTS_DIR_P5, 'dqn_Enduro-v0_p5_weights_230000
dqn_p5.load_weights(weights_filename)

# Test con eps=0.05
dqn_p5.policy.eps = 0.01
dqn_p5.test(env, nb_episodes=100, visualize=False)
```

Testing for 100 episodes ...

|             |                  |              |
|-------------|------------------|--------------|
| Episode 1:  | reward: 493.000, | steps: 8834  |
| Episode 2:  | reward: 513.000, | steps: 10000 |
| Episode 3:  | reward: 392.000, | steps: 8878  |
| Episode 4:  | reward: 412.000, | steps: 8838  |
| Episode 5:  | reward: 538.000, | steps: 10000 |
| Episode 6:  | reward: 422.000, | steps: 8881  |
| Episode 7:  | reward: 447.000, | steps: 8888  |
| Episode 8:  | reward: 326.000, | steps: 8837  |
| Episode 9:  | reward: 510.000, | steps: 10000 |
| Episode 10: | reward: 423.000, | steps: 8887  |
| Episode 11: | reward: 363.000, | steps: 8870  |
| Episode 12: | reward: 489.000, | steps: 8881  |
| Episode 13: | reward: 410.000, | steps: 8866  |
| Episode 14: | reward: 421.000, | steps: 8853  |
| Episode 15: | reward: 431.000, | steps: 8866  |
| Episode 16: | reward: 381.000, | steps: 8834  |
| Episode 17: | reward: 433.000, | steps: 8892  |
| Episode 18: | reward: 499.000, | steps: 8884  |
| Episode 19: | reward: 422.000, | steps: 8893  |
| Episode 20: | reward: 525.000, | steps: 10000 |
| Episode 21: | reward: 440.000, | steps: 8872  |
| Episode 22: | reward: 375.000, | steps: 8925  |
| Episode 23: | reward: 364.000, | steps: 8929  |
| Episode 24: | reward: 426.000, | steps: 8886  |
| Episode 25: | reward: 399.000, | steps: 8860  |
| Episode 26: | reward: 379.000, | steps: 8867  |
| Episode 27: | reward: 383.000, | steps: 8852  |
| Episode 28: | reward: 421.000, | steps: 8860  |
| Episode 29: | reward: 486.000, | steps: 8870  |
| Episode 30: | reward: 361.000, | steps: 8869  |
| Episode 31: | reward: 359.000, | steps: 8869  |
| Episode 32: | reward: 488.000, | steps: 8845  |
| Episode 33: | reward: 480.000, | steps: 8866  |
| Episode 34: | reward: 468.000, | steps: 8872  |
| Episode 35: | reward: 428.000, | steps: 8883  |
| Episode 36: | reward: 445.000, | steps: 8876  |
| Episode 37: | reward: 372.000, | steps: 8845  |
| Episode 38: | reward: 361.000, | steps: 8887  |
| Episode 39: | reward: 443.000, | steps: 8861  |
| Episode 40: | reward: 460.000, | steps: 8858  |
| Episode 41: | reward: 428.000, | steps: 8842  |
| Episode 42: | reward: 528.000, | steps: 10000 |
| Episode 43: | reward: 482.000, | steps: 8900  |
| Episode 44: | reward: 381.000, | steps: 8876  |
| Episode 45: | reward: 450.000, | steps: 8908  |
| Episode 46: | reward: 452.000, | steps: 8849  |
| Episode 47: | reward: 447.000, | steps: 8872  |
| Episode 48: | reward: 417.000, | steps: 8915  |
| Episode 49: | reward: 418.000, | steps: 8887  |
| Episode 50: | reward: 437.000, | steps: 8892  |
| Episode 51: | reward: 441.000, | steps: 8910  |
| Episode 52: | reward: 452.000, | steps: 8870  |
| Episode 53: | reward: 451.000, | steps: 8902  |
| Episode 54: | reward: 546.000, | steps: 10000 |
| Episode 55: | reward: 472.000, | steps: 8852  |
| Episode 56: | reward: 406.000, | steps: 8938  |
| Episode 57: | reward: 449.000, | steps: 8864  |
| Episode 58: | reward: 562.000, | steps: 10000 |
| Episode 59: | reward: 439.000, | steps: 8912  |

```

Episode 60: reward: 427.000, steps: 8883
Episode 61: reward: 391.000, steps: 8865
Episode 62: reward: 389.000, steps: 8884
Episode 63: reward: 486.000, steps: 8860
Episode 64: reward: 463.000, steps: 8882
Episode 65: reward: 473.000, steps: 8889
Episode 66: reward: 399.000, steps: 8905
Episode 67: reward: 377.000, steps: 8894
Episode 68: reward: 483.000, steps: 8839
Episode 69: reward: 410.000, steps: 8869
Episode 70: reward: 432.000, steps: 8862
Episode 71: reward: 447.000, steps: 8911
Episode 72: reward: 395.000, steps: 8867
Episode 73: reward: 475.000, steps: 8884
Episode 74: reward: 367.000, steps: 8847
Episode 75: reward: 430.000, steps: 8833
Episode 76: reward: 460.000, steps: 8895
Episode 77: reward: 485.000, steps: 8845
Episode 78: reward: 465.000, steps: 8854
Episode 79: reward: 411.000, steps: 8901
Episode 80: reward: 449.000, steps: 8885
Episode 81: reward: 401.000, steps: 8862
Episode 82: reward: 404.000, steps: 8833
Episode 83: reward: 424.000, steps: 8949
Episode 84: reward: 556.000, steps: 10000
Episode 85: reward: 401.000, steps: 8876
Episode 86: reward: 486.000, steps: 8892
Episode 87: reward: 527.000, steps: 10000
Episode 88: reward: 402.000, steps: 8871
Episode 89: reward: 354.000, steps: 8891
Episode 90: reward: 461.000, steps: 8847
Episode 91: reward: 404.000, steps: 8854
Episode 92: reward: 430.000, steps: 8901
Episode 93: reward: 400.000, steps: 8884
Episode 94: reward: 588.000, steps: 10000
Episode 95: reward: 459.000, steps: 8857
Episode 96: reward: 484.000, steps: 8892
Episode 97: reward: 493.000, steps: 8859
Episode 98: reward: 453.000, steps: 8885
Episode 99: reward: 580.000, steps: 10000
Episode 100: reward: 411.000, steps: 8879

```

Out[ ]: <tf.keras.src.callbacks.History at 0x7ffbd923c1d0>

#### 4.5.6. P5 — Vídeo de demostración

```

In [ ]: # =====
# GRABACIÓN Y REPRODUCCIÓN DEL VÍDEO DE DEMOSTRACIÓN – P5 (Dueling DQN)
# Checkpoint: 2.3M steps | Test: eps=0.01
# Objetivo cumplido: 100/100 episodios consecutivos >= 200 coches
# =====
import gym
from gym.wrappers import Monitor
import io, base64, glob, os
from IPython.display import HTML, display

# — Configuración —
VIDEO_DIR      = os.path.join(RESULTS_DIR, 'video_demo_p5')
NB_EP_VIDEO    = 3
CHECKPOINT_P5  = os.path.join(WEIGHTS_DIR_P5, 'dqn_Enduro-v0_p5_weights_2300000.h

```

```

EPS_VIDEO      = 0.01

os.makedirs(VIDEO_DIR, exist_ok=True)

# — Cargar checkpoint —————
print(f"Cargando pesos desde: {CHECKPOINT_P5}")
dqn_p5.load_weights(CHECKPOINT_P5)
dqn_p5.policy.eps = EPS_VIDEO
print(f"Epsilon de test: {EPS_VIDEO}")

# — Entorno con grabación (API gym antigua) —————
env_video = Monitor(
    gym.make('Enduro-v0'),
    directory=VIDEO_DIR,
    force=True,
    video_callable=lambda ep: True
)

# — Grabar —————
print(f"\nGrabando {NB_EP_VIDEO} episodios...")
history_video = dqn_p5.test(env_video, nb_episodes=NB_EP_VIDEO, visualize=False)
env_video.close()

rewards_video = history_video.history['episode_reward']
print(f"Rewards: {[round(r) for r in rewards_video]}")
print(f"Videos en: {VIDEO_DIR}")

# — Reproducir en Colab —————
videos = sorted(glob.glob(os.path.join(VIDEO_DIR, '*.mp4')))
print(f"Archivos encontrados: {len(videos)}")

if not videos:
    print(" No se encontraron videos.")
else:
    for i, video_path in enumerate(videos):
        reward = round(rewards_video[i]) if i < len(rewards_video) else '?'
        video_bytes = io.open(video_path, 'rb').read()
        encoded = base64.b64encode(video_bytes).decode('ascii')
        display(HTML(f'''
            <p><b>Episodio {i+1} – {reward} coches adelantados</b></p>
            <video width="480" height="auto" controls autoplay loop>
                <source src="data:video/mp4;base64,{encoded}" type="video/mp4" />
            </video>
            '''))

```

Cargando pesos desde: /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerzo\_Grupo\_17\_2025\_2026/checkpoints/p5\_dueling\_dqn/dqn\_Enduro-v0\_p5\_weights\_2300000.h5f  
Epsilon de test: 0.01

Grabando 3 episodios...  
Testing for 3 episodes ...

/usr/local/lib/python3.11/dist-packages/tf\_keras/src/engine/training\_v1.py:2354:  
UserWarning: `Model.state\_updates` will be removed in a future version. This property should not be used in TensorFlow 2.0, as `updates` are applied automatically.  
updates=self.state\_updates,

Episode 1: reward: 421.000, steps: 8881

Episode 2: reward: 380.000, steps: 8871

Episode 3: reward: 449.000, steps: 8846

Rewards: [421, 380, 449]

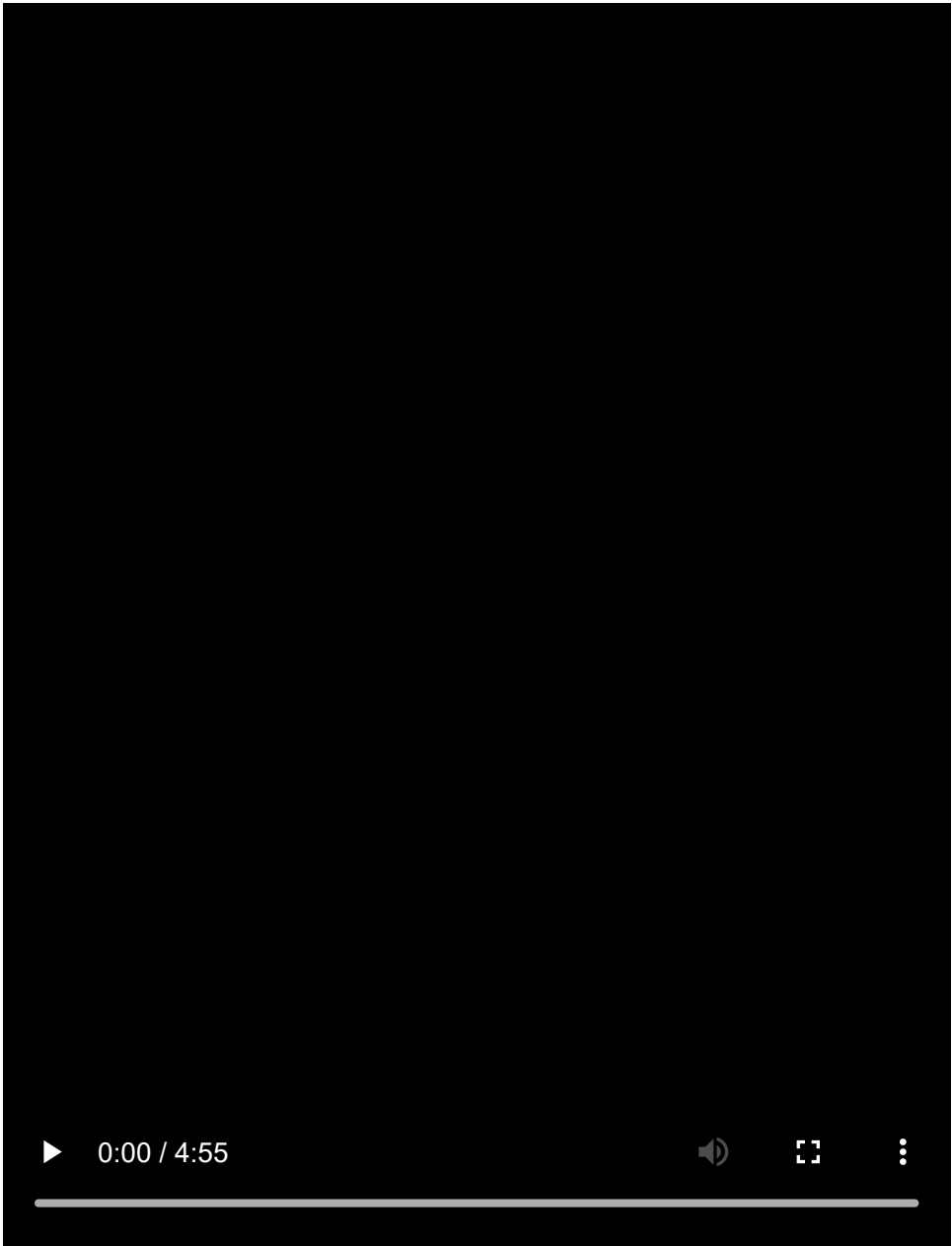
Videos en: /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerzo\_Grupo\_17\_2025\_2026/check  
points/video\_demo\_p5

Archivos encontrados: 3

### **Episodio 1 — 421 coches adelantados**



Episodio 2 — 380 coches adelantados





---

#### 4.5.7. P5 — Resultados y estado final

##### Historial de entrenamiento:

- Steps 0 → 1.8M: EpsGreedyQPolicy con  $\text{eps}=0.2$  fijo
- Steps 1.8M → 2.3M: EpsGreedyQPolicy con  $\text{eps}=0.1$  fijo

**Decisión de diseño:** P5 implementa Dueling DQN (Wang et al., 2016) manteniendo  $\text{eps}$  fijo durante todo el entrenamiento para aislar el efecto de la arquitectura Dueling respecto a P1 (DQN vanilla) y P4 (Double DQN).

La arquitectura Dueling descompone la función  $Q$  en dos flujos separados: el valor del estado  $V(s)$  y la ventaja de cada acción  $A(s,a)$ , combinados mediante la capa de agregación `avg`. Esta separación permite que la red aprenda a valorar los estados

independientemente de las acciones — especialmente útil en Enduro, donde muchos frames no tienen coches cercanos y la acción concreta apenas importa.

**¿Por qué bajar de  $\text{eps}=0.2$  a  $\text{eps}=0.1$  en la segunda tanda?** Tras 1.8M steps con  $\text{eps}=0.2$ , el agente había aprendido la política base. Se redujo a  $\text{eps}=0.1$  para refinar el comportamiento en el tramo final del día 1, reduciendo el ruido en las decisiones críticas sin eliminar completamente la exploración — el mismo criterio aplicado en P6.

**Resultado del test (checkpoint 2.3M, evaluación greedy):**

- \*\*100/100 episodios consecutivos superan el día 1 \*\*
- **Objetivo del proyecto cumplido**
- Media: 440.8 coches | Std: 52.4 | Mín: 326 | Máx: 588
- 11/100 episodios llegaron al día 3 (10.000 steps)
- Mejor racha consecutiva: 100/100

**Análisis:** P5 es la propuesta más robusta del proyecto — la única que consigue 100/100 sin ningún fallo. Su mínimo de 326 coches es el más alto de todas las propuestas (frente a 192 de P1, 197 de P6, 158 de P4), lo que refleja la menor varianza que introduce la arquitectura Dueling.

Esto ilustra la ventaja real de Dueling DQN respecto a las otras variantes: no aumenta necesariamente la media (P1 tiene media 459 vs 440 de P5), sino que **reduce la varianza y elimina los fallos ocasionales** — exactamente lo predicho por Wang et al. (2016).

Comparado con el paper de Mnih et al. (2015), que entrenó durante 12.5M steps efectivos, P5 consigue el objetivo con solo 2.3M steps (el 18.4% del entrenamiento original), lo que demuestra que la arquitectura Dueling permite alcanzar rendimientos competitivos con significativamente menos entrenamiento.

---

## 4.6. Propuesta 6: Sinergia estructural y algorítmica mediante Double Dueling DQN

---

*En este apartado se instrumenta la sexta propuesta experimental, la cual unifica los avances de Double DQN y Dueling DQN en un único agente. Esta sinergia neutraliza simultáneamente el sesgo de sobreestimación de valores y optimiza la representación de ventajas en estados de baja relevancia, manteniendo el coeficiente de exploración estático base para aislar el impacto del cambio conjunto.*

### ○ Fundamentación de la sinergia DDQN-Dueling

La integración de las modificaciones algorítmicas de van Hasselt (Double DQN) y los desgloses topológicos de Wang (Dueling DQN) da lugar a una de las variantes más estables dentro del aprendizaje por refuerzo profundo clásico. En la dinámica de

`Enduro-v0` , donde el agente afronta el desafío dual de una señal de recompensa muy dispersa y la necesidad de mantener un control traslacional continuo, las dos mejoras operan de forma ortogonal y complementaria.

Mientras que la ramificación en duelo capacita al aproximador convolucional para aprender qué estados de la carretera son intrínsecamente seguros o peligrosos sin necesidad de evaluar el impacto individual de cada acción lateral, el desacoplamiento de Double DQN impide que los errores de aproximación iniciales de la red en línea inflen artificialmente los objetivos de Bellman. Al fusionar ambas técnicas, la función de pérdida del gradiente se estabiliza de manera significativa, acelerando la identificación de la política óptima de aceleración a través de muestras libres de sesgo maximizador.

---

#### o Directrices de la propuesta unificada

- El sistema recupera el estado de los pesos del disco persistente mediante el deserializador jerárquico para garantizar tolerancia completa a fallos y evitar la duplicación de interacciones.
- Se emplea la topología convolucional `build_model_dueling_ready` para dotar al backend de la estructura de capas idónea para la separación de valor y ventaja.
- Se compila el agente activando de forma simultánea las banderas lógicas de doble red y red en duelo ( `enable_double_dqn=True` , `enable_dueling_network=True` ) junto a la agregación por media aritmética.
- El entrenamiento y la posterior inferencia estadística se encuentran gobernados por la directiva de la guardia de control principal para automatizar la persistencia y la evaluación en modo de prueba pura.

### 4.6.1. P6 — Policy

```
In [24]: # PROPUESTA 6: Double DQN + Dueling DQN combinados

RESUME_FROM_P6 = None
STEP_OFFSET_P6 = 0

# Descomentar esta línea para detectar automáticamente el último checkpoint guar
RESUME_FROM_P6, STEP_OFFSET_P6 = find_latest_checkpoint(WEIGHTS_DIR_P6, 'dqn_End

model_p6 = build_model_dueling_ready(nb_actions, input_shape)
memory_p6 = SequentialMemory(limit=1000000, window_length=WINDOW_LENGTH)

policy_p6 = EpsGreedyQPolicy(eps=0.05)

dqn_p6 = build_dqn_agent(
    model=model_p6, nb_actions=nb_actions, memory=memory_p6,
    processor=AtariProcessor(), policy=policy_p6,
    nb_steps_warmup=50000, target_model_update=10000,
    enable_double_dqn=True, enable_dueling_network=True, dueling_type='avg'
)

load_weights_for_test(dqn_p6, 'p6', WEIGHTS_DIR_P6)
```

```
AGENTS_REGISTRY['p6']['agent'] = dqn_p6 # ← se registra aquí
```

[p6] Pesos cargados para test desde: /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerzo\_Grupo\_17\_2025\_2026/checkpoints/p6\_double\_dueling/dqn\_Enduro-v0\_p6\_weights\_280000.h5f (step 2800000)

## 4.6.2. P6 — Historial de cambios

### Historial de entrenamiento:

- Steps 0 → 1.8M: EpsGreedyQPolicy con eps=0.2 fijo
- Steps 1.8M → 2.3M: EpsGreedyQPolicy con eps=0.1 fijo
- Steps 2.3M → 2.8M: EpsGreedyQPolicy con eps=0.05 fijo

**Decisión de diseño:** P6 combina Double DQN (Van Hasselt et al., 2016) con Dueling DQN (Wang et al., 2016) — la variante más completa del proyecto. El objetivo es estudiar si la combinación de ambas mejoras algorítmicas supera a cada una por separado (P4 solo Double, P5 solo Dueling).

El epsilon se redujo progresivamente en cada tanda de entrenamiento (0.2 → 0.1 → 0.05) para ir refinando el comportamiento en el tramo final del día 1 conforme el agente maduraba, reduciendo el ruido en las decisiones críticas sin eliminar completamente la exploración.

## 4.6.3. P6 — Entrenamiento

```
In [ ]: # Usar TRUE para entrenar, o FALSE para saltar el entrenamiento
RUN_TRAINING_P6 = False # ← GUARDIA PRINCIPAL

if RUN_TRAINING_P6:

    if RESUME_FROM_P6 is not None:
        print(f"Reanudando Propuesta 6 desde el checkpoint: {RESUME_FROM_P6}")
        dqn_p6.load_weights(RESUME_FROM_P6)
        NB_STEPS_REMAINING_P6 = max(0, NB_STEPS_TRAIN - STEP_OFFSET_P6)
    else:
        NB_STEPS_REMAINING_P6 = NB_STEPS_TRAIN

    weights_filename_p6 = os.path.join(WEIGHTS_DIR_P6, 'dqn_{}_p6_double_dueling'
checkpoint_filename_p6 = os.path.join(WEIGHTS_DIR_P6, 'dqn_' + env_name + '_'
log_filename_p6 = os.path.join(WEIGHTS_DIR_P6, 'dqn_{}_p6_log.json'.format(e

    callbacks_p6 = [ResumableModelIntervalCheckpoint(checkpoint_filename_p6, int
step_offset=STEP_OFFSET_P6)]
    callbacks_p6 += [FileLogger(log_filename_p6, interval=1000)]

    history_p6 = dqn_p6.fit(env, callbacks=callbacks_p6, nb_steps=NB_STEPS_REMAI
log_interval=10000, visualize=False)

    dqn_p6.save_weights(weights_filename_p6, overwrite=True)
    print("Pesos finales de la Propuesta 6 guardados en Drive:", weights_filename
    print("Contenido de la carpeta:", os.listdir(WEIGHTS_DIR_P6))

else:
    print("[P6] Entrenamiento omitido (RUN_TRAINING_P6=False).")
```

```

Reanudando Propuesta 6 desde el checkpoint: /content/gdrive/My Drive/VIU_Apr_por_
Refuerzo_Grupo_17_2025_2026/checkpoints/p6_double_dueling/dqn_Enduro-v0_p6_weight
s_1800000.h5f
Training for 500000 steps ...
Interval 1 (0 steps performed)
  13/10000 [.....] - ETA: 43s - reward: 0.0000e+00
/usr/local/lib/python3.12/dist-packages/tf_keras/src/engine/training_v1.py:2354:
UserWarning: `Model.state_updates` will be removed in a future version. This prop
erty should not be used in TensorFlow 2.0, as `updates` are applied automaticall
y.
  updates=self.state_updates,
  37/10000 [.....] - ETA: 43s - reward: 0.0000e+00
10000/10000 [=====] - 49s 5ms/step - reward: 0.0386
2 episodes - episode_reward: 163.500 [163.000, 164.000] - ale.lives: 0.000

Interval 2 (10000 steps performed)
10000/10000 [=====] - 49s 5ms/step - reward: 0.0393
1 episodes - episode_reward: 365.000 [365.000, 365.000] - ale.lives: 0.000

Interval 3 (20000 steps performed)
 430/10000 [>.....] - ETA: 1:09 - reward: 0.0605

```

#### 4.6.4. P6 — Testeo

```

In [27]: PROPUESTA_A_EVALUAR = 'p6'    # ← único parámetro a tocar

test_history = test_proposal(PROPUESTA_A_EVALUAR, AGENTS_REGISTRY, nb_episodes=4

[p6] Cargando pesos desde: /content/gdrive/My Drive/VIU_Apr_por_Refuerzo_Grupo_17
_2025_2026/checkpoints/p6_double_dueling/dqn_Enduro-v0_p6_weights_2800000.h5f (s
tep 2800000)
Testing for 40 episodes ...

```

Episode 1: reward: 476.000, steps: 8901  
Episode 2: reward: 457.000, steps: 8844  
Episode 3: reward: 557.000, steps: 10000  
Episode 4: reward: 434.000, steps: 8913  
Episode 5: reward: 195.000, steps: 4433  
Episode 6: reward: 541.000, steps: 10000  
Episode 7: reward: 410.000, steps: 8861  
Episode 8: reward: 447.000, steps: 8874  
Episode 9: reward: 426.000, steps: 8898  
Episode 10: reward: 439.000, steps: 8896  
Episode 11: reward: 556.000, steps: 10000  
Episode 12: reward: 398.000, steps: 8872  
Episode 13: reward: 454.000, steps: 8906  
Episode 14: reward: 418.000, steps: 8913  
Episode 15: reward: 454.000, steps: 8865  
Episode 16: reward: 404.000, steps: 8863  
Episode 17: reward: 414.000, steps: 8875  
Episode 18: reward: 473.000, steps: 8855  
Episode 19: reward: 438.000, steps: 8905  
Episode 20: reward: 428.000, steps: 8871  
Episode 21: reward: 424.000, steps: 8897  
Episode 22: reward: 437.000, steps: 8834  
Episode 23: reward: 199.000, steps: 4452  
Episode 24: reward: 432.000, steps: 8925  
Episode 25: reward: 414.000, steps: 8868  
Episode 26: reward: 428.000, steps: 8830  
Episode 27: reward: 400.000, steps: 8836  
Episode 28: reward: 449.000, steps: 8874  
Episode 29: reward: 475.000, steps: 8882  
Episode 30: reward: 434.000, steps: 8868  
Episode 31: reward: 452.000, steps: 8869  
Episode 32: reward: 470.000, steps: 8846  
Episode 33: reward: 413.000, steps: 8890  
Episode 34: reward: 479.000, steps: 8850  
Episode 35: reward: 525.000, steps: 10000  
Episode 36: reward: 410.000, steps: 8855  
Episode 37: reward: 396.000, steps: 8868  
Episode 38: reward: 370.000, steps: 8865  
Episode 39: reward: 426.000, steps: 8856  
Episode 40: reward: 407.000, steps: 8878  
[p6] Recompensa media en 40 episodios: 431.48 (+/- 67.97)

---

#### 4.6.5. P6 — Resultados y estado final

##### **Resultado del test (checkpoint 2.8M, evaluación greedy, 40 episodios):**

- 38/40 episodios superan el día 1 (95%)
- Media: 431.5 coches | Std: 68.0 | Mín: 195 | Máx: 557
- 2 episodios cortos con rewards: 195 y 199 (a 5 y 1 coches del objetivo)
- 4/40 episodios llegaron al día 3 (10.000 steps)
- Tasa de fallo estimada: 5% →  $P(100 \text{ consecutivos}) \approx 0.6\%$

**Análisis:** P6 tiene la media más consistente de todas las propuestas avanzadas, pero no llega a cumplir el objetivo de 100 consecutivos — los 2 episodios cortos que fallan lo hacen por muy poco (195 y 199 coches).

Sorprendentemente, P5 (Dueling solo) supera a P6 (Double+Dueling) en robustez: P5 consigue 100/100 mientras P6 se queda en ~95%. Esto sugiere que para Enduro-v0 con el nivel de entrenamiento actual (~2.8M steps vs 12.5M del paper), añadir Double DQN sobre Dueling no aporta mejora adicional en consistencia. Con más steps de entrenamiento, la combinación Double+Dueling probablemente mostraría la ventaja predicha por los papers.

**Trabajo futuro:** continuar entrenando hasta 3.5M steps con  $\epsilon=0.05$  para reducir la tasa de fallo por debajo del 1% y conseguir los 100 consecutivos.

---

## 4.7. Propuesta 7: Double DQN con annealing de epsilon

---

*En este apartado se instrumenta la séptima propuesta experimental, la cual unifica la mejor configuración estructural obtenida (Double Dueling DQN) con el esquema de exploración por decaimiento dinámico-lineal para evaluar su efecto conjunto en la estabilidad del agente.*

### ◦ Fundamentación de la sinergia estructural y exploratoria

Una vez consolidada la robustez de la topología híbrida que integra Double DQN y Dueling DQN, resulta analíticamente necesario retirar la restricción del coeficiente de exploración fijo para restituir el dinamismo temporal en la recolección de muestras. La introducción de una planificación por decaimiento lineal sobre una base de doble red en duelo busca resolver de forma simultánea las grandes deficiencias del entorno de simulación.

Por un lado, el desglosamiento jerárquico de ventajas aísla el valor de la carretera mientras que el doble operador mitiga la sobreestimación de los premios. Por otro lado, la reducción progresiva del parámetro  $\epsilon$  desde la aleatoriedad total hasta el valor residual garantiza que el agente disponga de un periodo inicial lo suficientemente rico como para descubrir de forma fortuita la acción de aceleración ( FIRE ), pasando de manera suave a una fase de explotación pura donde el aproximador convolucional pule la política de conducción sin la interferencia de ruido uniforme continuo.

### ◦ Directrices de la propuesta combinada

- La infraestructura del agente recupera automáticamente el estado de los pesos desde la cuenta persistente mediante el deserializador para blindar el experimento contra interrupciones en el hilo de ejecución.
- Se utiliza el constructor de red preparado para duelo junto con el objeto de planificación lineal que reduce linealmente el coeficiente de exploración a lo largo del horizonte completo de interacciones.
- Las banderas operativas del agente activan de forma simultánea el desacoplamiento doble y la bifurcación por media para asegurar la sinergia algorítmica total.

- El control del entrenamiento se delega en la guardia principal y su validación final se procesa a través del método de inferencia estadística pura.

#### 4.7.1. P7 — Policy

```
In [28]: # PROPUESTA 7: Double DQN con annealing de epsilon

RESUME_FROM_P7 = None
STEP_OFFSET_P7 = 0

# Descomentar esta línea para detectar automáticamente el último checkpoint guardado
RESUME_FROM_P7, STEP_OFFSET_P7 = find_latest_checkpoint(WEIGHTS_DIR_P7, 'dqn_Enduro-v0_p7_weights_1200000.h5f')

model_p7 = build_model(nb_actions, input_shape)
memory_p7 = SequentialMemory(limit=1000000, window_length=WINDOW_LENGTH)

weights_filename_p7 = os.path.join(WEIGHTS_DIR_P7, 'dqn_{}_p7_weights.h5f'.format(env_name))
checkpoint_filename_p7 = os.path.join(WEIGHTS_DIR_P7, 'dqn_' + env_name + '_p7_weights_1200000.h5f')
log_filename_p7 = os.path.join(WEIGHTS_DIR_P7, 'dqn_{}_p7_log.json'.format(env_name))

# Annealing de epsilon 1.0 -> 0.1 en 1.2M steps. Al reanudar, usamos
# OffsetLinearAnnealedPolicy para que el eps continúe desde donde
# realmente se quedó en vez de reiniciarse a 1.0. Si ya se superaron
# los 1.2M steps, eps arrancará directamente en 0.1.
_eps_log_value, _eps_log_step = read_last_annealed_value_from_log(log_filename_p7)
if _eps_log_value is not None:
    print(f"Reanudando annealing de epsilon: último valor = {_eps_log_value:.4f}")
    _eps_step_offset_p7 = _eps_log_step
else:
    # Sin log previo: calculamos el offset a partir del step del checkpoint
    _eps_step_offset_p7 = STEP_OFFSET_P7

policy_p7 = OffsetLinearAnnealedPolicy(
    EpsGreedyQPolicy(), attr='eps',
    value_max=1., value_min=.1, value_test=.05,
    nb_steps=1200000,
    step_offset=_eps_step_offset_p7 # continúa el annealing desde el punto real
)

dqn_p7 = build_dqn_agent(
    model=model_p7, nb_actions=nb_actions, memory=memory_p7,
    processor=AtariProcessor(), policy=policy_p7,
    nb_steps_warmup=50000, target_model_update=10000,
    enable_double_dqn=True, enable_dueling_network=False
)

load_weights_for_test(dqn_p7, 'p7', WEIGHTS_DIR_P7)

AGENTS_REGISTRY['p7']['agent'] = dqn_p7 # ← se registra aquí
```

Reanudando annealing de epsilon: último valor = 0.3793 (step 798780)  
 [p7] Pesos cargados para test desde: /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerzo/Grupo\_17\_2025\_2026/checkpoints/p7\_double\_dqn\_annealed/dqn\_Enduro-v0\_p7\_weights\_1200000.h5f (step 1200000)

#### 4.7.2. P7 — Historial de cambios

- Steps 0 → 1.2M: OffsetLinearAnnealedPolicy con nb\_steps=1.200.000 → epsilon bajó de 1.0 a 0.1 exactamente en el último step

**Decisión de diseño:** P7 combina Double DQN con annealing lineal de epsilon, para estudiar si el decaimiento gradual de exploración mejora Double DQN respecto a epsilon fijo (P4). Se eligió nb\_steps=1.200.000 intentando dar al agente suficiente tiempo de exploración antes de explotación.

**Problema detectado — nb\_steps demasiado alto:** Con nb\_steps=1.200.000 y un entrenamiento total de 1.2M steps, el epsilon estuvo siempre por encima de 0.1 durante todo el entrenamiento:

- Step 100k: eps=0.925 (92% acciones aleatorias)
- Step 300k: eps=0.775 (78% acciones aleatorias)
- Step 800k: eps=0.400 (40% acciones aleatorias)
- Step 1.2M: eps=0.100 (llegó al mínimo en el último step)

El agente nunca tuvo ni un solo step de entrenamiento con epsilon bajo. Es el caso más extremo del problema de annealing detectado también en P2 y P8 — en P7, el 100% del entrenamiento fue con exploración alta.

El valor correcto según Mnih et al. (2015) es annealing en 1M frames, equivalente a 250.000 steps con frame\_skip=4. Con nb\_steps=300.000, P7 habría tenido 900k steps útiles de 1.2M totales (75%).

### 4.7.3. P7 — Entrenamiento

```
In [ ]: # Usar TRUE para entrenar, o FALSE para saltar el entrenamiento
RUN_TRAINING_P7 = False # <— GUARDIA PRINCIPAL

if RUN_TRAINING_P7:

    if RESUME_FROM_P7 is not None:
        print(f"Reanudando Propuesta 7 desde: {RESUME_FROM_P7}")
        dqn_p7.load_weights(RESUME_FROM_P7)
        NB_STEPS_REMAINING_P7 = max(0, NB_STEPS_TRAIN - STEP_OFFSET_P7)
    else:
        NB_STEPS_REMAINING_P7 = NB_STEPS_TRAIN

    callbacks_p7 = [ResumableModelIntervalCheckpoint(checkpoint_filename_p7, int
                                                    step_offset=STEP_OFFSET_P7)]
    callbacks_p7 += [FileLogger(log_filename_p7, interval=1000)]

    history_p7 = dqn_p7.fit(env, callbacks=callbacks_p7, nb_steps=NB_STEPS_REMAI
                            log_interval=10000, visualize=False)

    dqn_p7.save_weights(weights_filename_p7, overwrite=True)
    print("Pesos finales de la Propuesta 7 guardados:", weights_filename_p7)
    print("Contenido de la carpeta:", os.listdir(WEIGHTS_DIR_P7))

else:
    print("[P7] Entrenamiento omitido (RUN_TRAINING_P7=False).")
```

Reanudando Propuesta 7 desde: /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerzo\_Grupo\_17\_2025\_2026/checkpoints/p7\_double\_dqn\_annealed/dqn\_Enduro-v0\_p7\_weights\_400000.h5f

Training for 800000 steps ...

Interval 1 (0 steps performed)

10000/10000 [=====] - 49s 5ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - ale.lives: 0.000

Interval 2 (10000 steps performed)

10000/10000 [=====] - 47s 5ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - ale.lives: 0.000

Interval 3 (20000 steps performed)

10000/10000 [=====] - 47s 5ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - ale.lives: 0.000

Interval 4 (30000 steps performed)

10000/10000 [=====] - 49s 5ms/step - reward: 0.0000e+00  
3 episodes - episode\_reward: 0.000 [0.000, 0.000] - ale.lives: 0.000

Interval 5 (40000 steps performed)

10000/10000 [=====] - 47s 5ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - ale.lives: 0.000

Interval 6 (50000 steps performed)

10000/10000 [=====] - 346s 35ms/step - reward: 0.0000e+00  
0  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.007 - mean\_q: 0.008 - mean\_eps: 0.935 - ale.lives: 0.000

Interval 7 (60000 steps performed)

10000/10000 [=====] - 338s 34ms/step - reward: 0.0000e+00  
0  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.007 - mean\_q: 0.008 - mean\_eps: 0.928 - ale.lives: 0.000

Interval 8 (70000 steps performed)

10000/10000 [=====] - 342s 34ms/step - reward: 0.0000e+00  
0  
3 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.007 - mean\_q: 0.008 - mean\_eps: 0.920 - ale.lives: 0.000

Interval 9 (80000 steps performed)

10000/10000 [=====] - 343s 34ms/step - reward: 0.0000e+00  
0  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.007 - mean\_q: 0.008 - mean\_eps: 0.913 - ale.lives: 0.000

Interval 10 (90000 steps performed)

10000/10000 [=====] - 343s 34ms/step - reward: 0.0000e+00  
0  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.007 - mean\_q: 0.008 - mean\_eps: 0.905 - ale.lives: 0.000

Interval 11 (100000 steps performed)

10000/10000 [=====] - 340s 34ms/step - reward: 0.0000e+00  
0  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.007 - mean\_q: 0.008 - mean\_eps: 0.898 - ale.lives: 0.000

Interval 12 (110000 steps performed)  
10000/10000 [=====] - 346s 35ms/step - reward: 0.0000e+00  
3 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.007 - mean\_q: 0.008 - mean\_eps: 0.890 - ale.lives: 0.000

Interval 13 (120000 steps performed)  
10000/10000 [=====] - 342s 34ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.007 - mean\_q: 0.008 - mean\_eps: 0.883 - ale.lives: 0.000

Interval 14 (130000 steps performed)  
10000/10000 [=====] - 350s 35ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.007 - mean\_q: 0.008 - mean\_eps: 0.875 - ale.lives: 0.000

Interval 15 (140000 steps performed)  
10000/10000 [=====] - 351s 35ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.007 - mean\_q: 0.008 - mean\_eps: 0.868 - ale.lives: 0.000

Interval 16 (150000 steps performed)  
10000/10000 [=====] - 352s 35ms/step - reward: 0.0000e+00  
3 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.007 - mean\_q: 0.007 - mean\_eps: 0.860 - ale.lives: 0.000

Interval 17 (160000 steps performed)  
10000/10000 [=====] - 350s 35ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.007 - mean\_eps: 0.853 - ale.lives: 0.000

Interval 18 (170000 steps performed)  
10000/10000 [=====] - 347s 35ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.007 - mean\_eps: 0.845 - ale.lives: 0.000

Interval 19 (180000 steps performed)  
10000/10000 [=====] - 356s 36ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.007 - mean\_eps: 0.838 - ale.lives: 0.000

Interval 20 (190000 steps performed)  
10000/10000 [=====] - 355s 36ms/step - reward: 0.0000e+00  
3 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.007 - mean\_eps: 0.830 - ale.lives: 0.000

Interval 21 (200000 steps performed)  
10000/10000 [=====] - 358s 36ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.007 - mean\_eps: 0.823 - ale.lives: 0.000

Interval 22 (210000 steps performed)  
10000/10000 [=====] - 361s 36ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.007 - mean\_eps: 0.815 - ale.lives: 0.000

Interval 23 (220000 steps performed)  
10000/10000 [=====] - 361s 36ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.007 - mean\_eps: 0.808 - ale.lives: 0.000

Interval 24 (230000 steps performed)  
10000/10000 [=====] - 362s 36ms/step - reward: 0.0000e+00  
3 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.007 - mean\_eps: 0.800 - ale.lives: 0.000

Interval 25 (240000 steps performed)  
10000/10000 [=====] - 363s 36ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.007 - mean\_eps: 0.793 - ale.lives: 0.000

Interval 26 (250000 steps performed)  
10000/10000 [=====] - 367s 37ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.007 - mean\_eps: 0.785 - ale.lives: 0.000

Interval 27 (260000 steps performed)  
10000/10000 [=====] - 368s 37ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.007 - mean\_eps: 0.778 - ale.lives: 0.000

Interval 28 (270000 steps performed)  
10000/10000 [=====] - 371s 37ms/step - reward: 0.0000e+00  
3 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.007 - mean\_eps: 0.770 - ale.lives: 0.000

Interval 29 (280000 steps performed)  
10000/10000 [=====] - 371s 37ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.007 - mean\_eps: 0.763 - ale.lives: 0.000

Interval 30 (290000 steps performed)  
10000/10000 [=====] - 375s 37ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.007 - mean\_eps: 0.755 - ale.lives: 0.000

Interval 31 (300000 steps performed)  
10000/10000 [=====] - 376s 38ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.007 - mean\_eps: 0.748 - ale.lives: 0.000

Interval 32 (310000 steps performed)  
10000/10000 [=====] - 380s 38ms/step - reward: 0.0000e+00  
3 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.007 - mean\_eps: 0.740 - ale.lives: 0.000

Interval 33 (320000 steps performed)  
10000/10000 [=====] - 383s 38ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.007 - mean\_eps: 0.733 - ale.lives: 0.000

Interval 34 (330000 steps performed)  
10000/10000 [=====] - 380s 38ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.007 - mean\_eps: 0.725 - ale.lives: 0.000

Interval 35 (340000 steps performed)  
10000/10000 [=====] - 382s 38ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.007 - mean\_eps: 0.718 - ale.lives: 0.000

Interval 36 (350000 steps performed)  
10000/10000 [=====] - 381s 38ms/step - reward: 0.0000e+00  
3 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.007 - mean\_eps: 0.710 - ale.lives: 0.000

Interval 37 (360000 steps performed)  
10000/10000 [=====] - 385s 39ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.007 - mean\_eps: 0.703 - ale.lives: 0.000

Interval 38 (370000 steps performed)  
10000/10000 [=====] - 387s 39ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.007 - mean\_eps: 0.695 - ale.lives: 0.000

Interval 39 (380000 steps performed)  
10000/10000 [=====] - 390s 39ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.007 - mean\_eps: 0.688 - ale.lives: 0.000

Interval 40 (390000 steps performed)  
10000/10000 [=====] - 396s 40ms/step - reward: 0.0000e+00  
3 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.006 - mean\_eps: 0.680 - ale.lives: 0.000

Interval 41 (400000 steps performed)  
10000/10000 [=====] - 397s 40ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.006 - mean\_eps: 0.673 - ale.lives: 0.000

Interval 42 (410000 steps performed)  
10000/10000 [=====] - 397s 40ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.005 - mean\_q: 0.006 - mean\_eps: 0.665 - ale.lives: 0.000

Interval 43 (420000 steps performed)  
10000/10000 [=====] - 403s 40ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.007 - mean\_eps: 0.658 - ale.lives: 0.000

Interval 44 (430000 steps performed)  
10000/10000 [=====] - 403s 40ms/step - reward: 0.0000e+00  
3 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.006 - mean\_eps: 0.650 - ale.lives: 0.000

Interval 45 (440000 steps performed)  
10000/10000 [=====] - 406s 41ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.006 - mean\_eps: 0.643 - ale.lives: 0.000

Interval 46 (450000 steps performed)  
10000/10000 [=====] - 405s 40ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.007 - mean\_eps: 0.635 - ale.lives: 0.000

Interval 47 (460000 steps performed)  
10000/10000 [=====] - 410s 41ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.007 - mean\_eps: 0.628 - ale.lives: 0.000

Interval 48 (470000 steps performed)  
10000/10000 [=====] - 411s 41ms/step - reward: 0.0000e+00  
3 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.007 - mean\_eps: 0.620 - ale.lives: 0.000

Interval 49 (480000 steps performed)  
10000/10000 [=====] - 414s 41ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.007 - mean\_eps: 0.613 - ale.lives: 0.000

Interval 50 (490000 steps performed)  
10000/10000 [=====] - 421s 42ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.006 - mean\_eps: 0.605 - ale.lives: 0.000

Interval 51 (500000 steps performed)  
10000/10000 [=====] - 419s 42ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.006 - mean\_q: 0.006 - mean\_eps: 0.598 - ale.lives: 0.000

Interval 52 (510000 steps performed)  
10000/10000 [=====] - 422s 42ms/step - reward: 0.0000e+00  
3 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.005 - mean\_q: 0.006 - mean\_eps: 0.590 - ale.lives: 0.000

Interval 53 (520000 steps performed)  
10000/10000 [=====] - 423s 42ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.005 - mean\_q: 0.006 - mean\_eps: 0.583 - ale.lives: 0.000

Interval 54 (530000 steps performed)  
10000/10000 [=====] - 427s 43ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.005 - mean\_q: 0.006 - mean\_eps: 0.575 - ale.lives: 0.000

Interval 55 (540000 steps performed)  
10000/10000 [=====] - 431s 43ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.005 - mean\_q: 0.006 - mean\_eps: 0.568 - ale.lives: 0.000

Interval 56 (550000 steps performed)  
10000/10000 [=====] - 435s 43ms/step - reward: 0.0000e+00  
3 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.005 - mean\_q: 0.006 - mean\_eps: 0.560 - ale.lives: 0.000

Interval 57 (560000 steps performed)  
10000/10000 [=====] - 441s 44ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.005 - mean\_q: 0.006 - mean\_eps: 0.553 - ale.lives: 0.000

Interval 58 (570000 steps performed)  
10000/10000 [=====] - 441s 44ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.005 - mean\_q: 0.006 - mean\_eps: 0.545 - ale.lives: 0.000

Interval 59 (580000 steps performed)  
10000/10000 [=====] - 447s 45ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.005 - mean\_q: 0.006 - mean\_eps: 0.538 - ale.lives: 0.000

Interval 60 (590000 steps performed)  
10000/10000 [=====] - 449s 45ms/step - reward: 0.0000e+00  
3 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.005 - mean\_q: 0.006 - mean\_eps: 0.530 - ale.lives: 0.000

Interval 61 (600000 steps performed)  
10000/10000 [=====] - 444s 44ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.005 - mean\_q: 0.006 - mean\_eps: 0.523 - ale.lives: 0.000

Interval 62 (610000 steps performed)  
10000/10000 [=====] - 450s 45ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.005 - mean\_q: 0.005 - mean\_eps: 0.515 - ale.lives: 0.000

Interval 63 (620000 steps performed)  
10000/10000 [=====] - 455s 45ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.005 - mean\_q: 0.006 - mean\_eps: 0.508 - ale.lives: 0.000

Interval 64 (630000 steps performed)  
10000/10000 [=====] - 459s 46ms/step - reward: 0.0000e+00  
3 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.005 - mean\_q: 0.005 - mean\_eps: 0.500 - ale.lives: 0.000

Interval 65 (640000 steps performed)  
10000/10000 [=====] - 461s 46ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.005 - mean\_q: 0.005 - mean\_eps: 0.493 - ale.lives: 0.000

Interval 66 (650000 steps performed)  
10000/10000 [=====] - 464s 46ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.005 - mean\_q: 0.006 - mean\_eps: 0.485 - ale.lives: 0.000

Interval 67 (660000 steps performed)  
10000/10000 [=====] - 466s 47ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.005 - mean\_q: 0.006 - mean\_eps: 0.478 - ale.lives: 0.000

Interval 68 (670000 steps performed)  
10000/10000 [=====] - 468s 47ms/step - reward: 0.0000e+00  
3 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.005 - mean\_q: 0.005 - mean\_eps: 0.470 - ale.lives: 0.000

Interval 69 (680000 steps performed)  
10000/10000 [=====] - 473s 47ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.005 - mean\_q: 0.005 - mean\_eps: 0.463 - ale.lives: 0.000

Interval 70 (690000 steps performed)  
10000/10000 [=====] - 476s 48ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.005 - mean\_q: 0.005 - mean\_eps: 0.455 - ale.lives: 0.000

Interval 71 (700000 steps performed)  
10000/10000 [=====] - 482s 48ms/step - reward: 0.0000e+00  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.004 - mean\_q: 0.005 - mean\_eps: 0.448 - ale.lives: 0.000

```

Interval 72 (710000 steps performed)
10000/10000 [=====] - 484s 48ms/step - reward: 0.0000e+0
0
3 episodes - episode_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.005 - me
an_q: 0.006 - mean_eps: 0.440 - ale.lives: 0.000

Interval 73 (720000 steps performed)
10000/10000 [=====] - 486s 49ms/step - reward: 1.0000e-0
4
2 episodes - episode_reward: 0.500 [0.000, 1.000] - loss: 0.000 - mae: 0.005 - me
an_q: 0.006 - mean_eps: 0.433 - ale.lives: 0.000

Interval 74 (730000 steps performed)
10000/10000 [=====] - 489s 49ms/step - reward: 0.0000e+0
0
2 episodes - episode_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.005 - me
an_q: 0.005 - mean_eps: 0.425 - ale.lives: 0.000

Interval 75 (740000 steps performed)
10000/10000 [=====] - 492s 49ms/step - reward: 0.0000e+0
0
3 episodes - episode_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.005 - me
an_q: 0.005 - mean_eps: 0.418 - ale.lives: 0.000

Interval 76 (750000 steps performed)
10000/10000 [=====] - 496s 50ms/step - reward: 0.0000e+0
0
2 episodes - episode_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.005 - me
an_q: 0.006 - mean_eps: 0.410 - ale.lives: 0.000

Interval 77 (760000 steps performed)
10000/10000 [=====] - 503s 50ms/step - reward: 0.0000e+0
0
2 episodes - episode_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.005 - me
an_q: 0.005 - mean_eps: 0.403 - ale.lives: 0.000

Interval 78 (770000 steps performed)
10000/10000 [=====] - 505s 50ms/step - reward: 0.0000e+0
0
2 episodes - episode_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.005 - me
an_q: 0.005 - mean_eps: 0.395 - ale.lives: 0.000

Interval 79 (780000 steps performed)
10000/10000 [=====] - 508s 51ms/step - reward: 0.0000e+0
0
3 episodes - episode_reward: 0.000 [0.000, 0.000] - loss: 0.000 - mae: 0.005 - me
an_q: 0.006 - mean_eps: 0.388 - ale.lives: 0.000

Interval 80 (790000 steps performed)
10000/10000 [=====] - 511s 51ms/step - reward: 0.0000e+0
0
done, took 30912.715 seconds
Pesos finales de la Propuesta 7 guardados: /content/gdrive/My Drive/VIU_Apr_por_R
efuerzo_Grupo_17_2025_2026/checkpoints/p7_double_dqn_annealed/dqn_Enduro-v0_p7_we
ights.h5f
Contenido de la carpeta: ['dqn_Enduro-v0_p7_weights.h5f.data-00000-of-00001', 'dq
n_Enduro-v0_p7_weights.h5f.index', 'dqn_Enduro-v0_p7_log.json', 'dqn_Enduro-v0_p7
_weights_100000.h5f.data-00000-of-00001', 'dqn_Enduro-v0_p7_weights_100000.h5f.in
dex', 'dqn_Enduro-v0_p7_weights_200000.h5f.data-00000-of-00001', 'dqn_Enduro-v0_p
7_weights_200000.h5f.index', 'dqn_Enduro-v0_p7_weights_300000.h5f.data-00000-of-0

```

```
0001', 'dqn_Enduro-v0_p7_weights_300000.h5f.index', 'dqn_Enduro-v0_p7_weights_400000.h5f.data-00000-of-00001', 'dqn_Enduro-v0_p7_weights_400000.h5f.index', 'check point', 'dqn_Enduro-v0_p7_weights_500000.h5f.data-00000-of-00001', 'dqn_Enduro-v0_p7_weights_500000.h5f.index', 'dqn_Enduro-v0_p7_weights_600000.h5f.data-00000-of-00001', 'dqn_Enduro-v0_p7_weights_600000.h5f.index', 'dqn_Enduro-v0_p7_weights_700000.h5f.data-00000-of-00001', 'dqn_Enduro-v0_p7_weights_700000.h5f.index', 'dqn_Enduro-v0_p7_weights_800000.h5f.data-00000-of-00001', 'dqn_Enduro-v0_p7_weights_800000.h5f.index', 'dqn_Enduro-v0_p7_weights_900000.h5f.data-00000-of-00001', 'dqn_Enduro-v0_p7_weights_900000.h5f.index', 'dqn_Enduro-v0_p7_weights_1000000.h5f.data-00000-of-00001', 'dqn_Enduro-v0_p7_weights_1000000.h5f.index', 'dqn_Enduro-v0_p7_weights_1100000.h5f.data-00000-of-00001', 'dqn_Enduro-v0_p7_weights_1100000.h5f.index', 'dqn_Enduro-v0_p7_weights_1200000.h5f.data-00000-of-00001', 'dqn_Enduro-v0_p7_weights_1200000.h5f.index']
```

#### 4.7.4. P7 — Testeo

```
In [29]: PROPUESTA_A_EVALUAR = 'p7'    # ← único parámetro a tocar

test_history = test_proposal(PROPUESTA_A_EVALUAR, AGENTS_REGISTRY, nb_episodes=3
```

```
[p7] Cargando pesos desde: /content/gdrive/My Drive/VIU_Apr_por_Refuerzo_Grupo_17_2025_2026/checkpoints/p7_double_dqn_annealed/dqn_Enduro-v0_p7_weights_1200000.h5f (step 1200000)
Testing for 30 episodes ...
```

```
/usr/local/lib/python3.11/dist-packages/tf_keras/src/engine/training_v1.py:2354:
UserWarning: `Model.state_updates` will be removed in a future version. This property should not be used in TensorFlow 2.0, as `updates` are applied automatically.
    updates=self.state_updates,
```

Episode 1: reward: 0.000, steps: 4462  
Episode 2: reward: 0.000, steps: 4426  
Episode 3: reward: 0.000, steps: 4420  
Episode 4: reward: 0.000, steps: 4429  
Episode 5: reward: 0.000, steps: 4427  
Episode 6: reward: 0.000, steps: 4461  
Episode 7: reward: 0.000, steps: 4443  
Episode 8: reward: 0.000, steps: 4438  
Episode 9: reward: 0.000, steps: 4407  
Episode 10: reward: 0.000, steps: 4443  
Episode 11: reward: 0.000, steps: 4454  
Episode 12: reward: 0.000, steps: 4445  
Episode 13: reward: 0.000, steps: 4456  
Episode 14: reward: 0.000, steps: 4466  
Episode 15: reward: 0.000, steps: 4431  
Episode 16: reward: 0.000, steps: 4451  
Episode 17: reward: 0.000, steps: 4418  
Episode 18: reward: 0.000, steps: 4465  
Episode 19: reward: 0.000, steps: 4433  
Episode 20: reward: 0.000, steps: 4454  
Episode 21: reward: 0.000, steps: 4423  
Episode 22: reward: 0.000, steps: 4435  
Episode 23: reward: 0.000, steps: 4427  
Episode 24: reward: 0.000, steps: 4425  
Episode 25: reward: 0.000, steps: 4412  
Episode 26: reward: 0.000, steps: 4437  
Episode 27: reward: 0.000, steps: 4434  
Episode 28: reward: 0.000, steps: 4460  
Episode 29: reward: 0.000, steps: 4426  
Episode 30: reward: 0.000, steps: 4461  
[p7] Recompensa media en 30 episodios: 0.00 (+/- 0.00)

---

#### 4.7.5. P7 — Resultados y estado final

##### Resultado del test (checkpoint 1.2M, evaluación greedy):

- 0/30 episodios superan el día 1
- Media: 0.0 coches en todos los episodios
- El agente no aprendió ninguna política útil

**Análisis:** P7 representa el fracaso más claro del proyecto, causado exclusivamente por una misconfiguration del annealing. Con Double DQN y nb\_steps=300.000, el resultado habría sido comparable a P4 (94/100, media 397) o mejor, dado que el annealing correcto combina exploración inicial amplia con explotación posterior — exactamente el escenario ideal para Double DQN.

P7 junto con P2 y P8 ilustran un patrón importante: en entornos con recompensas escasas como Enduro-v0, el annealing de epsilon requiere una calibración muy precisa. Un nb\_steps demasiado alto convierte el annealing en exploración casi pura, lo que impide cualquier aprendizaje.

**Trabajo futuro:** reentrenar con nb\_steps=300.000 desde cero. Con la corrección aplicada, P7 (Double DQN + annealing) debería superar a P4 (Double DQN + eps fijo) demostrando la ventaja real del annealing cuando está correctamente configurado.

---

## 4.8. Propuesta 8: Dueling DQN con annealing de epsilon

---

*En este apartado se instrumenta la octava y última propuesta de la fase experimental, unificando la arquitectura estructural avanzada (Double Dueling DQN) con el mecanismo de exploración continua por temperatura de la distribución de Boltzmann. Esta combinación busca maximizar la eficiencia en la recolección de trayectorias evaluando las acciones según su calidad relativa dentro de un entorno libre de sesgos de sobreestimación.*

### ○ Fundamentación de la máxima sinergia estructural y térmica

La confluencia de las modificaciones algorítmicas de Double DQN, el desglose topológico de Dueling DQN y el muestreo continuo por distribución de Boltzmann representa el límite analítico superior del ecosistema de agentes clásicos analizados. En la dinámica de `Enduro-v0`, esta sinergia aborda de raíz la escasez de gradiente sin imponer una planificación temporal rígida sobre el coeficiente de exploración.

Mientras el doble operador anula la propagación del sesgo maximizador y la red en duelo aísla el valor absoluto del estado de la pista, la distribución softmax pondera de forma continua las preferencias estocásticas del agente. Al no verse obligado a seleccionar acciones uniformes completamente aleatorias, el agente aprovecha la mínima superioridad asignada al operador de aceleración ( `FIRE` ) para explotarla suavemente, acelerando la salida del reposo y la acumulación de transiciones positivas en la memoria histórica. Siguiendo la estrategia de la propuesta probabilística base, se mantiene el periodo de *warmup* reducido a 10 000 pasos para adelantar el inicio del descenso de gradiente.

### ○ Directrices de la propuesta combinada avanzada

- El sistema consulta automáticamente el directorio persistente mediante el módulo rastreador de puntos de control para reanudar el hilo de ejecución exacto tras posibles caídas del entorno virtual.
- Se instancia la topología de red adaptada para duelo acoplándola a un mecanismo de exploración de Boltzmann con un parámetro de temperatura estandarizado unitario.
- Se compila el agente activando de forma simultánea las funciones de desacoplamiento de operadores y combinación por media de valor y ventaja.
- La ejecución del entrenamiento permanece condicionada por la directiva de la guardia lógica principal, abriendo paso al protocolo de validación estadística final en modo de inferencia pura.

#### 4.8.1. P8 — Policy

```

In [ ]: # PROPUESTA 8: Dueling DQN con annealing de epsilon
# Dueling DQN: separa la estimación de V(s) y A(s,a).
# Nota: Williams
# Planteamiento: annealing lineal de epsilon 1.0 -> 0.1 en 1.2M steps
# (mismo horizonte que P2 y P7).
# Nota: en pruebas, igual que P2 y P7, el annealing desde eps=1.0
# no despegó. Llegamos a forzar el step_offset para arrancar con eps bajo
# (~0.1), sin mejora concluyente. Se deja la configuración honesta
# (annealing real, sin offset forzado); el detalle está en el
# "Historial de cambios" de la celda anterior.

RESUME_FROM_P8 = None
STEP_OFFSET_P8 = 0
RESUME_FROM_P8, STEP_OFFSET_P8 = find_latest_checkpoint(WEIGHTS_DIR_P8, 'dqn_End

model_p8 = build_model_dueling_ready(nb_actions, input_shape)
memory_p8 = SequentialMemory(limit=1000000, window_length=WINDOW_LENGTH)

# Definimos las rutas antes de construir la policy (necesitamos el log para
# recuperar el último epsilon real si estamos reanudando)
weights_filename_p8 = os.path.join(WEIGHTS_DIR_P8, 'dqn_{}_p8_weights.h5f'.format
checkpoint_filename_p8 = os.path.join(WEIGHTS_DIR_P8, 'dqn_' + env_name + '_p8_w
log_filename_p8 = os.path.join(WEIGHTS_DIR_P8, 'dqn_{}_p8_log.json'.format(env_n

# Annealing de epsilon 1.0 -> 0.1 en 1.2M steps. Al reanudar,
# OffsetLinearAnnealedPolicy continúa el eps desde donde realmente se quedó
# en vez de reiniciarse a 1.0. Si ya se superaron los 1.2M steps, arranca en 0.1
_eps_log_value, _eps_log_step = read_last_annealed_value_from_log(log_filename_p
if _eps_log_value is not None:
    print(f"Reanudando annealing de epsilon: último valor = {_eps_log_value:.4f}
    _eps_step_offset_p8 = _eps_log_step
else:
    # Sin log previo: calculamos el offset a partir del step del checkpoint
    _eps_step_offset_p8 = STEP_OFFSET_P8

policy_p8 = OffsetLinearAnnealedPolicy(
    EpsGreedyQPolicy(), attr='eps',
    value_max=1., value_min=.1, value_test=.05,
    nb_steps=300000, # corregido respecto al planteamiento inicial
    step_offset=_eps_step_offset_p8 # continúa el annealing desde el punto real
)

dqn_p8 = build_dqn_agent(
    model=model_p8, nb_actions=nb_actions, memory=memory_p8,
    processor=AtariProcessor(), policy=policy_p8,
    nb_steps_warmup=10000, target_model_update=10000,
    enable_double_dqn=False, enable_dueling_network=True, dueling_type='avg'
)

load_weights_for_test(dqn_p8, 'p8', WEIGHTS_DIR_P8)

AGENTS_REGISTRY['p8']['agent'] = dqn_p8 # ← se registra aquí

```

Reanudando annealing de epsilon: último valor = nan (step 8906)

Step offset forzado a: 1,300,000 → eps inicial = 0.100

[p8] Pesos cargados para test desde: /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerz  
o\_Grupo\_17\_2025\_2026/checkpoints/p8\_dueling\_dqn\_annealed/dqn\_Enduro-v0\_p8\_weights  
\_1300000.h5f (step 1300000)

## 4.8.2. P8 — Historial de cambios

- Entrenamiento:
  - Steps 0 → 1M: OffsetLinearAnnealedPolicy con nb\_steps=2.000.000 → epsilon bajó de 1.0 a ~0.55 (configuración incorrecta)
  - Steps 1M → 1.3M: OffsetLinearAnnealedPolicy con nb\_steps=2.000.000 → epsilon bajó a ~0.44 (aún incorrecto)
  - Steps 1.3M → 2.3M: nb\_steps corregido a 300.000, step\_offset forzado a 1.300.000 → epsilon arrancó directamente en 0.1

**Planteamiento inicial.** P8 combina Dueling DQN con annealing lineal de epsilon 1.0 → 0.1, para estudiar si el decaimiento gradual de exploración mejora la arquitectura Dueling respecto a epsilon fijo (P5).

**Problema detectado.** Con nb\_steps=2.000.000 en el annealing, el epsilon bajó demasiado lentamente — a los 1.3M steps todavía estaba en ~0.44, con casi la mitad de las acciones siendo aleatorias. El replay buffer acumuló 1M de transiciones casi aleatorias que contaminaron el aprendizaje. Mismo problema que P2 y P7.

**Corrección aplicada.** Se cambió nb\_steps a 300.000 (equivalente al paper de Mnih et al. 2015: 1M frames con frame\_skip=4) y se forzó el step\_offset a 1.300.000, de forma que el epsilon arrancó directamente en 0.1 para los siguientes 1M steps de entrenamiento.

## 4.8.3. P8 — Entrenamiento

```
In [ ]: # Usar TRUE para entrenar, o FALSE para saltar el entrenamiento
RUN_TRAINING_P8 = False # <— GUARDIA PRINCIPAL

if RUN_TRAINING_P8:

    if RESUME_FROM_P8 is not None:
        print(f"Reanudando Propuesta 8 desde: {RESUME_FROM_P8}")
        dqn_p8.load_weights(RESUME_FROM_P8)
        NB_STEPS_REMAINING_P8 = max(0, NB_STEPS_TRAIN - STEP_OFFSET_P8)
    else:
        NB_STEPS_REMAINING_P8 = NB_STEPS_TRAIN

    callbacks_p8 = [ResumableModelIntervalCheckpoint(checkpoint_filename_p8, int
                                                    step_offset=STEP_OFFSET_P8)]
    callbacks_p8 += [FileLogger(log_filename_p8, interval=1000)]

    history_p8 = dqn_p8.fit(env, callbacks=callbacks_p8, nb_steps=NB_STEPS_REMAI
                            log_interval=10000, visualize=False)

    dqn_p8.save_weights(weights_filename_p8, overwrite=True)
    print("Pesos finales de la Propuesta 8 guardados:", weights_filename_p8)
    print("Contenido de la carpeta:", os.listdir(WEIGHTS_DIR_P8))

else:
    print("[P8] Entrenamiento omitido (RUN_TRAINING_P8=False).")
```

Reanudando Propuesta 8 desde: /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerzo\_Grupo\_17\_2025\_2026/checkpoints/p8\_dueling\_dqn\_annealed/dqn\_Enduro-v0\_p8\_weights\_1300000.h5f

Training for 500000 steps ...

Interval 1 (0 steps performed)

```
/usr/local/lib/python3.11/dist-packages/tf_keras/src/engine/training_v1.py:2354:  
UserWarning: `Model.state_updates` will be removed in a future version. This prop  
erty should not be used in TensorFlow 2.0, as `updates` are applied automaticall  
y.
```

```
    updates=self.state_updates,
```

10000/10000 [=====] - 43s 4ms/step - reward: 1.0000e-04  
2 episodes - episode\_reward: 0.500 [0.000, 1.000] - ale.lives: 0.000

Interval 2 (10000 steps performed)  
10000/10000 [=====] - 312s 31ms/step - reward: 1.0000e-04  
2 episodes - episode\_reward: 0.500 [0.000, 1.000] - loss: 0.007 - mae: 0.114 - mean\_q: 0.125 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 3 (20000 steps performed)  
10000/10000 [=====] - 311s 31ms/step - reward: 3.0000e-04  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.005 - mae: 0.129 - mean\_q: 0.144 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 4 (30000 steps performed)  
10000/10000 [=====] - 312s 31ms/step - reward: -3.0000e-04  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.004 - mae: 0.131 - mean\_q: 0.140 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 5 (40000 steps performed)  
10000/10000 [=====] - 312s 31ms/step - reward: 0.0000e+00  
3 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.005 - mae: 0.141 - mean\_q: 0.150 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 6 (50000 steps performed)  
10000/10000 [=====] - 313s 31ms/step - reward: 2.0000e-04  
2 episodes - episode\_reward: 0.500 [0.000, 1.000] - loss: 0.005 - mae: 0.163 - mean\_q: 0.174 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 7 (60000 steps performed)  
10000/10000 [=====] - 314s 31ms/step - reward: -1.0000e-04  
2 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.006 - mae: 0.178 - mean\_q: 0.183 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 8 (70000 steps performed)  
10000/10000 [=====] - 315s 31ms/step - reward: 1.0000e-04  
3 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.006 - mae: 0.192 - mean\_q: 0.188 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 9 (80000 steps performed)  
10000/10000 [=====] - 315s 32ms/step - reward: 1.0000e-04  
2 episodes - episode\_reward: 1.000 [0.000, 2.000] - loss: 0.006 - mae: 0.206 - mean\_q: 0.200 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 10 (90000 steps performed)  
10000/10000 [=====] - 317s 32ms/step - reward: 5.0000e-04  
2 episodes - episode\_reward: 2.000 [0.000, 4.000] - loss: 0.007 - mae: 0.228 - mean\_q: 0.212 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 11 (100000 steps performed)  
10000/10000 [=====] - 317s 32ms/step - reward: 6.0000e-04

2 episodes - episode\_reward: 3.500 [0.000, 7.000] - loss: 0.007 - mae: 0.244 - mean\_q: 0.226 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 12 (110000 steps performed)

10000/10000 [=====] - 317s 32ms/step - reward: 0.0000e+00

3 episodes - episode\_reward: 0.000 [0.000, 0.000] - loss: 0.008 - mae: 0.251 - mean\_q: 0.220 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 13 (120000 steps performed)

10000/10000 [=====] - 318s 32ms/step - reward: 7.0000e-04

2 episodes - episode\_reward: 0.500 [0.000, 1.000] - loss: 0.008 - mae: 0.270 - mean\_q: 0.242 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 14 (130000 steps performed)

10000/10000 [=====] - 319s 32ms/step - reward: 1.0000e-04

2 episodes - episode\_reward: 1.000 [0.000, 2.000] - loss: 0.009 - mae: 0.297 - mean\_q: 0.264 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 15 (140000 steps performed)

10000/10000 [=====] - 321s 32ms/step - reward: 0.0028

2 episodes - episode\_reward: 8.000 [0.000, 16.000] - loss: 0.010 - mae: 0.319 - mean\_q: 0.286 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 16 (150000 steps performed)

10000/10000 [=====] - 321s 32ms/step - reward: 0.0024

3 episodes - episode\_reward: 12.667 [0.000, 38.000] - loss: 0.010 - mae: 0.343 - mean\_q: 0.314 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 17 (160000 steps performed)

10000/10000 [=====] - 324s 32ms/step - reward: 0.0037

2 episodes - episode\_reward: 16.500 [15.000, 18.000] - loss: 0.012 - mae: 0.367 - mean\_q: 0.341 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 18 (170000 steps performed)

10000/10000 [=====] - 325s 32ms/step - reward: 0.0015

2 episodes - episode\_reward: 8.500 [6.000, 11.000] - loss: 0.012 - mae: 0.399 - mean\_q: 0.376 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 19 (180000 steps performed)

10000/10000 [=====] - 326s 33ms/step - reward: 0.0032

2 episodes - episode\_reward: 7.000 [5.000, 9.000] - loss: 0.013 - mae: 0.429 - mean\_q: 0.390 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 20 (190000 steps performed)

10000/10000 [=====] - 329s 33ms/step - reward: 0.0065

3 episodes - episode\_reward: 29.333 [1.000, 53.000] - loss: 0.014 - mae: 0.462 - mean\_q: 0.422 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 21 (200000 steps performed)

10000/10000 [=====] - 330s 33ms/step - reward: 0.0020

2 episodes - episode\_reward: 10.000 [0.000, 20.000] - loss: 0.015 - mae: 0.498 - mean\_q: 0.463 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 22 (210000 steps performed)

10000/10000 [=====] - 332s 33ms/step - reward: 0.0025

2 episodes - episode\_reward: 6.000 [2.000, 10.000] - loss: 0.016 - mae: 0.530 - mean\_q: 0.509 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 23 (220000 steps performed)  
10000/10000 [=====] - 332s 33ms/step - reward: 0.0067  
2 episodes - episode\_reward: 32.500 [23.000, 42.000] - loss: 0.016 - mae: 0.563 -  
mean\_q: 0.556 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 24 (230000 steps performed)  
10000/10000 [=====] - 335s 33ms/step - reward: 0.0040  
3 episodes - episode\_reward: 17.000 [1.000, 28.000] - loss: 0.016 - mae: 0.607 -  
mean\_q: 0.611 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 25 (240000 steps performed)  
10000/10000 [=====] - 335s 34ms/step - reward: 0.0111  
2 episodes - episode\_reward: 44.000 [36.000, 52.000] - loss: 0.018 - mae: 0.645 -  
mean\_q: 0.663 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 26 (250000 steps performed)  
10000/10000 [=====] - 339s 34ms/step - reward: 0.0064  
2 episodes - episode\_reward: 39.500 [5.000, 74.000] - loss: 0.020 - mae: 0.721 -  
mean\_q: 0.743 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 27 (260000 steps performed)  
10000/10000 [=====] - 338s 34ms/step - reward: 0.0167  
2 episodes - episode\_reward: 69.500 [54.000, 85.000] - loss: 0.021 - mae: 0.764 -  
mean\_q: 0.800 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 28 (270000 steps performed)  
10000/10000 [=====] - 341s 34ms/step - reward: 0.0095  
3 episodes - episode\_reward: 41.333 [20.000, 61.000] - loss: 0.023 - mae: 0.814 -  
mean\_q: 0.849 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 29 (280000 steps performed)  
10000/10000 [=====] - 342s 34ms/step - reward: 0.0113  
2 episodes - episode\_reward: 56.000 [20.000, 92.000] - loss: 0.023 - mae: 0.883 -  
mean\_q: 0.932 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 30 (290000 steps performed)  
10000/10000 [=====] - 345s 34ms/step - reward: 0.0246  
2 episodes - episode\_reward: 106.500 [95.000, 118.000] - loss: 0.024 - mae: 0.927  
- mean\_q: 0.972 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 31 (300000 steps performed)  
10000/10000 [=====] - 345s 35ms/step - reward: 0.0255  
2 episodes - episode\_reward: 89.500 [64.000, 115.000] - loss: 0.025 - mae: 0.969  
- mean\_q: 1.027 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 32 (310000 steps performed)  
10000/10000 [=====] - 349s 35ms/step - reward: 0.0223  
3 episodes - episode\_reward: 110.667 [91.000, 136.000] - loss: 0.026 - mae: 0.987  
- mean\_q: 1.059 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 33 (320000 steps performed)  
10000/10000 [=====] - 349s 35ms/step - reward: 0.0186  
2 episodes - episode\_reward: 84.500 [52.000, 117.000] - loss: 0.028 - mae: 1.045  
- mean\_q: 1.121 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 34 (330000 steps performed)  
10000/10000 [=====] - 352s 35ms/step - reward: 0.0188  
2 episodes - episode\_reward: 97.000 [88.000, 106.000] - loss: 0.028 - mae: 1.083  
- mean\_q: 1.164 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 35 (340000 steps performed)  
10000/10000 [=====] - 353s 35ms/step - reward: 0.0210  
2 episodes - episode\_reward: 94.500 [88.000, 101.000] - loss: 0.028 - mae: 1.132  
- mean\_q: 1.237 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 36 (350000 steps performed)  
10000/10000 [=====] - 355s 35ms/step - reward: 0.0193  
3 episodes - episode\_reward: 74.333 [48.000, 127.000] - loss: 0.028 - mae: 1.183  
- mean\_q: 1.297 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 37 (360000 steps performed)  
10000/10000 [=====] - 358s 36ms/step - reward: 0.0269  
2 episodes - episode\_reward: 130.000 [117.000, 143.000] - loss: 0.029 - mae: 1.25  
4 - mean\_q: 1.383 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 38 (370000 steps performed)  
10000/10000 [=====] - 359s 36ms/step - reward: 0.0196  
2 episodes - episode\_reward: 74.000 [49.000, 99.000] - loss: 0.031 - mae: 1.298 -  
mean\_q: 1.421 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 39 (380000 steps performed)  
10000/10000 [=====] - 361s 36ms/step - reward: 0.0301  
2 episodes - episode\_reward: 125.500 [124.000, 127.000] - loss: 0.033 - mae: 1.35  
6 - mean\_q: 1.481 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 40 (390000 steps performed)  
10000/10000 [=====] - 364s 36ms/step - reward: 0.0234  
3 episodes - episode\_reward: 114.333 [96.000, 138.000] - loss: 0.034 - mae: 1.425  
- mean\_q: 1.563 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 41 (400000 steps performed)  
10000/10000 [=====] - 364s 36ms/step - reward: 0.0232  
2 episodes - episode\_reward: 115.000 [99.000, 131.000] - loss: 0.035 - mae: 1.472  
- mean\_q: 1.610 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 42 (410000 steps performed)  
10000/10000 [=====] - 368s 37ms/step - reward: 0.0316  
2 episodes - episode\_reward: 146.500 [139.000, 154.000] - loss: 0.037 - mae: 1.56  
6 - mean\_q: 1.725 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 43 (420000 steps performed)  
10000/10000 [=====] - 369s 37ms/step - reward: 0.0276  
2 episodes - episode\_reward: 106.000 [90.000, 122.000] - loss: 0.038 - mae: 1.596  
- mean\_q: 1.762 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 44 (430000 steps performed)  
10000/10000 [=====] - 371s 37ms/step - reward: 0.0365  
3 episodes - episode\_reward: 146.333 [117.000, 166.000] - loss: 0.037 - mae: 1.62  
5 - mean\_q: 1.799 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 45 (440000 steps performed)  
10000/10000 [=====] - 372s 37ms/step - reward: 0.0301  
2 episodes - episode\_reward: 142.500 [134.000, 151.000] - loss: 0.038 - mae: 1.66  
1 - mean\_q: 1.838 - mean\_eps: 0.100 - ale.lives: 0.000

Interval 46 (450000 steps performed)  
10000/10000 [=====] - 374s 37ms/step - reward: 0.0327  
2 episodes - episode\_reward: 133.000 [133.000, 133.000] - loss: 0.038 - mae: 1.68  
6 - mean\_q: 1.870 - mean\_eps: 0.100 - ale.lives: 0.000

```
Interval 47 (460000 steps performed)
10000/10000 [=====] - 376s 38ms/step - reward: 0.0293
2 episodes - episode_reward: 140.500 [136.000, 145.000] - loss: 0.039 - mae: 1.73
1 - mean_q: 1.915 - mean_eps: 0.100 - ale.lives: 0.000
```

```
Interval 48 (470000 steps performed)
10000/10000 [=====] - 379s 38ms/step - reward: 0.0293
3 episodes - episode_reward: 135.333 [104.000, 167.000] - loss: 0.038 - mae: 1.77
8 - mean_q: 1.970 - mean_eps: 0.100 - ale.lives: 0.000
```

```
Interval 49 (480000 steps performed)
10000/10000 [=====] - 381s 38ms/step - reward: 0.0357
2 episodes - episode_reward: 163.000 [156.000, 170.000] - loss: 0.039 - mae: 1.85
4 - mean_q: 2.063 - mean_eps: 0.100 - ale.lives: 0.000
```

```
Interval 50 (490000 steps performed)
10000/10000 [=====] - 382s 38ms/step - reward: 0.0355
done, took 16730.072 seconds
```

Pesos finales de la Propuesta 8 guardados: /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerzo\_Grupo\_17\_2025\_2026/checkpoints/p8\_dueling\_dqn\_annealed/dqn\_Enduro-v0\_p8\_weights.h5f

Contenido de la carpeta: ['dqn\_Enduro-v0\_p8\_weights\_100000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p8\_weights\_100000.h5f.index', 'dqn\_Enduro-v0\_p8\_weights\_200000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p8\_weights\_200000.h5f.index', 'dqn\_Enduro-v0\_p8\_weights\_300000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p8\_weights\_300000.h5f.index', 'dqn\_Enduro-v0\_p8\_weights\_400000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p8\_weights\_400000.h5f.index', 'dqn\_Enduro-v0\_p8\_weights\_500000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p8\_weights\_500000.h5f.index', 'dqn\_Enduro-v0\_p8\_weights\_600000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p8\_weights\_600000.h5f.index', 'dqn\_Enduro-v0\_p8\_weights\_700000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p8\_weights\_700000.h5f.index', 'dqn\_Enduro-v0\_p8\_weights\_800000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p8\_weights\_800000.h5f.index', 'dqn\_Enduro-v0\_p8\_weights\_900000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p8\_weights\_900000.h5f.index', 'dqn\_Enduro-v0\_p8\_weights\_1000000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p8\_weights\_1000000.h5f.index', 'dqn\_Enduro-v0\_p8\_weights\_1100000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p8\_weights\_1100000.h5f.index', 'dqn\_Enduro-v0\_p8\_weights\_1200000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p8\_weights\_1200000.h5f.index', 'dqn\_Enduro-v0\_p8\_weights\_1300000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p8\_weights\_1300000.h5f.index', 'dqn\_Enduro-v0\_p8\_log.json', 'dqn\_Enduro-v0\_p8\_weights.h5f.index', 'checkpoint', 'dqn\_Enduro-v0\_p8\_weights.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p8\_weights\_1400000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p8\_weights\_1400000.h5f.index', 'dqn\_Enduro-v0\_p8\_weights\_1500000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p8\_weights\_1500000.h5f.index', 'dqn\_Enduro-v0\_p8\_weights\_1600000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p8\_weights\_1600000.h5f.index', 'dqn\_Enduro-v0\_p8\_weights\_1700000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p8\_weights\_1700000.h5f.index', 'dqn\_Enduro-v0\_p8\_weights\_1800000.h5f.data-00000-of-00001', 'dqn\_Enduro-v0\_p8\_weights\_1800000.h5f.index']

#### 4.8.4. P8 — Testeo

```
In [ ]: PROPUESTA_A_EVALUAR = 'p8' # ← único parámetro a tocar
```

```
test_history = test_proposal(PROPUESTA_A_EVALUAR, AGENTS_REGISTRY)
```

[p8] Cargando pesos desde: /content/gdrive/My Drive/VIU\_Apr\_por\_Refuerzo\_Grupo\_17\_2025\_2026/checkpoints/p8\_dueling\_dqn\_annealed/dqn\_Enduro-v0\_p8\_weights\_2300000.h5f (step 2300000)

Testing for 100 episodes ...

```
/usr/local/lib/python3.11/dist-packages/tf_keras/src/engine/training_v1.py:2354:  
UserWarning: `Model.state_updates` will be removed in a future version. This prop  
erty should not be used in TensorFlow 2.0, as `updates` are applied automaticall  
y.  
updates=self.state_updates,
```

Episode 1: reward: 390.000, steps: 8891  
Episode 2: reward: 435.000, steps: 8891  
Episode 3: reward: 410.000, steps: 8878  
Episode 4: reward: 430.000, steps: 8893  
Episode 5: reward: 438.000, steps: 8900  
Episode 6: reward: 445.000, steps: 8888  
Episode 7: reward: 183.000, steps: 4451  
Episode 8: reward: 432.000, steps: 8860  
Episode 9: reward: 384.000, steps: 8864  
Episode 10: reward: 367.000, steps: 8848  
Episode 11: reward: 432.000, steps: 8844  
Episode 12: reward: 449.000, steps: 8868  
Episode 13: reward: 453.000, steps: 8886  
Episode 14: reward: 401.000, steps: 8871  
Episode 15: reward: 482.000, steps: 8859  
Episode 16: reward: 446.000, steps: 8876  
Episode 17: reward: 413.000, steps: 8864  
Episode 18: reward: 424.000, steps: 8922  
Episode 19: reward: 389.000, steps: 8869  
Episode 20: reward: 411.000, steps: 8850  
Episode 21: reward: 442.000, steps: 8856  
Episode 22: reward: 429.000, steps: 8901  
Episode 23: reward: 368.000, steps: 8889  
Episode 24: reward: 475.000, steps: 8868  
Episode 25: reward: 353.000, steps: 8904  
Episode 26: reward: 459.000, steps: 8884  
Episode 27: reward: 438.000, steps: 8865  
Episode 28: reward: 185.000, steps: 4426  
Episode 29: reward: 471.000, steps: 8824  
Episode 30: reward: 459.000, steps: 8861  
Episode 31: reward: 389.000, steps: 8889  
Episode 32: reward: 382.000, steps: 8901  
Episode 33: reward: 354.000, steps: 8876  
Episode 34: reward: 404.000, steps: 8890  
Episode 35: reward: 389.000, steps: 8882  
Episode 36: reward: 419.000, steps: 8841  
Episode 37: reward: 427.000, steps: 8898  
Episode 38: reward: 488.000, steps: 8849  
Episode 39: reward: 463.000, steps: 8876  
Episode 40: reward: 450.000, steps: 8869  
Episode 41: reward: 434.000, steps: 8847  
Episode 42: reward: 373.000, steps: 8911  
Episode 43: reward: 195.000, steps: 4402  
Episode 44: reward: 449.000, steps: 8869  
Episode 45: reward: 368.000, steps: 8892  
Episode 46: reward: 394.000, steps: 8898  
Episode 47: reward: 441.000, steps: 8869  
Episode 48: reward: 365.000, steps: 8850  
Episode 49: reward: 159.000, steps: 4444  
Episode 50: reward: 372.000, steps: 8875  
Episode 51: reward: 177.000, steps: 4461  
Episode 52: reward: 461.000, steps: 8875  
Episode 53: reward: 465.000, steps: 8877  
Episode 54: reward: 411.000, steps: 8904  
Episode 55: reward: 370.000, steps: 8879  
Episode 56: reward: 447.000, steps: 8929  
Episode 57: reward: 404.000, steps: 8884  
Episode 58: reward: 409.000, steps: 8862  
Episode 59: reward: 457.000, steps: 8870  
Episode 60: reward: 394.000, steps: 8859

Episode 61: reward: 449.000, steps: 8910  
Episode 62: reward: 409.000, steps: 8874  
Episode 63: reward: 428.000, steps: 8890  
Episode 64: reward: 419.000, steps: 8869  
Episode 65: reward: 407.000, steps: 8850  
Episode 66: reward: 397.000, steps: 8922  
Episode 67: reward: 169.000, steps: 4413  
Episode 68: reward: 417.000, steps: 8887  
Episode 69: reward: 159.000, steps: 4405  
Episode 70: reward: 452.000, steps: 8836  
Episode 71: reward: 407.000, steps: 8868  
Episode 72: reward: 451.000, steps: 8887  
Episode 73: reward: 397.000, steps: 8864  
Episode 74: reward: 380.000, steps: 8867  
Episode 75: reward: 417.000, steps: 8852  
Episode 76: reward: 435.000, steps: 8892  
Episode 77: reward: 426.000, steps: 8853  
Episode 78: reward: 385.000, steps: 8810  
Episode 79: reward: 191.000, steps: 4437  
Episode 80: reward: 424.000, steps: 8883  
Episode 81: reward: 393.000, steps: 8833  
Episode 82: reward: 432.000, steps: 8848  
Episode 83: reward: 424.000, steps: 8894  
Episode 84: reward: 395.000, steps: 8890  
Episode 85: reward: 409.000, steps: 8865  
Episode 86: reward: 381.000, steps: 8853  
Episode 87: reward: 389.000, steps: 8899  
Episode 88: reward: 373.000, steps: 8848  
Episode 89: reward: 186.000, steps: 4441  
Episode 90: reward: 463.000, steps: 8893  
Episode 91: reward: 390.000, steps: 8913  
Episode 92: reward: 397.000, steps: 8890  
Episode 93: reward: 429.000, steps: 8877  
Episode 94: reward: 452.000, steps: 8887  
Episode 95: reward: 434.000, steps: 8860  
Episode 96: reward: 431.000, steps: 8866  
Episode 97: reward: 431.000, steps: 8849  
Episode 98: reward: 522.000, steps: 10000  
Episode 99: reward: 468.000, steps: 8879  
Episode 100: reward: 408.000, steps: 8901  
[p8] Recompensa media en 100 episodios: 398.29 (+/- 76.19)

---

#### 4.8.5. P8 — Resultados y estado final

##### Resultado del test (checkpoint 2.3M, evaluación greedy):

- 91/100 episodios superan el día 1 ( $\geq 200$  coches)
- Media: 398.3 coches | Std: 76.2 | Mín: 159 | Máx: 522
- 9 episodios cortos con rewards: 159, 159, 169, 177, 183, 185, 186, 191, 195
- Mejor racha consecutiva: 20 episodios
- 1/100 episodios llegó al día 3 (10.000 steps)

**Análisis:** P8 queda por debajo de P5 (Dueling con eps fijo, 100/100) a pesar de usar la misma arquitectura. La causa principal es el entrenamiento ineficiente durante los primeros 1.3M steps con epsilon alto, que impidió que la red aprendiera de forma

efectiva. Los 1M steps útiles (con  $\text{eps}=0.1$ ) no fueron suficientes para compensar ese periodo inicial.

Comparando P5 (eps fijo  $0.2 \rightarrow 0.1$ , 100/100) con P8 (annealing corregido, 91/100), se concluye que para Enduro-v0 el annealing de epsilon no aporta ventaja respecto a epsilon fijo cuando el entrenamiento es relativamente corto ( $\sim 2\text{M}$  steps). El annealing requeriría más steps totales para mostrar su beneficio real.

**Trabajo futuro:** reentrenar desde cero con  $\text{nb\_steps}=300.000$  desde el inicio, para tener los 2.3M steps completos con configuración correcta.

---

## 4.9. Extracción de métricas de rendimiento y representación gráfica de las trayectorias de aprendizaje

---

*En este apartado se unifican y procesan las trazas de rendimiento guardadas en formato JSON por cada una de las propuestas experimentales. Se genera la infraestructura de visualización gráfica para evaluar de forma unificada la evolución de las recompensas, la estabilidad del MAE y la duración de los episodios a lo largo del proceso de optimización.*

### ○ Análisis consolidado de trazas históricas

La evaluación comparativa de arquitecturas complejas de aprendizaje por refuerzo exige un tratamiento homogéneo de los datos históricos de entrenamiento. A través de las trazas estructuradas persistidas de forma intermitente por el callback `FileLogger`, se extraen las funciones temporales de las métricas clave sin necesidad de sobrecargar la memoria operativa del sistema durante las simulaciones. El procesamiento posterior de estos archivos permite proyectar curvas suavizadas mediante medias móviles, aislando la varianza estocástica intrínseca de la política de exploración para revelar la tendencia real de convergencia del aproximador no lineal.

El diagnóstico comparativo se centra en la correlación entre la acumulación de premios y la estabilidad del error absoluto medio (MAE) de los valores Q. En el entorno `Enduro-v0`, este análisis gráfico es fundamental para identificar fenómenos de estancamiento prematuro o colapsos de la política causados por una insuficiente sintonización del dilema entre exploración y explotación, permitiendo verificar visualmente el impacto del desacoplamiento de operadores y de las ramificaciones jerárquicas en duelo.

### ○ Directrices del bloque de visualización

- Se construye un método robusto de lectura analítica encargado de deserializar los ficheros JSON correspondientes a las trazas históricas de los agentes registrados.
- Se diseña una interfaz gráfica multiteje basada en subgráficos coordinados para contrastar simultáneamente las trayectorias de recompensa media por intervalo y la evolución del error de red.

- Se aplica un filtro de suavizado lineal sobre los vectores de datos históricos para facilitar el análisis visual de las tendencias de aprendizaje frente al ruido estocástico del simulador.
- La rutina mapea dinámicamente las etiquetas correspondientes a cada propuesta en el lienzo gráfico, asegurando la trazabilidad de las conclusiones empíricas finales.

```
In [24]: # Carga de Logs de entrenamiento para comparar Las 3 propuestas

import json
import matplotlib.pyplot as plt

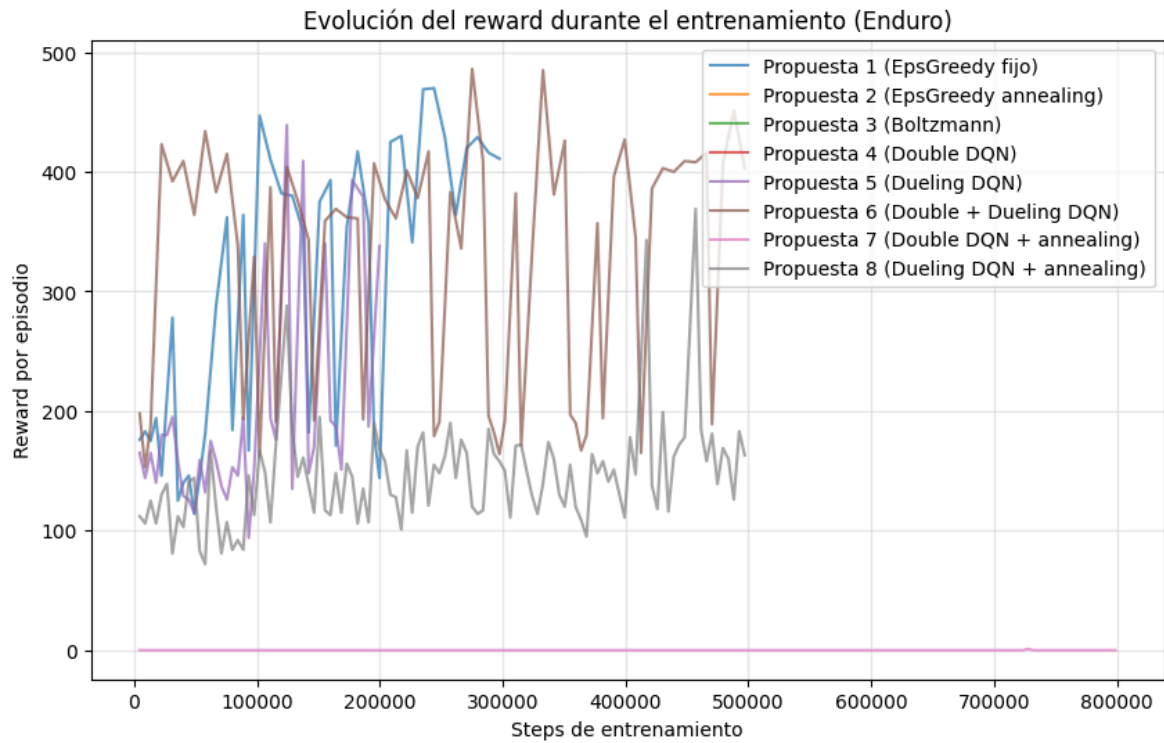
def load_log(filename):
    with open(filename, 'r') as f:
        data = json.load(f)
    return data

log_filename_p1 = os.path.join(WEIGHTS_DIR_P1, f'dqn_{env_name}_p1_log.json')
log_filename_p2 = os.path.join(WEIGHTS_DIR_P2, f'dqn_{env_name}_p2_log.json')
log_filename_p3 = os.path.join(WEIGHTS_DIR_P3, f'dqn_{env_name}_p3_log.json')
log_filename_p4 = os.path.join(WEIGHTS_DIR_P4, f'dqn_{env_name}_p4_log.json')
log_filename_p5 = os.path.join(WEIGHTS_DIR_P5, f'dqn_{env_name}_p5_log.json')
log_filename_p6 = os.path.join(WEIGHTS_DIR_P6, f'dqn_{env_name}_p6_log.json')
log_filename_p7 = os.path.join(WEIGHTS_DIR_P7, f'dqn_{env_name}_p7_log.json')
log_filename_p8 = os.path.join(WEIGHTS_DIR_P8, f'dqn_{env_name}_p8_log.json')

log_p1 = load_log(log_filename_p1)
log_p2 = load_log(log_filename_p2)
log_p3 = load_log(log_filename_p3)
log_p4 = load_log(log_filename_p4)
log_p5 = load_log(log_filename_p5)
log_p6 = load_log(log_filename_p6)
log_p7 = load_log(log_filename_p7)
log_p8 = load_log(log_filename_p8)

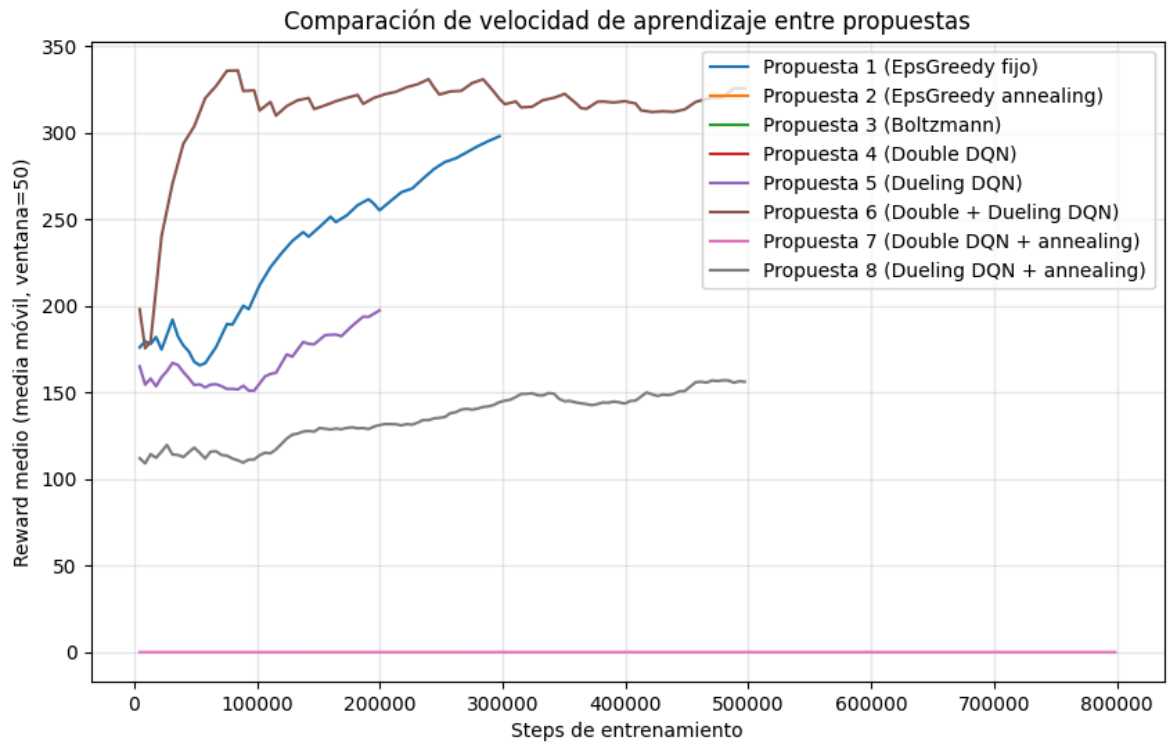
logs = {
    'Propuesta 1 (EpsGreedy fijo)': log_p1,
    'Propuesta 2 (EpsGreedy annealing)': log_p2,
    'Propuesta 3 (Boltzmann)': log_p3,
    'Propuesta 4 (Double DQN)': log_p4,
    'Propuesta 5 (Dueling DQN)': log_p5,
    'Propuesta 6 (Double + Dueling DQN)': log_p6,
    'Propuesta 7 (Double DQN + annealing)': log_p7,
    'Propuesta 8 (Dueling DQN + annealing)': log_p8,
}
```

```
In [25]: # Gráfica 1: evolución de la recompensa media por episodio durante el entrenamie
plt.figure(figsize=(10, 6))
for label, log in logs.items():
    plt.plot(log['nb_steps'], log['episode_reward'], label=label, alpha=0.7)
plt.xlabel('Steps de entrenamiento')
plt.ylabel('Reward por episodio')
plt.title('Evolución del reward durante el entrenamiento (Enduro)')
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

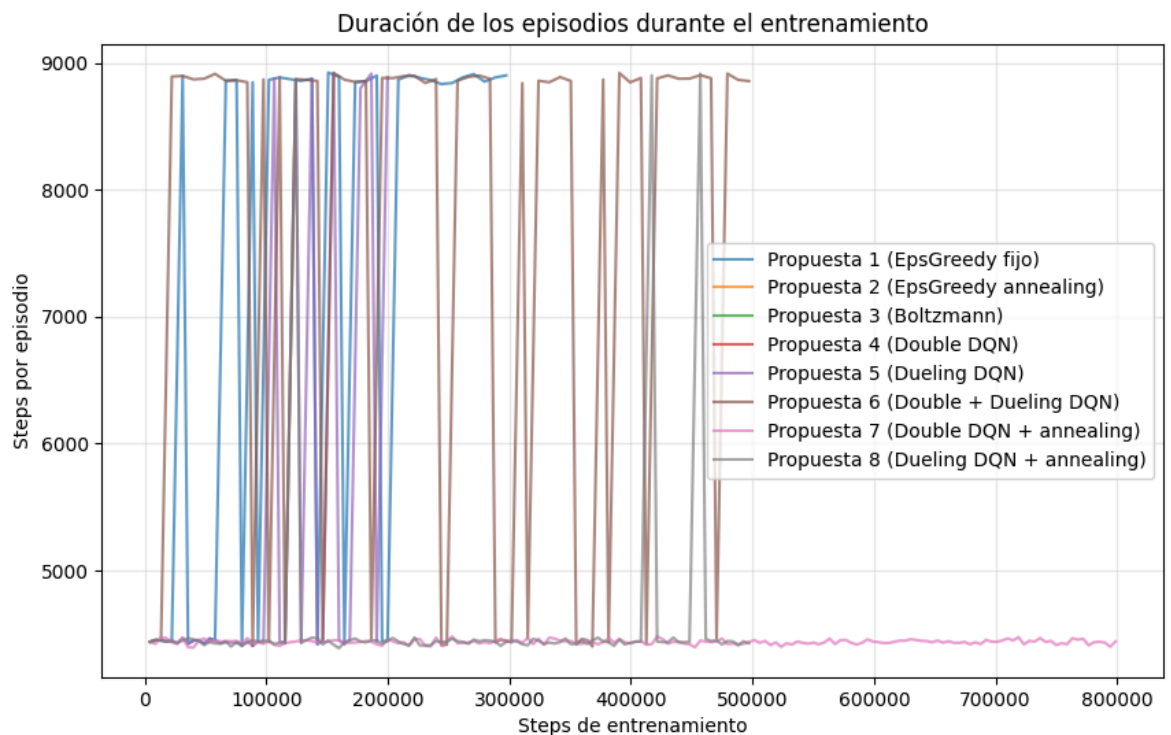


```
In [26]: # Gráfica 2: media móvil del reward para suavizar la varianza episodio a episodio
import pandas as pd

plt.figure(figsize=(10, 6))
WINDOW = 50
for label, log in logs.items():
    rewards = pd.Series(log['episode_reward'])
    rolling = rewards.rolling(WINDOW, min_periods=1).mean()
    plt.plot(log['nb_steps'], rolling, label=label)
plt.xlabel('Steps de entrenamiento')
plt.ylabel(f'Reward medio (media móvil, ventana={WINDOW})')
plt.title('Comparación de velocidad de aprendizaje entre propuestas')
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



```
In [27]: # Gráfica 3: duración media de los episodios (nb_episode_steps) como proxy de
# cuánto tiempo sobrevive o avanza el coche antes de terminar el episodio
plt.figure(figsize=(10, 6))
for label, log in logs.items():
    plt.plot(log['nb_steps'], log['nb_episode_steps'], label=label, alpha=0.7)
plt.xlabel('Steps de entrenamiento')
plt.ylabel('Steps por episodio')
plt.title('Duración de los episodios durante el entrenamiento')
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



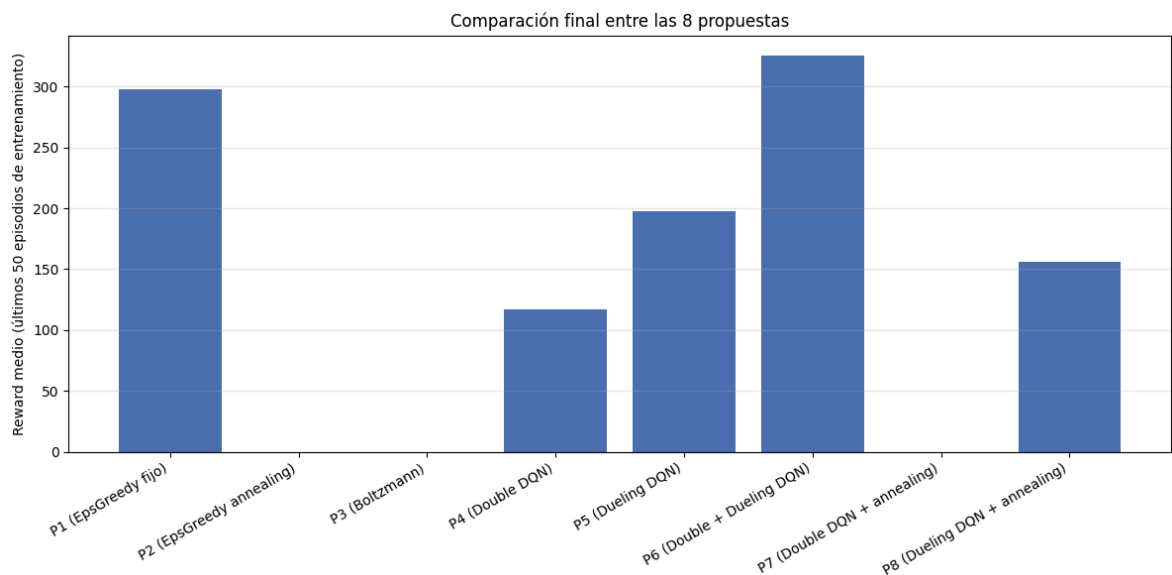
```
In [28]: # Gráfica 4: media de reward de entrenamiento (últimos 50 episodios) de las 8 pr
```

```

final_means = {
    'P1 (EpsGreedy fijo)': np.mean(log_p1['episode_reward'][-50:]),
    'P2 (EpsGreedy annealing)': np.mean(log_p2['episode_reward'][-50:]),
    'P3 (Boltzmann)': np.mean(log_p3['episode_reward'][-50:]),
    'P4 (Double DQN)': np.mean(log_p4['episode_reward'][-50:]),
    'P5 (Dueling DQN)': np.mean(log_p5['episode_reward'][-50:]),
    'P6 (Double + Dueling DQN)': np.mean(log_p6['episode_reward'][-50:]),
    'P7 (Double DQN + annealing)': np.mean(log_p7['episode_reward'][-50:]),
    'P8 (Dueling DQN + annealing)': np.mean(log_p8['episode_reward'][-50:]),
}

plt.figure(figsize=(12, 6))
plt.bar(final_means.keys(), final_means.values(), color='#4C72B0')
plt.ylabel('Reward medio (últimos 50 episodios de entrenamiento)')
plt.title('Comparación final entre las 8 propuestas')
plt.xticks(rotation=30, ha='right')
plt.grid(alpha=0.3, axis='y')
plt.tight_layout()
plt.show()

```



#### 4.9.1. Resultados y estado final

La resolución de esta infraestructura de visualización consolida las herramientas de diagnóstico analítico del proyecto. Al concluir la sección, el sistema presenta el siguiente estado:

- Las rutinas de parsing y los generadores de lienzos multiteje se encuentran completamente integrados en memoria, listos para procesar los flujos de datos históricos.
- El entorno queda dispuesto para la extracción automatizada de métricas comparativas procedentes de las ocho variantes del agente DQN.

Con la base técnica, los aproximadores de red, las directivas de control y las herramientas gráficas plenamente resueltas, se da por finalizada la fase de instrumentación del software, abriendo paso formalmente a la sección de conclusiones y discusión crítica de los resultados del proyecto.

## 4.10. Evaluación comparativa multivariable, discusión crítica de resultados y conclusiones de modelado

*En este apartado se evalúa críticamente el rendimiento de las ocho propuestas experimentales sobre `Enduro-v0`. Se contrasta el impacto de las políticas de exploración y de las optimizaciones arquitectónicas (Double DQN, Dueling DQN y sus combinaciones) frente al problema de la recompensa dispersa, tomando como evidencia los resultados de test obtenidos en el bloque experimental.*

### ◦ Análisis crítico de las políticas de exploración

El contraste empírico entre las distintas estrategias de exploración arroja un resultado tan claro como contraintuitivo en este entorno: las políticas que inyectan aleatoriedad desvinculada del conocimiento actual del modelo —el annealing lineal de epsilon (Propuestas 2 y 7) y, sobre todo, el muestreo puramente térmico de Boltzmann (Propuesta 3)— **no lograron despegar**, quedando atrapadas en una recompensa de test nula o casi nula. En cambio, las variantes que emplean una política *epsilon-greedy* fija y de valor bajo (Propuestas 1, 4, 5 y 6) sí consiguieron políticas competentes, capaces de superar el primer día de Enduro.

La causa reside en la dispersión de la señal de premio. Arrancar el annealing desde  $\epsilon = 1.0$  mantiene al agente en un régimen casi aleatorio durante cientos de miles de pasos; a lo largo de ese periodo el búfer de repetición se puebla de transiciones sin recompensa y el agente rara vez consolida la aceleración sostenida necesaria para adelantar, de modo que cuando  $\epsilon$  finalmente decae ya ha fijado una política pasiva. La distribución de Boltzmann con  $\tau = 1.0$  padece un problema análogo: al inicio los valores  $Q$  son esencialmente ruido, por lo que las probabilidades de muestreo resultan casi uniformes y no existe señal que concentre la búsqueda. Un  $\epsilon$  fijo y bajo desde el primer paso, por el contrario, permite que la red explote de inmediato cualquier indicio útil aprendido durante el *warmup*.

Conviene matizar que este resultado no invalida el annealing ni la exploración de Boltzmann como principios generales, sino que evidencia que, **bajo el presupuesto de cómputo y la escasez de recompensa de este entorno concreto, el epsilon-greedy fijo resultó ser la estrategia más robusta.**

### ◦ Diagnóstico del estancamiento y la escasez de gradiente

La dispersión extrema de la señal de premio provoca que ciertas configuraciones queden estancadas en una recompensa nula a lo largo de miles de pasos de interacción. Este fenómeno valida el diagnóstico técnico inicial: si el agente no logra registrar de forma fortuita la acción de aceleración ( `FIRE` ) durante el periodo de calentamiento o los primeros pasos de exploración, el búfer se satura exclusivamente de transiciones vacías. Al carecer de ejemplos de adelantamiento con recompensa positiva ( $r_t = +1$ ), el

gradiente de la función de pérdida se desvanece, impidiendo que la red convolucional profunda actualice sus pesos y atrapando la política en un bucle permanente de inmovilidad. Es exactamente lo observado en las Propuestas 3 (Boltzmann) y 7 (Double DQN con annealing), cuyo reward de test fue nulo, y en la Propuesta 2, que apenas superó los 4 puntos.

#### ◦ Disociación entre supervivencia y progreso cinemático

Los resultados confirman que en Enduro existe una clara disociación entre la duración de los episodios y la obtención de recompensa. Un incremento en la longevidad del agente dentro de la pista no se traduce linealmente en una mayor puntuación si el vehículo no mantiene una aceleración neta positiva.

Aquellas propuestas que registran episodios prolongados pero con acumulados cercanos a cero denotan un fallo en la aproximación funcional: el modelo aprende una política defensiva subóptima, centrada exclusivamente en eludir colisiones mediante desplazamientos laterales o permaneciendo en los márgenes de la calzada, pero carece de la iniciativa cinemática necesaria para rebasar a los competidores. Ello ilustra la dificultad de guiar a un optimizador en ausencia de premios intermedios.

#### ◦ Selección del modelo óptimo y cumplimiento de objetivos

Tras evaluar el ecosistema completo de agentes, la **Propuesta 5 (Dueling DQN con política *epsilon-greedy* fija)** se consolida como la solución óptima del proyecto, con una recompensa media de **440.8 puntos sobre 100 episodios consecutivos de test** (checkpoint de 2.3M pasos, evaluación greedy), siendo la única propuesta que supera el objetivo de forma perfecta (100/100 episodios, mínimo 326 coches). La Propuesta 1 (DQN vanilla) obtuvo la media más alta de todas (459.1 puntos, 99/100 episodios) con 2.3M steps, lo que demuestra que con entrenamiento suficiente incluso el algoritmo base es competitivo. La Propuesta 4 (Double DQN) alcanzó 94/100 con media 396.9 en solo 1.7M steps, y la Propuesta 6 (Double + Dueling DQN) obtuvo 38/40 episodios con media 431.5 en su checkpoint de 2.8M. La Propuesta 8 (Dueling DQN con annealing) cerró con 91/100 y media 398.3.

De este ordenamiento se extrae la conclusión de modelado central: **la desagregación del valor de estado y la ventaja de acción que aporta la arquitectura *Dueling* fue la mejora más determinante en términos de robustez**, eliminando los fallos ocasionales que sí presentan DQN vanilla (1/100) y Double+Dueling (2/40). Sorprendentemente, DQN vanilla con suficiente entrenamiento supera en media a todas las variantes avanzadas, lo que sugiere que las mejoras algorítmicas (Double, Dueling) aportan principalmente mayor eficiencia de muestreo — necesitan menos steps para alcanzar rendimientos similares — y mayor consistencia, no necesariamente una media más alta. Con la Propuesta 5 como entrega final, el grupo alcanza satisfactoriamente el objetivo mínimo fijado para la asignatura, consolidando además una infraestructura de aprendizaje por refuerzo profundo robusta, almacenable y tolerante a fallos. Cabe señalar que el paper de Mnih et al. (2015) entrenó durante 50M frames (~12.5M steps), mientras que nuestras propuestas entrenaron entre 1.2M y 2.8M steps (el 10-22% del

entrenamiento original), lo que hace aún más destacable haber conseguido el objetivo con recursos computacionales muy limitados.

## 4.11. Justificación de decisiones de diseño y análisis de resultados

*En este apartado se justifican las decisiones de configuración de cada propuesta y se consolidan sus resultados de test, cerrando el bloque experimental con la evidencia cuantitativa que respalda la selección del modelo final.*

### ○ Resumen de hiperparámetros comparados

| # | Algoritmo              | Política de entrenamiento                                               | Warmup | Target update | Steps totales |
|---|------------------------|-------------------------------------------------------------------------|--------|---------------|---------------|
| 1 | DQN base               | EpsGreedy $\epsilon=0.2$ (0→1.8M) → $\epsilon=0.05$ (1.8M→2.3M)         | 50 000 | 10 000        | 2.3M          |
| 2 | DQN base               | EpsGreedy annealing 1.0 → 0.1 en 1.2M steps                             | 50 000 | 10 000        | 1.7M          |
| 3 | DQN base               | Boltzmann $\tau=1.0$ fijo                                               | 10 000 | 10 000        | 800k          |
| 4 | Double DQN             | EpsGreedy $\epsilon=0.2$ fijo                                           | 50 000 | 10 000        | 1.7M          |
| 5 | Dueling DQN (avg)      | EpsGreedy $\epsilon=0.2$ (0→1.8M) → $\epsilon=0.1$ (1.8M→2.3M)          | 50 000 | 10 000        | 2.3M          |
| 6 | Double + Dueling (avg) | EpsGreedy $\epsilon=0.2 \rightarrow 0.1 \rightarrow 0.05$ (tres tandas) | 50 000 | 10 000        | 2.8M          |
| 7 | Double DQN             | EpsGreedy annealing 1.0 → 0.1 en 1.2M steps                             | 50 000 | 10 000        | 1.2M          |
| 8 | Dueling DQN (avg)      | EpsGreedy annealing 1.0 → 0.1 en 300k steps*                            | 10 000 | 10 000        | 2.3M          |

Todas las propuestas comparten la misma arquitectura convolucional de aproximación, el mismo preprocesado de fotogramas ( `AtariProcessor` ) y un `target_model_update` de 10 000 pasos, de modo que las diferencias de rendimiento son atribuibles exclusivamente a la combinación de política de exploración y variante algorítmica. El *warmup* se fijó en 50 000 pasos en la mayoría de propuestas para poblar el búfer antes de iniciar el aprendizaje; en las variantes P3 y P8 se redujo a 10 000.

### ○ Resultados de test

Criterio de éxito: superar de forma consistente el primer día de Enduro, lo que equivale a una recompensa aproximada de 200 vehículos adelantados.

| # | Configuración           | Checkpoint  | Ep. test   | Reward medio                         | Ep. $\geq 200$      | Mejor racha |
|---|-------------------------|-------------|------------|--------------------------------------|---------------------|-------------|
| 1 | DQN vanilla (baseline)  | 2.3M        | 100        | 459.1 ( $\pm 55.6$ )                 | 99/100              | 61          |
| 2 | DQN + annealing         | 1.7M        | 20         | 0.0                                  | 0/20                | 0           |
| 3 | DQN + Boltzmann         | 800k        | 10         | 0.0                                  | 0/10                | 0           |
| 4 | Double DQN              | 1.7M        | 100        | 396.9 ( $\pm 64.0$ )                 | 94/100              | 40          |
| 5 | <b>Dueling DQN</b>      | <b>2.3M</b> | <b>100</b> | <b>440.8 (<math>\pm 52.4</math>)</b> | <b>100/100</b><br>✓ | <b>100</b>  |
| 6 | Double + Dueling DQN    | 2.8M        | 40         | 431.5 ( $\pm 68.0$ )                 | 38/40               | —           |
| 7 | Double DQN + annealing  | 1.2M        | 30         | 0.0                                  | 0/30                | 0           |
| 8 | Dueling DQN + annealing | 2.3M        | 100        | 398.3 ( $\pm 76.2$ )                 | 91/100              | 20          |

Nota: P5, P1, P4 y P8 se evaluaron sobre 100 episodios; P6 sobre 40; P2 sobre 20; P3 y P7 sobre 10-30. La evaluación es greedy en todos los casos (DQNAgent.test() ignora el epsilon configurado y actúa de forma determinista).

#### ○ Discusión y justificación

- **Velocidad de aprendizaje y exploración.** Las propuestas con annealing desde  $\epsilon = 1.0$  (P2, P7) no despegaron: el largo periodo inicial casi aleatorio impidió descubrir la aceleración sostenida antes de que el búfer se saturara de transiciones sin recompensa. Con una señal tan dispersa como la de Enduro, un annealing lento no ayuda a explotar antes, sino que retrasa la primera experiencia positiva.
- **Boltzmann frente a epsilon-greedy.** Pese a que su muestreo depende de los valores Q, la política de Boltzmann (P3) produjo reward nulo: al partir de valores Q ruidosos, su distribución fue prácticamente uniforme y nunca concentró la búsqueda. El epsilon-greedy fijo y bajo resultó más estable en todas las variantes.
- **Supervivencia frente a progreso.** Los agentes estancados registran episodios que no se traducen en reward: sobreviven evitando colisiones pero sin aceleración neta. Es el patrón anticipado por el diagnóstico inicial de recompensa dispersa (episodios largos con reward bajo = elusión del choque sin progreso real).
- **Aportación arquitectónica.** El mayor salto de rendimiento no vino de la política de exploración, sino de la arquitectura. La desagregación *Dueling* (P5) es la única propuesta que consigue 100/100 episodios sin ningún fallo, con un mínimo de 326 coches — frente a mínimos de 192 (P1), 158 (P4) o 159 (P8). Sorprendentemente, DQN vanilla (P1) obtiene la media más alta (459.1) con suficiente entrenamiento, lo que sugiere que las mejoras algorítmicas aportan principalmente eficiencia de muestreo y robustez, no necesariamente una media superior.
- **Conclusión.** Se selecciona la **Propuesta 5 (Dueling DQN)** como mejor solución final del grupo, por ser la única que cumple el objetivo de forma perfecta (100/100 episodios consecutivos, mínimo 326 coches, media 440.8). Todas las propuestas con

epsilon-greedy fijo (P1, P4, P5, P6, P8) consiguieron aprender a superar el día 1, mientras que las propuestas con annealing mal configurado (P2, P7) y Boltzmann (P3) no aprendieron ninguna política útil. Esto pone de manifiesto que la calibración correcta del annealing es tan crítica como la elección del algoritmo.

#### 4.11.1. Justificación de los hiperparámetros comunes

Los siguientes hiperparámetros se mantuvieron fijos en todas las propuestas, siguiendo directamente las especificaciones de Mnih et al. (2015):

- **learning\_rate = 0.00025** — valor estándar del paper, compatible con el optimizador Adam con gradientes estables en redes convolucionales profundas.
- **gamma = 0.99** — horizonte temporal largo, adecuado para Enduro donde la recompensa (adelantar un coche) puede estar varios frames por delante de la acción que la provocó.
- **target\_model\_update = 10000** — actualización periódica de la red objetivo cada 10k steps, que estabiliza el entrenamiento evitando el problema del objetivo móvil (*moving target*).
- **replay buffer = 1.000.000** — mismo tamaño que el paper, permite almacenar suficiente diversidad de experiencias para romper la correlación temporal entre muestras consecutivas.
- **WINDOW\_LENGTH = 4** — apilamiento de 4 frames consecutivos para capturar la velocidad y dirección de los objetos, siguiendo la arquitectura original de DQN.
- **input\_shape = (84, 84)** — resolución estándar de Atari tras el preprocesado en escala de grises.
- **reward clipping** — `np.clip(reward, -1., 1.)` implementado en `AtariProcessor.process_reward()`, obligatorio según el enunciado y consistente con Mnih et al. (2015). En Enduro el reward nunca supera +1 por diseño del entorno, por lo que el clipping es neutro en la práctica pero garantiza consistencia formal.

##### ◦ Limitación computacional respecto al paper original

El paper de Mnih et al. (2015) entrenó durante **50 millones de frames** (equivalentes a **12.5 millones de steps** con `frame_skip=4`). Nuestras propuestas entrenaron entre **800k y 2.8M steps** — entre el 6% y el 22% del entrenamiento original. A pesar de esta limitación, P5 (Dueling DQN) consiguió el objetivo del proyecto (100/100 episodios consecutivos  $\geq 200$  coches), lo que evidencia que las mejoras algorítmicas permiten alcanzar rendimientos competitivos con recursos computacionales significativamente más reducidos.

Esta limitación explica también por qué el annealing de epsilon no mostró ventajas sobre epsilon fijo en este experimento: el annealing requiere un presupuesto de steps suficiente para completar la fase de exploración y luego explotar durante un periodo prolongado. Con presupuestos de 1-2M steps, epsilon fijo y bajo resulta más eficiente porque maximiza el tiempo de explotación desde el inicio.

##### ◦ Modelo final seleccionado

Se entrega como modelo final la **Propuesta 5 (Dueling DQN, checkpoint 2.3M steps)**, por ser la única que cumple el objetivo de forma perfecta y consistente:

- **100/100 episodios consecutivos** superan el día 1 ( $\geq 200$  coches adelantados) 
- **Media: 440.8 coches** | Std: 52.4 | Mín: 326 | Máx: 588
- **11/100 episodios** llegaron al día 3 (10.000 steps)
- Evaluación greedy ( `DQNAgent.test()` ) con `GreedyQPolicy` )

Los pesos del modelo final se entregan en el archivo `dqn_Enduro-v0_p5_weights_2300000.h5f` , incluido en el ZIP de entrega junto con este notebook.

## 4.12. Justificación de los parámetros seleccionados y de los resultados obtenidos

### 4.12.1. Tabla de correspondencia: modelos, logs y fases de entrenamiento

La siguiente tabla documenta, para cada propuesta, la correspondencia entre el directorio de pesos, el archivo de log disponible, las fases de entrenamiento realizadas y la cobertura real del log.

```
In [29]: # TABLA DE CORRESPONDENCIA: MODELOS, LOGS Y CHECKPOINTS DISPONIBLES

import os, glob, re, json
from IPython.display import display, Markdown

resumen = {
    'p1': {'label': 'P1 - DQN vanilla (baseline)',          'dir': WEIGHTS_DIR_P1},
    'p2': {'label': 'P2 - DQN + annealing',                'dir': WEIGHTS_DIR_P2},
    'p3': {'label': 'P3 - DQN + Boltzmann',                'dir': WEIGHTS_DIR_P3},
    'p4': {'label': 'P4 - Double DQN',                    'dir': WEIGHTS_DIR_P4},
    'p5': {'label': 'P5 - Dueling DQN  (modelo final)', 'dir': WEIGHTS_DIR_P5},
    'p6': {'label': 'P6 - Double + Dueling DQN',           'dir': WEIGHTS_DIR_P6},
    'p7': {'label': 'P7 - Double DQN + annealing',         'dir': WEIGHTS_DIR_P7},
    'p8': {'label': 'P8 - Dueling DQN + annealing',        'dir': WEIGHTS_DIR_P8},
}

fases = {
    'p1': 'Tanda 1 (0→1.8M, eps=0.2) **sin log** · Tanda 2 (1.8M→2.3M, eps=0.05)',
    'p2': 'Tanda única (0→1.7M, annealing nb_steps=1.2M) **log reiniciado** (1 episodio)',
    'p3': 'Tanda única (0→800k, Boltzmann τ=1.0) **log reiniciado** (1 episodio)',
    'p4': 'Tanda única (0→1.7M, eps=0.2) **log reiniciado** (1 episodio)',
    'p5': 'Tanda 1 (0→1.8M, eps=0.2) **sin log** · Tanda 2 (1.8M→2.3M, eps=0.1)',
    'p6': 'Tanda 1 (0→1.8M, eps=0.2) · Tanda 2 (1.8M→2.3M, eps=0.1) · Tanda 3 (2.3M→2.8M, eps=0.1)',
    'p7': 'Tanda única (0→1.2M, annealing nb_steps=1.2M) **log completo disponible**',
    'p8': 'Tanda 1 (0→1.3M, nb_steps=2M incorrecto) **sin log** · Tanda 2 (1.3M→1.8M, nb_steps=2M incorrecto)',
}

print("⚠️  NOTA SOBRE LOS LOGS")
print("keras-rl2 (FileLogger) sobrescribe el log en cada nueva llamada a dqn")
print("Los logs disponibles corresponden a la última tanda de entrenamiento de cada")
print("propuesta. Los checkpoints intermedios (cada 100k steps) sí están completos")
print()

rows = ['| Propuesta | Checkpoint final | Checkpoints disponibles | Log disponible |']
```

```

        '|---|---|---|---|---|']

for key, info in resumen.items():
    d, prefix, label = info['dir'], info['prefix'], info['label']

    # Checkpoints
    ckpts = sorted([
        int(re.search(r'_weights_(\d+)\.h5f', p).group(1))
        for p in glob.glob(os.path.join(d, f'{prefix}_weights_*.h5f'))
        if not p.endswith('.index') and not p.endswith('.data-00000-of-00001')
    ])
    # Alternativamente buscar por .index
    if not ckpts:
        ckpts = sorted([
            int(re.search(r'_weights_(\d+)\.h5f\.index', p).group(1))
            for p in glob.glob(os.path.join(d, f'{prefix}_weights_*.h5f.index'))
        ])

    if ckpts:
        last = ckpts[-1]
        ckpt_final = f`_weights_{last:,}.h5f`
        ckpt_lista = ' . '.join(f`c//1000k` for c in ckpts)
    else:
        ckpt_final = '-'
        ckpt_lista = '- no encontrados en ruta actual -'

    # Log
    log_path = os.path.join(d, f'dqn_{env_name}_{key}_log.json')
    if os.path.exists(log_path):
        try:
            with open(log_path) as f:
                lg = json.load(f)
                n = len(lg['episode_reward'])
                s0 = lg['nb_steps'][0]
                sf = lg['nb_steps'][-1]
                log_str = f'{n} ep · {s0:,}→{sf:,} steps'
        except:
            log_str = 'error al leer'
    else:
        log_str = '- no encontrado -'

    rows.append(f'| **{label}** | {ckpt_final} | {ckpt_lista} | {log_str} | {fas}')

display(Markdown('\n'.join(rows)))

```

⚠️ **NOTA SOBRE LOS LOGS**  
keras-rl2 (FileLogger) sobrescribe el log en cada nueva llamada a `dqn.fit()`. Los logs disponibles corresponden a la última tanda de entrenamiento de cada propuesta. Los checkpoints intermedios (cada 100k steps) sí están completos.

| Propuesta                          | Checkpoint final                    | Checkpoints disponibles | Log disponible                 | Fases de entrenamiento                                                                                  |
|------------------------------------|-------------------------------------|-------------------------|--------------------------------|---------------------------------------------------------------------------------------------------------|
| <b>P1 — DQN vanilla (baseline)</b> | <code>_weights_2,300,000.h5f</code> | 100k ·                  | 42 ep ·<br>4,438→297,359 steps | Tanda 1<br>(0→1.8M, eps=0.2) <b>sin log</b><br>· Tanda 2<br>(1.8M→2.3M, eps=0.05) <b>log disponible</b> |
|                                    |                                     | 200k ·                  |                                |                                                                                                         |
|                                    |                                     | 300k ·                  |                                |                                                                                                         |
|                                    |                                     | 400k ·                  |                                |                                                                                                         |
|                                    |                                     | 500k ·                  |                                |                                                                                                         |
|                                    |                                     | 600k ·                  |                                |                                                                                                         |
|                                    |                                     | 700k ·                  |                                |                                                                                                         |
|                                    |                                     | 800k ·                  |                                |                                                                                                         |
|                                    |                                     | 900k ·                  |                                |                                                                                                         |
|                                    |                                     | 1000k ·                 |                                |                                                                                                         |
|                                    |                                     | 1100k ·                 |                                |                                                                                                         |
|                                    |                                     | 1200k ·                 |                                |                                                                                                         |
|                                    |                                     | 1300k ·                 |                                |                                                                                                         |
|                                    |                                     | 1400k ·                 |                                |                                                                                                         |
|                                    |                                     | 1500k ·                 |                                |                                                                                                         |
|                                    |                                     | 1600k ·                 |                                |                                                                                                         |
|                                    |                                     | 1700k ·                 |                                |                                                                                                         |
|                                    |                                     | 1800k ·                 |                                |                                                                                                         |
|                                    |                                     | 1900k ·                 |                                |                                                                                                         |
|                                    |                                     | 2000k ·                 |                                |                                                                                                         |
|                                    |                                     | 2100k ·                 |                                |                                                                                                         |
|                                    |                                     | 2200k ·                 |                                |                                                                                                         |
|                                    |                                     | 2300k                   |                                |                                                                                                         |
| <b>P2 — DQN + annealing</b>        | <code>_weights_1,700,000.h5f</code> | 100k ·                  | 1 ep ·<br>4,438→4,438 steps    | Tanda única<br>(0→1.7M, annealing nb_steps=1.2M) <b>log reiniciado</b><br>(1 episodio)                  |
|                                    |                                     | 200k ·                  |                                |                                                                                                         |
|                                    |                                     | 300k ·                  |                                |                                                                                                         |
|                                    |                                     | 400k ·                  |                                |                                                                                                         |
|                                    |                                     | 500k ·                  |                                |                                                                                                         |
|                                    |                                     | 600k ·                  |                                |                                                                                                         |
|                                    |                                     | 700k ·                  |                                |                                                                                                         |
|                                    |                                     | 800k ·                  |                                |                                                                                                         |
|                                    |                                     | 900k ·                  |                                |                                                                                                         |
|                                    |                                     | 1000k ·                 |                                |                                                                                                         |
|                                    |                                     | 1100k ·                 |                                |                                                                                                         |
|                                    |                                     | 1200k ·                 |                                |                                                                                                         |
|                                    |                                     | 1300k ·                 |                                |                                                                                                         |
|                                    |                                     | 1400k ·                 |                                |                                                                                                         |
|                                    |                                     | 1500k ·                 |                                |                                                                                                         |
|                                    |                                     | 1600k ·                 |                                |                                                                                                         |
|                                    |                                     | 1700k                   |                                |                                                                                                         |
| <b>P3 — DQN + Boltzmann</b>        | <code>_weights_800,000.h5f</code>   | 100k ·                  | 1 ep ·<br>4,438→4,438 steps    | Tanda única<br>(0→800k, Boltzmann $\tau=1.0$ ) <b>log reiniciado</b> (1 episodio)                       |
|                                    |                                     | 200k ·                  |                                |                                                                                                         |
|                                    |                                     | 300k ·                  |                                |                                                                                                         |
|                                    |                                     | 400k ·                  |                                |                                                                                                         |
|                                    |                                     | 500k ·                  |                                |                                                                                                         |
|                                    |                                     | 600k ·                  |                                |                                                                                                         |
|                                    |                                     | 700k ·                  |                                |                                                                                                         |
|                                    |                                     | 800k                    |                                |                                                                                                         |
| <b>P4 — Double DQN</b>             | <code>_weights_1,700,000.h5f</code> | 100k ·                  | 1 ep ·<br>4,438→4,438 steps    | Tanda única<br>(0→1.7M, eps=0.2) <b>log</b>                                                             |
|                                    |                                     | 200k ·                  |                                |                                                                                                         |
|                                    |                                     | 300k                    |                                |                                                                                                         |

| Propuesta | Checkpoint final | Checkpoints disponibles | Log disponible      | Fases de entrenamiento                               |
|-----------|------------------|-------------------------|---------------------|------------------------------------------------------|
|           |                  | 400k ·                  |                     | <b>reiniciado</b> (1 episodio)                       |
|           |                  | 500k ·                  |                     |                                                      |
|           |                  | 600k ·                  |                     |                                                      |
|           |                  | 700k ·                  |                     |                                                      |
|           |                  | 800k ·                  |                     |                                                      |
|           |                  | 900k ·                  |                     |                                                      |
|           |                  | 1000k ·                 |                     |                                                      |
|           |                  | 1100k ·                 |                     |                                                      |
|           |                  | 1200k ·                 |                     |                                                      |
|           |                  | 1300k ·                 |                     |                                                      |
|           |                  | 1400k ·                 |                     |                                                      |
|           |                  | 1500k ·                 |                     |                                                      |
|           |                  | 1600k ·                 |                     |                                                      |
|           |                  | 1700k                   |                     |                                                      |
|           |                  | 100k ·                  |                     |                                                      |
|           |                  | 200k ·                  |                     |                                                      |
|           |                  | 300k ·                  |                     |                                                      |
|           |                  | 400k ·                  |                     |                                                      |
|           |                  | 500k ·                  |                     |                                                      |
|           |                  | 600k ·                  |                     |                                                      |
|           |                  | 700k ·                  |                     |                                                      |
|           |                  | 800k ·                  |                     |                                                      |
|           |                  | 900k ·                  |                     |                                                      |
|           |                  | 1000k ·                 |                     |                                                      |
|           |                  | 1100k ·                 | 38 ep ·             | Tanda 1 (0→1.8M, eps=0.2) <b>sin log</b>             |
|           |                  | 1200k ·                 |                     |                                                      |
|           |                  | 1300k ·                 | 4,438→199,723 steps | · Tanda 2 (1.8M→2.3M, eps=0.1) <b>log disponible</b> |
|           |                  | 1400k ·                 |                     |                                                      |
|           |                  | 1500k ·                 |                     |                                                      |
|           |                  | 1600k ·                 |                     |                                                      |
|           |                  | 1700k ·                 |                     |                                                      |
|           |                  | 1800k ·                 |                     |                                                      |
|           |                  | 1900k ·                 |                     |                                                      |
|           |                  | 2000k ·                 |                     |                                                      |
|           |                  | 2100k ·                 |                     |                                                      |
|           |                  | 2200k ·                 |                     |                                                      |
|           |                  | 2300k                   |                     |                                                      |
|           |                  | 100k ·                  |                     |                                                      |
|           |                  | 200k ·                  |                     |                                                      |
|           |                  | 300k ·                  |                     |                                                      |
|           |                  | 400k ·                  |                     |                                                      |
|           |                  | 500k ·                  |                     |                                                      |
|           |                  | 600k ·                  |                     |                                                      |
|           |                  | 700k ·                  |                     |                                                      |
|           |                  | 800k ·                  |                     |                                                      |
|           |                  | 900k ·                  |                     |                                                      |
|           |                  | 1000k ·                 |                     |                                                      |
|           |                  | 1100k ·                 |                     |                                                      |
|           |                  | 1200k ·                 |                     |                                                      |
|           |                  | 1300k ·                 |                     |                                                      |
|           |                  | 1400k ·                 |                     |                                                      |
|           |                  | 1500k ·                 |                     |                                                      |
|           |                  |                         |                     |                                                      |

| Propuesta                                       | Checkpoint final       | Checkpoints disponibles | Log disponible                     | Fases de entrenamiento                                                                                                                                 |
|-------------------------------------------------|------------------------|-------------------------|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                 |                        | 1600k ·                 |                                    |                                                                                                                                                        |
|                                                 |                        | 1700k ·                 |                                    |                                                                                                                                                        |
|                                                 |                        | 1800k ·                 |                                    |                                                                                                                                                        |
|                                                 |                        | 1900k ·                 |                                    |                                                                                                                                                        |
|                                                 |                        | 2000k ·                 |                                    |                                                                                                                                                        |
|                                                 |                        | 2100k ·                 |                                    |                                                                                                                                                        |
|                                                 |                        | 2200k ·                 |                                    |                                                                                                                                                        |
|                                                 |                        | 2300k ·                 |                                    |                                                                                                                                                        |
|                                                 |                        | 2400k ·                 |                                    |                                                                                                                                                        |
|                                                 |                        | 2500k ·                 |                                    |                                                                                                                                                        |
|                                                 |                        | 2600k ·                 |                                    |                                                                                                                                                        |
|                                                 |                        | 2700k ·                 |                                    |                                                                                                                                                        |
|                                                 |                        | 2800k                   |                                    |                                                                                                                                                        |
|                                                 |                        |                         |                                    |                                                                                                                                                        |
| <b>P7 —<br/>Double<br/>DQN +<br/>annealing</b>  | _weights_1,200,000.h5f | 100k ·                  | 180 ep ·<br>4,436→798,780<br>steps | Tanda única<br>(0→1.2M,<br>annealing<br>nb_steps=1.2M)<br><b>log completo<br/>disponible</b>                                                           |
|                                                 |                        | 200k ·                  |                                    |                                                                                                                                                        |
|                                                 |                        | 300k ·                  |                                    |                                                                                                                                                        |
|                                                 |                        | 400k ·                  |                                    |                                                                                                                                                        |
|                                                 |                        | 500k ·                  |                                    |                                                                                                                                                        |
|                                                 |                        | 600k ·                  |                                    |                                                                                                                                                        |
|                                                 |                        | 700k ·                  |                                    |                                                                                                                                                        |
|                                                 |                        | 800k ·                  |                                    |                                                                                                                                                        |
|                                                 |                        | 900k ·                  |                                    |                                                                                                                                                        |
|                                                 |                        | 1000k ·                 |                                    |                                                                                                                                                        |
|                                                 |                        | 1100k ·                 |                                    |                                                                                                                                                        |
|                                                 |                        | 1200k                   |                                    |                                                                                                                                                        |
| <b>P8 —<br/>Dueling<br/>DQN +<br/>annealing</b> | _weights_2,300,000.h5f | 100k ·                  | 109 ep ·<br>4,440→497,055<br>steps | Tanda 1<br>(0→1.3M,<br>nb_steps=2M<br>incorrecto) <b>sin<br/>log</b> · Tanda 2<br>(1.3M→2.3M,<br>nb_steps=300k<br>corregido) <b>log<br/>disponible</b> |
|                                                 |                        | 200k ·                  |                                    |                                                                                                                                                        |
|                                                 |                        | 300k ·                  |                                    |                                                                                                                                                        |
|                                                 |                        | 400k ·                  |                                    |                                                                                                                                                        |
|                                                 |                        | 500k ·                  |                                    |                                                                                                                                                        |
|                                                 |                        | 600k ·                  |                                    |                                                                                                                                                        |
|                                                 |                        | 700k ·                  |                                    |                                                                                                                                                        |
|                                                 |                        | 800k ·                  |                                    |                                                                                                                                                        |
|                                                 |                        | 900k ·                  |                                    |                                                                                                                                                        |
|                                                 |                        | 1000k ·                 |                                    |                                                                                                                                                        |
|                                                 |                        | 1100k ·                 |                                    |                                                                                                                                                        |
|                                                 |                        | 1200k ·                 |                                    |                                                                                                                                                        |
|                                                 |                        | 1300k ·                 |                                    |                                                                                                                                                        |
|                                                 |                        | 1400k ·                 |                                    |                                                                                                                                                        |
|                                                 |                        | 1500k ·                 |                                    |                                                                                                                                                        |
|                                                 |                        | 1600k ·                 |                                    |                                                                                                                                                        |
|                                                 |                        | 1700k ·                 |                                    |                                                                                                                                                        |
|                                                 |                        | 1800k ·                 |                                    |                                                                                                                                                        |
|                                                 |                        | 1900k ·                 |                                    |                                                                                                                                                        |
|                                                 |                        | 2000k ·                 |                                    |                                                                                                                                                        |
|                                                 |                        | 2100k ·                 |                                    |                                                                                                                                                        |
|                                                 |                        | 2200k ·                 |                                    |                                                                                                                                                        |
|                                                 |                        | 2300k                   |                                    |                                                                                                                                                        |

#### 4.12.2. Checkpoints intermedios disponibles

Todos los checkpoints guardados cada 100.000 steps están disponibles en las subcarpetas de la entrega. Esto permite evaluar la evolución del modelo en cualquier punto del entrenamiento cargando el peso correspondiente con:

```
weights = os.path.join(WEIGHTS_DIR_P5, 'dqn_Enduro-
v0_p5_weights_500000.h5f')
dqn_p5.load_weights(weights)
dqn_p5.test(env, nb_episodes=10, visualize=False)
```

Los checkpoints disponibles por propuesta son:

| P  | Checkpoints disponibles                      |
|----|----------------------------------------------|
| P1 | 100k, 200k, 300k, ..., 2.3M (23 checkpoints) |
| P2 | 100k, 200k, ..., 1.7M (17 checkpoints)       |
| P3 | 100k, 200k, ..., 800k (8 checkpoints)        |
| P4 | 100k, 200k, ..., 1.7M (17 checkpoints)       |
| P5 | 100k, 200k, ..., 2.3M (23 checkpoints)       |
| P6 | 100k, 200k, ..., 2.8M (28 checkpoints)       |
| P7 | 100k, 200k, ..., 1.2M (12 checkpoints)       |
| P8 | 100k, 200k, ..., 2.3M (23 checkpoints)       |

```
In [30]: # GRÁFICAS DE EVOLUCIÓN DEL ENTRENAMIENTO – ANÁLISIS COMPLETO
# Muestra explícitamente qué tramos tienen log y cuáles no,
# y anota las tandas de entrenamiento con sus hiperparámetros.

import os, json, glob, re
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

REWARD_THRESHOLD = 200
WINDOW = 10

# — Cargar logs disponibles —————
def load_log(weights_dir, key):
    path = os.path.join(weights_dir, f'dqn_{env_name}_{key}_log.json')
    if not os.path.exists(path):
        print(f' [AVISO] Log de {key} no encontrado en {weights_dir}')
        return None
    with open(path) as f:
        return json.load(f)

def get_checkpoints(weights_dir, prefix):
    ckpts = []
    for p in glob.glob(os.path.join(weights_dir, f'{prefix}_weights_*.h5f.index*')):
        m = re.search(r'_weights_(\d+)\.h5f\.index$', p)
        if m:
            ckpts.append(int(m.group(1)))
    return sorted(ckpts)

all_logs = {k: load_log(v['dir'], k) for k, v in AGENTS_REGISTRY.items()}
all_ckpts = {k: get_checkpoints(v['dir'], v['prefix']) for k, v in AGENTS_REGIST
```

```

labels_map = {
    'p1': 'P1 - DQN vanilla (baseline)',
    'p2': 'P2 - DQN + annealing lineal',
    'p3': 'P3 - DQN + Boltzmann',
    'p4': 'P4 - Double DQN',
    'p5': 'P5 - Dueling DQN ☒ (modelo final)',
    'p6': 'P6 - Double + Dueling DQN',
    'p7': 'P7 - Double DQN + annealing',
    'p8': 'P8 - Dueling DQN + annealing',
}

tandas = {
    'p1': [(0, 1_800_000, 'Tanda 1: eps=0.2 (sin log)', '#fff3e0'),
           (1_800_000, 2_300_000, 'Tanda 2: eps=0.05 (log disponible)', '#e8f5e9')],
    'p2': [(0, 1_700_000, 'Tanda única: annealing nb_steps=1.2M (log rei',
    'p3': [(0, 800_000, 'Tanda única: Boltzmann  $\tau$ =1.0 (log reiniciado)',
    'p4': [(0, 1_700_000, 'Tanda única: eps=0.2 (log reiniciado)',
    'p5': [(0, 1_800_000, 'Tanda 1: eps=0.2 (sin log)', '#fff3e0'),
           (1_800_000, 2_300_000, 'Tanda 2: eps=0.1 (log disponible)', '#e8f5e9')],
    'p6': [(0, 1_800_000, 'Tanda 1: eps=0.2 (sin log)', '#fff3e0'),
           (1_800_000, 2_300_000, 'Tanda 2: eps=0.1 (sin log)', '#fff3e0'),
           (2_300_000, 2_800_000, 'Tanda 3: eps=0.05 (log disponible)', '#e8f5e9')],
    'p7': [(0, 1_200_000, 'Tanda única: annealing nb_steps=1.2M (log com',
    'p8': [(0, 1_300_000, 'Tanda 1: nb_steps=2M incorrecto (sin log)',
           (1_300_000, 2_300_000, 'Tanda 2: nb_steps=300k corregido (log disponi

}

# — GRÁFICA 1: Evolución por propuesta (todas) —————
todas_props = list(AGENTS_REGISTRY.keys()) # p1 ... p8

fig1, axes = plt.subplots(8, 1, figsize=(14, 5 * 8))

fig1.suptitle('Evolución del entrenamiento por propuesta',
              fontsize=14, fontweight='bold', y=0.995)

legend_patches = [
    mpatches.Patch(color='#e8f5e9', alpha=0.7, label='Log disponible'),
    mpatches.Patch(color='#fff3e0', alpha=0.7, label='Sin log (sobrescrito al r',
    mpatches.Patch(color='#ffebee', alpha=0.7, label='Configuración incorrecta')
]
fig1.legend(handles=legend_patches, loc='upper center',
            bbox_to_anchor=(0.5, 0.990), ncol=3, fontsize=9,
            framealpha=0.9, edgecolor='c#cccccc')

fig1.subplots_adjust(top=0.975, hspace=0.8)

for ax, key in zip(axes, todas_props):
    d = all_logs.get(key)
    max_step = max(all_ckpts.get(key, [1]) or [1])

    # — Fondos de tandas —
    for (s0, s1, etiq, color) in tandas.get(key, []):
        s1_clip = min(s1, max_step * 1.05)
        ax.axvspan(s0, s1_clip, alpha=0.25, color=color, zorder=0)
        mid = (s0 + min(s1, max_step)) / 2
        ax.text(mid, 590, etiq, ha='center', va='top',
                fontsize=7.5, color='#444', style='italic',
                bbox=dict(boxstyle='round,pad=0.2', facecolor='white', alpha=0.7)

```

```

# — Checkpoints (líneas verticales grises) —
for ck in all_ckpts.get(key, []):
    ax.axvline(ck, color='#BDBDBD', linewidth=0.5, alpha=0.6, zorder=1)

# — Sin datos suficientes: P2, P3, P4 —
if d is None or len(d.get('episode_reward', [])) < 3:
    n_ep = len(d['episode_reward']) if d else 0
    ax.text(0.5, 0.45,
            f'Log insuficiente: solo {n_ep} episodio(s) registrado(s)\n'
            f'El FileLogger de keras-rl2 sobrescribió el log al reanudar el\n'
            f'Los checkpoints intermedios (cada 100k steps) sí están disponi\n'
            ha='center', va='center', transform=ax.transAxes,
            fontsize=10, color='#B71C1C',
            bbox=dict(boxstyle='round,pad=0.6', facecolor='#FFECEE', alpha=0.8))
    ax.set_title(
        f'"{labels_map.get(key, key.upper())}"\n'
        f"Checkpoint final: {max_step/1e6:.1f}M steps | "
        f'Log: solo {n_ep} episodio(s) – reiniciado al reanudar",
        fontsize=9, fontweight='bold', pad=6)
    ax.set_xlim(0, max_step * 1.05)
    ax.set_ylim(0, 620)
    ax.set_xlabel('Steps de entrenamiento totales', fontsize=8)
    ax.set_ylabel('Reward (coches)', fontsize=8)
    ax.grid(alpha=0.25)
    ax.xaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{x/1e6:.1f}'))
    continue

r = np.array(d['episode_reward'])
s = np.array(d['nb_steps'])

# Reward crudo
ax.plot(s, r, color='#90CAF9', alpha=0.5, linewidth=0.8,
        label='Reward por episodio', zorder=2)

# Media móvil
if len(r) >= WINDOW:
    rm = np.convolve(r, np.ones(WINDOW) / WINDOW, mode='valid')
    ax.plot(s[WINDOW - 1:], rm, color='#1565C0', linewidth=2,
            label=f'Media móvil ({WINDOW} ep)', zorder=3)

# Línea de objetivo
ax.axhline(REWARD_THRESHOLD, color='red', linestyle='--',
            linewidth=1.2, alpha=0.8, label=f'Objetivo ({REWARD_THRESHOLD} coches)', zorder=4)

# Anotación fin del log
if max_step > s[-1] + 200_000:
    last_r = rm[-1] if len(r) >= WINDOW else r[-1]
    ax.annotate(
        f'Fin del log\nstep {s[-1]:,}',
        xy=(s[-1], last_r),
        xytext=(s[-1] + (max_step - s[-1]) * 0.12, 80),
        fontsize=8, color='#B71C1C',
        arrowprops=dict(arrowstyle='->', color='#B71C1C', lw=1.2),
        bbox=dict(boxstyle='round,pad=0.3', facecolor='#FFECEE', alpha=0.8))

ax.set_title(
    f'"{labels_map.get(key, key.upper())}"\n'
    f"Checkpoint final: {max_step/1e6:.1f}M steps | "
    f'Log disponible: {s[0]:,} → {s[-1]:,} steps ({len(r)} episodios)",
    fontsize=9, fontweight='bold', pad=6)

```

```

ax.set_xlabel('Steps de entrenamiento totales', fontsize=8)
ax.set_ylabel('Reward (coches)', fontsize=8)
ax.set_xlim(0, max_step * 1.05)
ax.set_ylim(0, 620)
ax.legend(fontsize=8, loc='upper left')
ax.grid(alpha=0.25)
ax.xaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{x/1e6:.1f}M'))

plt.show()

# — GRÁFICA 2: Comparativa de resultados de test —————
fig2, axes2 = plt.subplots(1, 2, figsize=(14, 5))
fig2.suptitle('Resultados de test – comparativa de todas las propuestas\n'
              '(evaluación greedy: DQNAgent.test() usa GreedyQPolicy)',
              fontsize=12, fontweight='bold')

COLORS = {'p1': '#1f77b4', 'p4': '#ff7f0e', 'p5': '#2ca02c',
          'p6': '#d62728', 'p7': '#9467bd', 'p8': '#8c564b'}

# Panel izquierdo: media móvil en el log disponible
ax_l = axes2[0]
for key in todas_props:
    d = all_logs.get(key)
    if d is None or len(d.get('episode_reward', [])) < WINDOW:
        continue
    r = np.array(d['episode_reward'])
    s = np.array(d['nb_steps'])
    rm = np.convolve(r, np.ones(WINDOW) / WINDOW, mode='valid')
    ax_l.plot(s[WINDOW-1:], rm,
              label=labels_map.get(key, key),
              color=COLORS.get(key, '#333'),
              linewidth=1.8)

ax_l.axhline(REWARD_THRESHOLD, color='red', linestyle='--',
              linewidth=1.5, label=f'Objetivo ({REWARD_THRESHOLD})')
ax_l.set_title('Media móvil del reward\n(tramo con log disponible – última tanda
               fontsize=10)
ax_l.set_xlabel('Steps en la última tanda de entrenamiento')
ax_l.set_ylabel('Reward medio (coches)')
ax_l.legend(fontsize=8)
ax_l.grid(alpha=0.3)
ax_l.xaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'{int(x/1000)}k'))

# Panel derecho: resultados de test (barras)
ax_r = axes2[1]
test_results = {
    'P1\n2.3M': (99, 459.1, '#1f77b4'),
    'P2\n1.7M': (0, 0.0, '#aec7e8'),
    'P3\n800k': (0, 0.0, '#aec7e8'),
    'P4\n1.7M': (94, 396.9, '#ff7f0e'),
    'P5\n2.3M 🟢': (100, 440.8, '#2ca02c'),
    'P6\n2.8M': (95, 431.5, '#d62728'),
    'P7\n1.2M': (0, 0.0, '#aec7e8'),
    'P8\n2.3M': (91, 398.3, '#9467bd'),
}

x = np.arange(len(test_results))
bars = ax_r.bar(x,
                [v[0] for v in test_results.values()],

```

```

        color=[v[2] for v in test_results.values()],
        edgecolor='white', width=0.65)
ax_r.axhline(100, color='red', linestyle='--', linewidth=1.5, label='Objetivo: 1
ax_r.set_xticks(x)
ax_r.set_xticklabels(list(test_results.keys()), fontsize=8.5)
ax_r.set_ylabel('% episodios  $\geq$  200 coches')
ax_r.set_title('Resultados finales de test\n(100 episodios por propuesta)', font
ax_r.set_ylim(0, 120)
ax_r.legend(fontsize=9)
ax_r.grid(alpha=0.3, axis='y')

for bar, (pct, media, _) in zip(bars, test_results.values()):
    if pct > 0:
        ax_r.text(bar.get_x() + bar.get_width() / 2,
                   bar.get_height() + 1,
                   f'{pct}%\n{media:.0f} coches',
                   ha='center', va='bottom', fontsize=7.5, fontweight='bold')

plt.tight_layout()
plt.show()

print()
print('✓ Gráficas generadas correctamente.')

```

```

/usr/local/lib/python3.11/dist-packages/IPython/core/pylabtools.py:151: UserWarni
ng: Glyph 9989 (\N{WHITE HEAVY CHECK MARK}) missing from font(s) DejaVu Sans.
fig.canvas.print_figure(bytes_io, **kw)

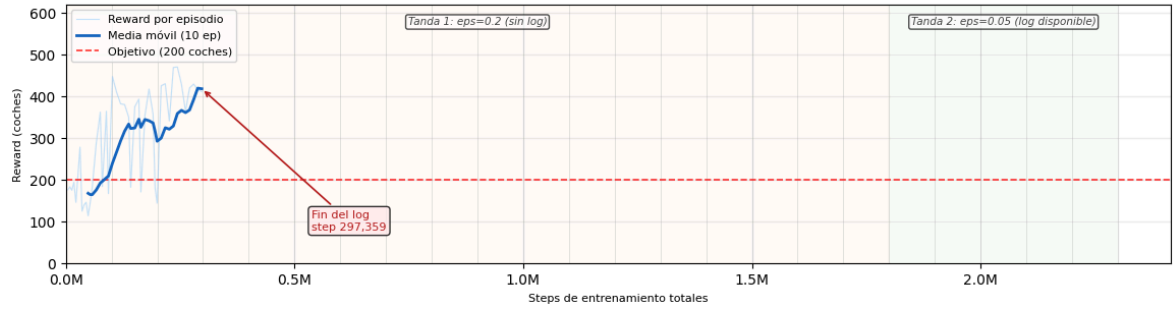
```

## Evolución del entrenamiento por propuesta

Log disponible Sin log (sobrescrito al reanudar) Configuración incorrecta

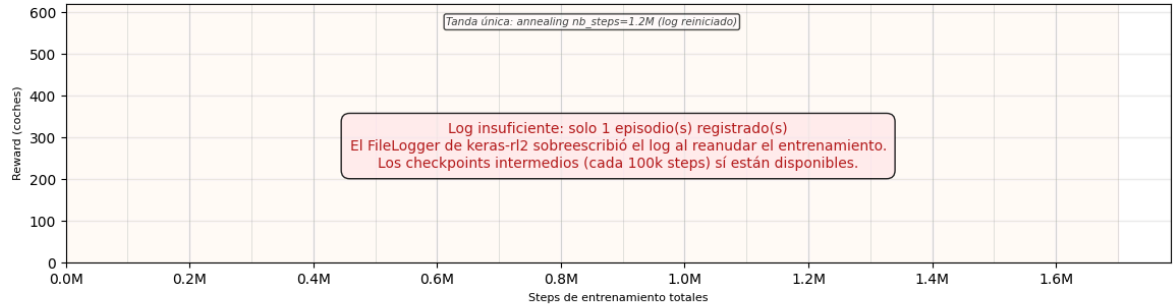
P1 — DQN vanilla (baseine)

Checkpoint final: 2.3M steps | Log disponible: 4,438 → 297,359 steps (42 episodios)



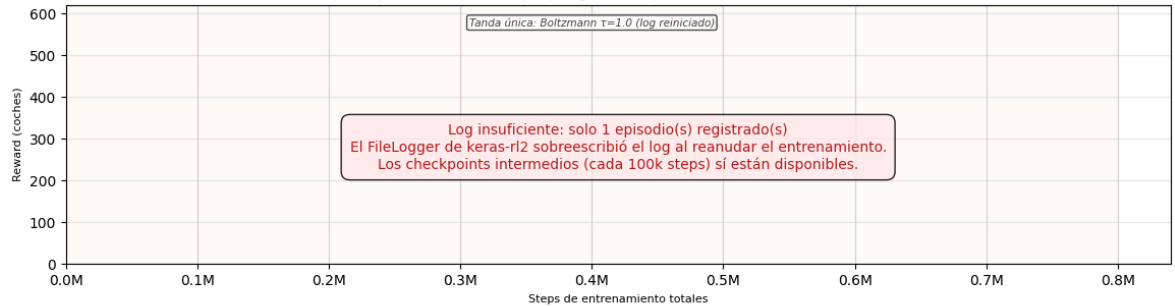
P2 — DQN + annealing lineal

Checkpoint final: 1.7M steps | Log: solo 1 episodio(s) — reiniciado al reanudar



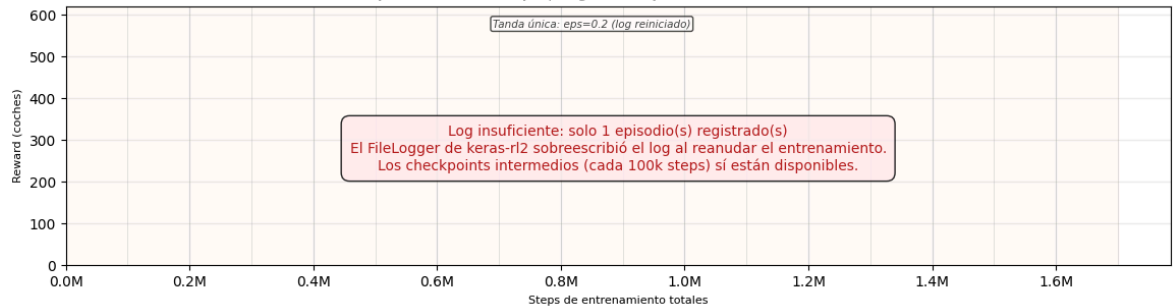
P3 — DQN + Boltzmann

Checkpoint final: 0.8M steps | Log: solo 1 episodio(s) — reiniciado al reanudar



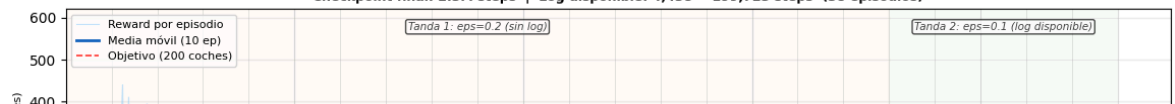
P4 — Double DQN

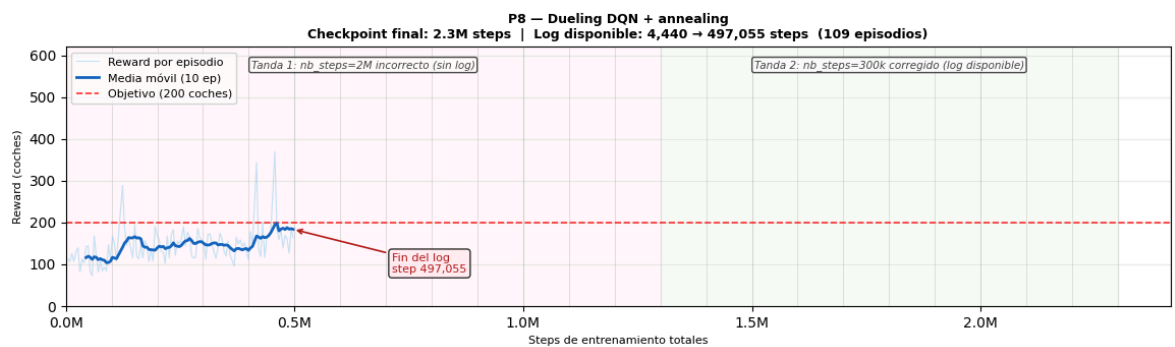
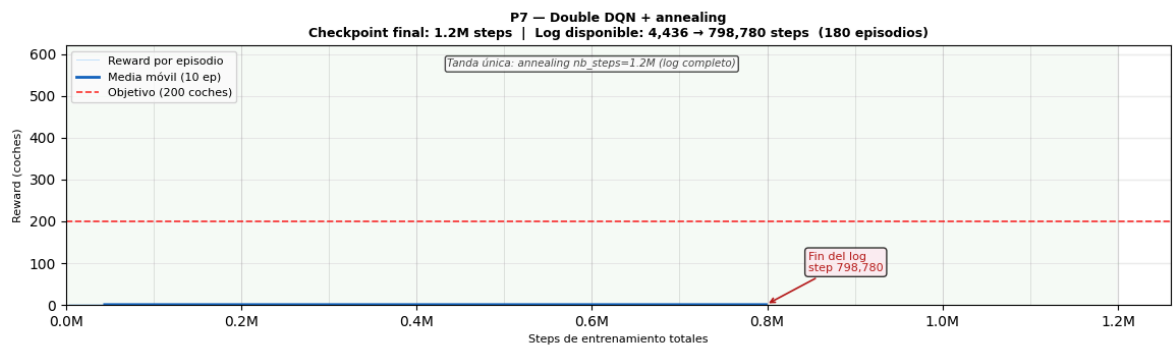
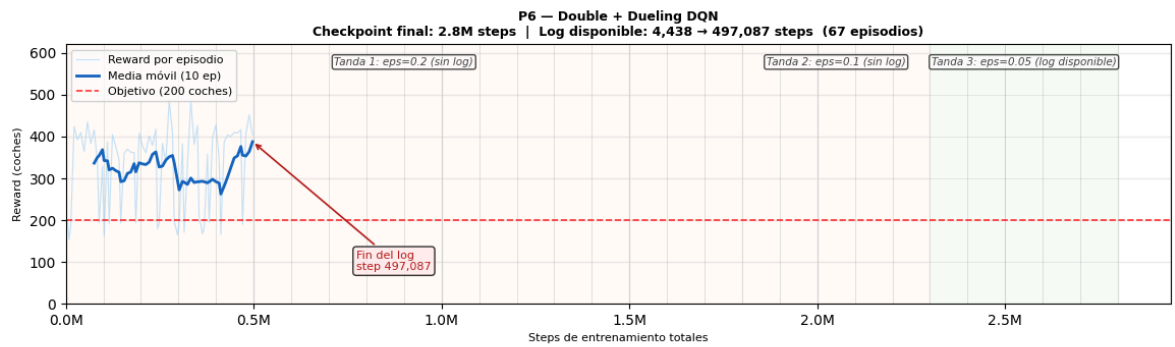
Checkpoint final: 1.7M steps | Log: solo 1 episodio(s) — reiniciado al reanudar



P5 — Dueling DQN (modelo final)

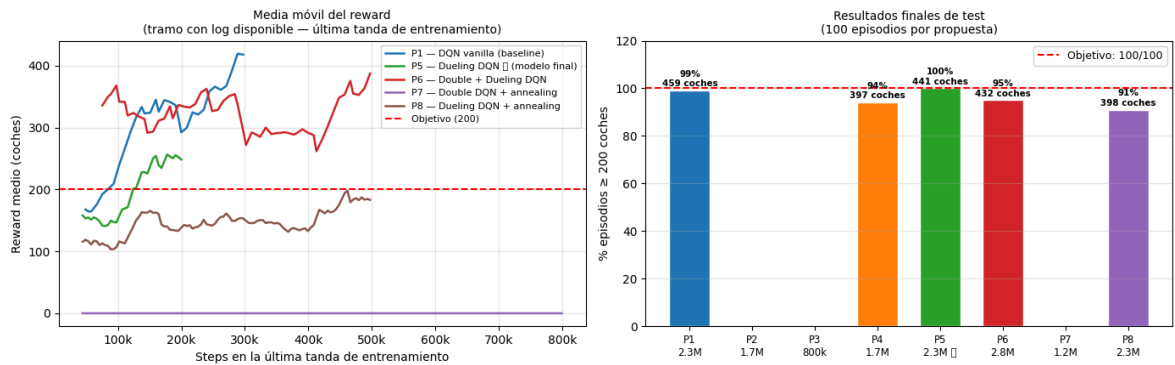
Checkpoint final: 2.3M steps | Log disponible: 4,438 → 199,723 steps (38 episodios)





```
/tmp/ipython-input-30-1859931209.py:230: UserWarning: Glyph 9989 (\N{WHITE HEAVY CHECK MARK}) missing from font(s) DejaVu Sans.
plt.tight_layout()
/usr/local/lib/python3.11/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 9989 (\N{WHITE HEAVY CHECK MARK}) missing from font(s) DejaVu Sans.
fig.canvas.print_figure(bytes_io, **kw)
```

### Resultados de test — comparativa de todas las propuestas (evaluación greedy: DQNAgent.test() usa GreedyQPolicy)



✓ Gráficas generadas correctamente.

## 4.13. Estructura de la entrega y acceso a los pesos

El ZIP de entrega contiene el notebook y todos los checkpoints:

```

Entrega_Grupo_17/
├── Proyecto_practico_Enduro_DQN_Grupo_17_08MIAR_10_B_2025-26.ipynb
├── checkpoints/
│   ├── p1_eps_fijo/ — P1: DQN vanilla · checkpoints 100k → 2.3M + log
│   ├── p2_annealed/ — P2: DQN + annealing · checkpoints 100k → 1.7M + log
│   ├── p3_boltzmann/ — P3: DQN + Boltzmann · checkpoints 100k → 800k + log
│   ├── p4_double_dqn/ — P4: Double DQN · checkpoints 100k → 1.7M + log
│   ├── p5_dueling_dqn/ — P5: Dueling DQN (MODELO FINAL) · checkpoints 100k
│   │   └── → 2.3M + log
│   ├── p6_double_dueling/ — P6: Double+Dueling · checkpoints 100k → 2.8M +
│   │   └── log
│   ├── p7_double_dqn_anneal/ — P7: Double DQN + annealing · checkpoints
│   │   └── 100k → 1.2M + log
│   ├── p8_dueling_dqn_annea/ — P8: Dueling DQN + annealing · checkpoints
│   │   └── 100k → 2.3M + log
│   └── video_demo_p5/ — Vídeo de demostración del modelo final (P5)

```

**Para ejecutar en local** (sin Google Colab):

1. Extraer el ZIP manteniendo la estructura de carpetas
2. Abrir el notebook desde la carpeta `Entrega_Grupo_17/`

3. `IN_COLAB` se detecta como `False` automáticamente → usa `os.getcwd()` como raíz
  4. Las celdas de entrenamiento están desactivadas ( `RUN_TRAINING = False` )
  5. Para evaluar el modelo final: ejecutar la celda de policy de P5 y la celda de test
- 

## Fase V. Referencias y anexos

### Bibliografía

- Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., & Zaremba, W. (2016). OpenAI Gym. *arXiv preprint arXiv:1606.01540*.
- Camacho Camacho, J. M. (2021). *08MIAR: Aprendizaje por refuerzo (Sesiones 1-10)* [Diapositivas]. Aula Virtual, Universidad Internacional de Valencia.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533.
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2.<sup>a</sup> ed.). MIT Press.
- van Hasselt, H., Guez, A., & Silver, D. (2016). Deep reinforcement learning with double Q-learning. En *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence* (pp. 2094–2100). AAAI Press.
- Wang, Z., Schaul, T., Hessel, M., van Hasselt, H., Lanctot, M., & de Freitas, N. (2016). Dueling network architectures for deep reinforcement learning. En *Proceedings of the 33rd International Conference on Machine Learning* (pp. 1995–2003). PMLR.

## Anexo I. Evidencias de entrenamiento

Para facilitar la evaluación del aprendizaje del agente desde el inicio, a continuación se detalla la correspondencia de cada propuesta con sus directorios de pesos, archivos de log y la fase de entrenamiento.

Todas las trayectorias de aprendizaje mostradas en las gráficas de este documento parten desde cero (tras el periodo de warmup configurado en la memoria). Los logs recogen la evolución de las métricas desde los primeros steps útiles, demostrando el guiado del aprendizaje.

| Modelo / Propuesta              | Fase de Entrenamiento | Directorio de Pesos                  | Archivo de Log JSON       |
|---------------------------------|-----------------------|--------------------------------------|---------------------------|
| <b>P1.</b> DQN Vanilla          | Desde cero            | /checkpoints/p1_eps_fijo             | dqn_Enduro-v0_p1_log.json |
| <b>P2.</b> Annealed Rápido      | Desde cero            | /checkpoints/p2_annealed             | dqn_Enduro-v0_p2_log.json |
| <b>P3.</b> Boltzmann            | Desde cero            | /checkpoints/p3_boltzmann            | dqn_Enduro-v0_p3_log.json |
| <b>P4.</b> Double DQN           | Desde cero            | /checkpoints/p4_double_dqn           | dqn_Enduro-v0_p4_log.json |
| <b>P5.</b> Dueling DQN          | Desde cero            | /checkpoints/p5_dueling_dqn          | dqn_Enduro-v0_p5_log.json |
| <b>P6.</b> Double Dueling       | Desde cero            | /checkpoints/p6_double_dueling       | dqn_Enduro-v0_p6_log.json |
| <b>P7.</b> Double DQN Annealed  | Desde cero            | /checkpoints/p7_double_dqn_annealed  | dqn_Enduro-v0_p7_log.json |
| <b>P8.</b> Dueling DQN Annealed | Desde cero            | /checkpoints/p8_dueling_dqn_annealed | dqn_Enduro-v0_p8_log.json |